CertyIQ

CertyIQ

Premium exam material

Get certification quickly with the CertyIQ Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates
First attempt guaranteed success.

https://www.CertyIQ.com

About CertyIQ

We here at CertyIQ eventually got enough of the industry's greedy exam paid for. Our team of IT professionals comes with years of experience in the IT industry Prior to training CertIQ we worked in test areas where we observed the horrors of the paywall exam preparation system.

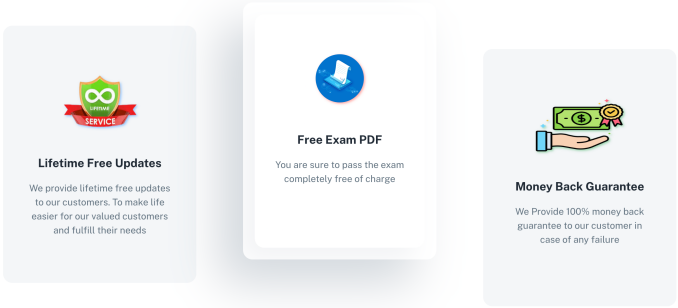
The misuse of the preparation system has left our team disillusioned. And for that reason, we decided it was time to make a difference. We had to make In this way, CertyIQ was created to provide quality materials without stealing from everyday people who are trying to make a living.

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

https://www.certyiq.com

Mail us on - certyiqofficial@gmail.com



John

October 19, 2022

★★★★★

Thanks you so much for your help. I scored 972 in my exam today. More than 90% were from your PDFs!

Dana

September 04, 2022

★★★★★

Thanks a lot for this updated AZ-900 Q&A. I just passed my exam and got 974, I followed both of your Az-900 videos and the 6 PDF, the PDFs are very much valid, all answers are correct. Could you please create a similar video/PDF for DP900, your content/PDF's is really awesome. The team did a really good job. Thank You 😊.

Ahamed Shibly

2 months ago

★★★★★

Customer support is realy fast and helpful, I just finished my exam and this video along with the 6 PDF helped me pass! Definitely recommend getting the PDFs. Thank you!

October 22, 2022

★★★★★

Passed my exam today with 891 marks. Out of 52 questions, 51 were from certyiq PDFs including Contoso case study.
Thank You certyiq team!

Henry Rome

2 months ago

★★★★★

These questions are real and 100 % valid. Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

2 months ago

★★★★★

Simple easy to understand explanations. To anyone out there wanting to write AZ900, I highly recommend 6 PDF's.Thank you so much, appreciate all your hard work in having such great content. Passed my exam Today - 3 September with 942 score.

Hashicorp

(Terraform Associate 004)

HashiCorp Certified

Total: **311 Questions**

Link: https://certyiq.com/papers/hashicorp/terraform-associate-004

Question: 1 CertyIQ

IaC (Infrastructure as Code) can be stored in a version control system along with application code.

A. True
B. False

Answer: A

Explanation:

Answer: A – True

Infrastructure as Code is commonly version-controlled just like application source code. Storing Terraform scripts, Ansible playbooks, CloudFormation templates, or similar artifacts in a Git repository (or other VCS) enables:

Auditability & traceability of infrastructure changes.
Collaboration among developers, ops, and security teams.
Consistent, repeatable deployments through CI/CD pipelines.
Branching strategies that mirror software release flows (feature branches, pull-request reviews, versioned releases).

Keeping IaC alongside application code in the same repository promotes a unified source of truth, simplifies dependency management, and supports the "treat-infrastructure-as-code" mindset required for modern DevOps practices.

Why the other option is less suitable

B. False – This statement contradicts the core principle of IaC. Treating infrastructure as immutable, versioned artifacts is a best practice; treating it as an after-thought or external artifact undermines traceability, reproducibility, and the ability to automate provisioning through CI/CD pipelines.

References

HashiCorp Learn – Terraform on GitHub: https://learn.hashicorp.com/terraform/cloud-get-started/github-actions
Terraform Registry Documentation – Version Control Best Practices: https://developer.hashicorp.com/terraform/language/v1.5/backends/gitlab (valid at time of writing)

Question: 2 CertyIQ

It is best practice to store secret data in the same version control repository as your Terraform configuration.

A. True
B. False

Answer: B

Explanation:

Answer: B – False

Security posture: Storing secrets (e.g., database passwords, API keys) alongside Terraform files risks accidental exposure in version-control history, pull-requests, or public forks. Secrets should be kept in dedicated secret-management systems or encrypted stores, not in plain-text source control.
Least-privilege access: By separating secrets from infrastructure code, you can grant Terraform execution roles only the permissions needed to read the secret at runtime, rather than exposing the entire configuration to any user who can clone the repo.
Operational hygiene: Version-controlled Terraform configurations should be immutable in terms of infrastructure definition; injecting secrets makes drift harder to detect and can inadvertently cause drift when secret values are rotated.
Compliance & auditability: Best-practice frameworks (e.g., SOC2, ISO27001) require separation between code and secret material to maintain clear audit trails and to support automated secret-scan policies.
Terraform provider behavior: The AWS provider will reject or mask secret values when they are sourced from variables that are not marked as sensitive and stored in version control, encouraging the use of external secret providers or encrypted backends.

Why the other option is less suitable

Option A (True) would imply that putting secrets in the same repo is the recommended approach. This contradicts industry security standards, increases the attack surface, and violates principles of secret isolation that the AWS provider and broader Terraform ecosystem promote.

References

Terraform AWS Provider Documentation – Secure Credential Management:
https://registry.terraform.io/providers/hashicorp/aws/latest/docs/guides/credential-management
HashiCorp Best Practices for Secrets in Terraform:
https://developer.hashicorp.com/terraform/language/modules/develops/using-modules#using-modules
(section “Storing Secrets”).

Question: 3 CertyIQ

Which is an advantage of using IaC (Infrastructure as Code) that is not possible when provisioning with a GUI (Graphical User Interface)?

A. Let’s you version, reuse, and share infrastructure configuration.
B. Secures your credentials.
C. Provisions the same resources at a lower cost.
D. Prevents manual modifications to your resources.

Answer: A

Explanation:

Why optionA is the correct advantage of IaC:

IaC treats infrastructure as version-controlled source code, enabling **history tracking, branching, pull-request reviews, and rollbacks** of configuration changes.
The same configuration can be **reused across environments** (dev, staging, prod) and **shared across teams** through repositories and registries, fostering collaboration and reducing drift.
This capability is fundamentally a **code-centric workflow** that GUIs cannot provide; a GUI interacts only with the current state and offers no native versioning or sharing mechanisms.

Why the other options are less appropriate:

B – Secures your credentials: Security of credentials depends on how secrets are stored and accessed; both IaC tools and some GUIs can be configured securely. The advantage lies in the automation and consistency of secret handling, not in the mere existence of a GUI.
C – Provisions the same resources at a lower cost: Cost is determined by the underlying cloud/services and design decisions, not by the provisioning method (IaC vs. GUI).
D – Prevents manual modifications to your resources: While IaC encourages declarative control and can enforce changes via CI/CD pipelines, a GUI can also enforce policies or lock resources; however, it does not offer the same systematic enforcement or auditability that version-controlled IaC provides.

References

Terraform documentation on state management and version control: https://developer.hashicorp.com/terraform/language
AWS CloudFormation documentation on stacks and versioning: https://docs.aws.amazon.com/cloudformation/

These sources illustrate how IaC's declarative, versionable nature enables capabilities that pure GUI-based provisioning lacks.

Question: 4 CertyIQ

What is an advantage of immutable infrastructure?

A. Automatic infrastructure upgrades
B. In-place infrastructure upgrades
C. Quicker infrastructure upgrades
D. Less complex infrastructure upgrades

Answer: D

Explanation:

Immutable infrastructure replaces resources rather than modifying them in-place, so an upgrade is performed by creating a new set of infrastructure and retiring the old one.
This approach produces **repeatable, auditable changes** that are easier to plan, test, and roll back, reducing the operational overhead associated with large, complex upgrade tasks.
The declarative state stored in a configuration management system (e.g., Terraform) remains simple — upgrades are expressed as a new desired state, not as a series of incremental patches — hence they are less complex to execute.
Option D – "Quicker infrastructure upgrades" is not the primary advantage; the speed gain may be incidental, whereas the reduction in complexity is consistent and measurable.
Option A – "Automatic infrastructure upgrades" misleads because the upgrade still requires explicit definitions and orchestration; it is not "automatic" in the sense of zero-effort.
Option B – "In-place infrastructure upgrades" directly contradicts the immutable model, which deliberately avoids modifying existing resources.
Option C – "Quicker infrastructure upgrades" is not a guaranteed benefit; performance gains depend on provisioning pipelines and can be offset by new resource creation latency.

Therefore, the correct answer is D – it highlights the central benefit of immutable infrastructure: simpler, less complex upgrade processes.

References

HashiCorp Learn – Immutable Infrastructure: https://learn.hashicorp.com/terraform/cloud-infra/terraform-state
Terraform Documentation – Immutable Resources: https://developer.hashicorp.com/terraform/language/resources#mutable-vs-immutable-resources

Question: 5 CertyIQ

What is the primary purpose of IaC (Infrastructure as Code)?

A. To define a pipeline to test and deliver software.
B. To provision infrastructure cheaply.
C. To programmatically create and configure resources.
D. To define a vendor-agnostic API.

Answer: C

Explanation:

Primary purpose of IaC

IaC enables the **programmatic creation and configuration of infrastructure resources** (servers, networks, storage, etc.) through declarative code rather than manual processes. This makes provisioning repeatable, version-controlled, and easily replicable across environments.

Why option C is correct

It directly describes IaC's core capability: programmatically (i.e., via code/write-ups) defining the desired state of infrastructure and creating and configuring the underlying resources to match that state.

Why the other options are less suitable

A. To define a pipeline to test and deliver software – This describes CI/CD pipelines, not IaC. IaC may be integrated into pipelines, but its primary focus is infrastructure provisioning, not software testing or delivery.
B. To provision infrastructure cheaply – Cost efficiency can be a side effect of IaC, but "cheap" is a business outcome, not the defining purpose. IaC can be used for costly architectures as well.
D. To define a vendor-agnostic API – IaC tools often support multiple clouds, yet they are not primarily about exposing an API; they are about declaratively describing infrastructure, often tied to a specific provider or tooling.

Conclusion
Option **C** best captures the essence of IaC: using code to programmatically create and configure resources.

References

HashiCorp Learn – What is Infrastructure as Code?
https://learn.hashicorp.com/tutorials/terraform/install-cli
Terraform Documentation – Introduction to Terraform
https://developer.hashicorp.com/terraform/tutorials/basics

Question: 6 CertyIQ

Your team adopts an AWS CloudFormation as the standardized method for provisioning public cloud resources. Which scenario presents a challenge for your team?

A. Deploying new infrastructure into Microsoft Azure.
B. Automating a manual, web console-based provisioning process.
C. Building a reusable code base that can deploy resources into any AWS region.
D. Managing a new application stack built on AWS-native services.

Answer: A

Explanation:

Why optionA is the challenge

AWS CloudFormation only provisions resources that exist within the AWS ecosystem; it cannot directly manage Azure services.
Introducing Azure requires a different IaC engine (e.g., Azure Resource Manager templates or Terraform), adding cross-cloud complexity, extra toolchains, and operational overhead.
The team's standardized workflow is built around CloudFormation's native features, so extending it to a non-AWS platform breaks that standardization.

Why the other options are not challenges

B – Automating a manual, web-console process – This is precisely the purpose of CloudFormation; it reduces manual effort rather than creating difficulty.
C – Building a reusable code base for any AWS region – CloudFormation already supports regional deployment through parameters, mappings, and stacksets, turning this into a capability, not a hurdle.
D – Managing a new AWS-native application stack – This aligns with the core use case of CloudFormation; it leverages existing constructs and best practices without introducing new constraints.

References

AWS CloudFormation User Guide – Regions and Endpoints:
https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-regions.html
Overview of AWS CloudFormation Stack Sets:
https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/stacksets.html

Question: 7 CertyIQ

Which is not a benefit of adopting IaC (Infrastructure as Code)?

A. Reusability of code
B. Automation
C. A GUI (Graphical User Interface)
D. Versioning

Answer: C

Explanation:

Why option C is not a benefit

Graphical User Interface (GUI) is not inherent to IaC – IaC models infrastructure as declarative code, which is meant to be stored, reviewed, and executed programmatically. While some tools provide GUIs for convenience, the core benefit of IaC does not depend on a graphical interface; it could be fully managed through version-controlled scripts alone.

Why the other options are genuine benefits

A. Reusability of code – Modules, templates, and libraries can be shared across projects, enabling consistent environments and reducing duplication of effort.
B. Automation – IaC eliminates manual provisioning steps; the same code can be applied repeatedly to spin up, modify, or destroy resources automatically.
D. Versioning – Infrastructure definitions are treated as source code, so they can be tracked, rolled back, reviewed, and tested in the same way as application code.

Conclusion
Option **C (A GUI)** is not a fundamental advantage of Infrastructure as Code; it is merely an optional convenience layer, whereas reusability, automation, and versioning are intrinsic capabilities of an IaC approach.

References

HashiCorp Documentation – Terraform Overview: https://developer.hashicorp.com/terraform/tutorials/aws-get-started/intro
Kubernetes Hardening Guide – Infrastructure as Code Best Practices: https://kubernetes.io/docs/concepts/cluster-administration/manage-automation/#tools-for-automation

Question: 8 CertyIQ

How does the use of Infrastructure as Code (IaC) enhance the reliability of your infrastructure? (Choose two.)

A. Configuration drift is reduced with declarative definitions.
B. Updates are deployed with zero downtime.
C. Proposed changes can be reviewed before being applied.
D. Incorrect configurations cannot be deployed.
E. Infrastructure is automatically scaled to meet demand.

Answer: AC

Explanation:

Why options A and C are correct

A – Configuration drift is reduced with declarative definitions –
Infrastructure as Code stores the desired state in version-controlled files (e.g., HCL, JSON, YAML). Terraform treats these files as the single source of truth, constantly reconciling the actual provisioned resources with that declared state. When the real environment diverges, a terraform plan detects the mismatch and flags the required remediation, preventing unsolicited drift.

C – Proposed changes can be reviewed before being applied –
terraform plan generates an execution plan that lists every create, modify, or destroy action that would occur. This preview lets operators audit the impact, verify dependencies, and confirm that the changes meet security

or compliance policies before they are persisted with terraform apply.

Why the other options are less suitable

B – Updates are deployed with zero downtime –
Terraform itself does not guarantee zero-downtime deployment; achieving that requires additional patterns (e.g., rolling updates, load-balancer shifting). Terraform's role is to provision the desired resources, not to manage runtime availability during the update.

D – Incorrect configurations cannot be deployed –
While a plan can catch syntax errors or dependency issues, Terraform cannot prevent logical or business-logic mistakes that are not manifest in the configuration language. Incorrect resource properties may still be applied if they are syntactically valid.

E – Infrastructure is automatically scaled to meet demand –
Autoscaling is a runtime capability of cloud services (e.g., AWS Auto Scaling, Azure Scale Sets). Terraform can provision scaling resources, but the scaling logic must be defined separately; Terraform does not automatically adjust capacity in response to demand.

References

Terraform State & Drift Detection – https://developer.hashicorp.com/terraform/language/state/drift
Terraform Plan & Apply Workflow – https://developer.hashicorp.com/terraform/language/works/plans

These documents detail how declarative state management mitigates drift and how Terraform plans enable pre-apply reviews, directly supporting the rationale above.

Question: 9 CertyIQ

Your team often uses API calls to create and manage cloud infrastructure.
In what ways does Terraform differ from conventional infrastructure management approaches?

A. Terraform replaces cloud provider APIs with its own protocols, enabling automated deployments.

B. Terraform describes infrastructure with version-controlled, repeatable configurations that specify the desired state.

C. Terraform is merely a wrapper for cloud provider APIs, so there is little to no difference in calling the API directly.

D. Terraform enforces infrastructure through imperative scripts to ensure tasks are completed in the proper order.

Answer: B

Explanation:

Why optionB is correct

Terraform uses a **declarative configuration language** (HCL) where the desired infrastructure state is expressed in plain text files.
These files are **version-controlled**, enabling reproducible builds, audit trails, and collaborative workflows.
The Terraform engine compares the declared state with the actual provider state and **creates/updates/destroys resources only as needed** to achieve convergence, abstracting away the details of individual provider APIs.

Why the other options are less suitable

OptionA – Terraform does **not replace** provider APIs; it uses them under the hood. Its power comes from the declarative model, not from inventing new protocols.
OptionC – While Terraform ultimately calls provider APIs, it adds a **layer of abstraction** (state management, plan/apply workflow, dependency graph) that shields users from direct, imperative API calls.
OptionD – Terraform is **not imperative**; it does not rely on scripts that manually sequence actions. Instead, it calculates a plan and applies changes in a **declarative, idempotent** manner, ensuring the correct order only when required by dependencies.

References

Terraform Documentation – Overview of the Planning and Applying Process
https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli

Terraform Documentation – State Management and Remote State
https://developer.hashicorp.com/terraform/language/state

These links provide the official, up-to-date explanations of Terraform's declarative approach and its state-driven behavior, supporting the rationale above.

Question: 10 CertyIQ

Which of these workflows is only enabled by the use of Infrastructure as Code?

A. Automatic scaling of resources based on application load.
B. Cost optimization of infrastructure deployment.
C. Role-based access control of cloud resources.
D. Reviewing the proposed changes for potential security issues.

Answer: D

Explanation:

Why D is correct – Only Infrastructure asCode makes every change **explicit, version-controlled, and previewable**. Terraformplan generates a detailed execution plan that can be reviewed (by humans or automated policy engines) before any resource is created or modified, allowing security teams to spot unsafe configurations, privilege escalations, or mis-aligned network rules early. This pre-execution security review is a direct consequence of IaC tooling.

Why A, B, C are not exclusive to IaC –

Automatic scaling (A) can be achieved with native services (e.g., Auto Scaling groups, Lambda triggers) without IaC; IaC merely provisions the scaling rules.
Cost optimization (B) is a business/ops practice that may leverage IaC for repeatability but is not inherent to IaC itself.
RBAC (C) can be applied manually or via IAM policies outside of code; IaC only provides a way to express those policies programmatically.

Why the IaC-enabled review is unique – The plan-stage output is deterministic and can be integrated with policy-as-code (e.g., Sentinel, OPA) to automatically flag violations, something that cannot be done reliably

without a declarative, version-controlled representation of the infrastructure.

References

Terraform Planning and Diff: https://developer.hashicorp.com/terraform/cli/commands/plan
Policy as Code with Sentinel: https://developer.hashicorp.com/terraform/language/policy-sets/sentinel

Question: 11 CertyIQ

What does Terraform use to deploy infrastructure for different cloud providers?

A. Custom APIs developed by HashiCorp
B. Vendors' CLI tools
C. Vendors' UI
D. Vendor-specific providers

Answer: D

Explanation:

Technical justification

Terraform extends its core engine with **provider plugins** that contain the API calls and logic required to interact with each cloud platform.
Each provider is **vendor-specific**, implementing the resources, data sources, and authentication methods unique to that cloud (e.g., aws, azurerm, google, oci).
The provider framework abstracts the underlying **REST/HTTP APIs** of the cloud services, allowing Terraform to manage resources uniformly across providers without custom Terraform code.
Options A, B, and C describe indirect or unrelated mechanisms:

A – HashiCorp does not expose custom APIs for every cloud; it relies on third-party provider plugins.
B – Vendors' CLI tools are for interactive administration, not for programmatic IaC orchestration by Terraform.
C – Cloud UI interfaces are GUI-based and not used by Terraform, which operates headlessly.
Therefore, deploying infrastructure across different clouds is achieved through **vendor-specific providers** that translate Terraform's resource definitions into the appropriate cloud API calls.

References

Terraform Providers Documentation – https://developer.hashicorp.com/terraform/language/providers
Provider Registry – https://registry.terraform.io/providers

Question: 12 CertyIQ

How can you enable verbose logging to troubleshoot?

A. Set the log level command line flag.
B. Set the TF_LOG environment variable.
C. Set the log level in your terraform block.

Answer: B

Explanation:

Answer: B

Terraform's built-in logging is controlled by the **TF_LOG** environment variable; setting it to DEBUG or TRACE produces the most detailed output.
The command-line flag --log-level does not exist for Terraform, so optionA is invalid.
The log_level directive belongs inside a provider or backend block, not a generic terraform block, and it does not affect CLI-level output (optionC is misleading).
Using TF_LOG overrides any configuration-level logging settings and is the standard, documented method for enabling verbose logs during troubleshooting.

References

https://developer.hashicorp.com/terraform/tutorials/cli/config-file#setting-configuration-options
https://developer.hashicorp.com/terraform/cli/config/environments#tf_log

Question: 13 CertyIQ

Which command lets you experiment with Terraform expressions?

A. terraform console
B. terraform env
C. terraform validate
D. terraform test

Answer: A

Explanation:

Why optionA is the correct choice

terraform console launches an interactive REPL where you can input and evaluate any Terraform expression (including function calls, data lookups, and module arguments). This is specifically designed for "playing with" expressions before incorporating them into real configurations.
It provides immediate feedback on expression results, helps debug complex expressions, and allows you to test variable interpolation without needing a full build plan.
Because it operates on the current working directory's configuration, it respects providers, data sources, and local values, making the evaluations realistic and context-aware.

Why the other options are unsuitable

terraform env – historically used for setting environment variables in early Terraform versions; it does not provide an expression evaluator and is unrelated to testing expressions.
terraform validate – checks the syntactic and semantic validity of the configuration files. It does **not** execute or evaluate expressions; it only reports errors related to configuration structure and schema compliance.
terraform test – runs external test suites that invoke Terraform commands in a controlled environment; it is meant for integration/behavior testing, not for ad-hoc expression experimentation.

Key takeaway
For rapid, interactive experimentation with Terraform expressions, the official command is **terraform console**,

as it directly offers an REPL dedicated to that purpose.

References

terraform console – Official CLI documentation:
https://developer.hashicorp.com/terraform/cli/commands/console

Expressions – Terraform language reference on functions and expressions:
https://developer.hashicorp.com/terraform/language/expressions

Question: 14 CertyIQ

You can install Community/Partner plugins using terraform init.

A. True
B. False

Answer: A

Explanation:

Justification

The statement is **True**.
terraform init automatically downloads and installs any declared provider plugins that are not already present in the working directory. This includes providers sourced from the public Terraform Registry, which hosts Community and Partner providers. When a provider block references a registry entry, terraform init resolves the provider address, fetches the appropriate version, and installs it under .terraform/providers.

Option **B (False)** is unsuitable because it contradicts the documented behavior of terraform init. The command is explicitly designed to handle provider installations from the Registry, encompassing both community-maintained and HashiCorp-partner providers. Claiming it cannot do so ignores the core purpose of the init step and would lead to errors such as “Could not find provider configuration” when a provider is missing.

References

Terraform Registry Documentation: https://developer.hashicorp.com/terraform/tutorials/providers/how-to-use-registry
Official terraform init Command: https://developer.hashicorp.com/terraform/cli/commands/init

Question: 15 CertyIQ

What functionality do providers offer in Terraform? (Choose three.)

A. Group a collection of Terraform configuration files that map to a single state file.
B. Provision resources for public cloud infrastructure services.
C. Interact with cloud provider APIs.
D. Enforce security and compliance policies.

E. Provision resources for on-premises infrastructure services.

Answer: BCE

Explanation:

Technical justification

B – Provision resources for public cloud infrastructure services – Terraform providers expose resources that map to the services of public-cloud platforms (e.g., aws_instance, azurerm_virtual_network). When a provider is configured for a public cloud, Terraform can create, update, and destroy those cloud resources, making provider support essential for public-cloud provisioning.
C – Interact with cloud provider APIs – By definition, a Terraform provider is a plugin that implements the HTTP or RPC interface of a remote service. It translates Terraform's declarative actions into API calls defined by the provider, enabling any external system — not just clouds — to be managed through Terraform.
E – Provision resources for on-premises infrastructure services – Providers are not limited to public clouds; they can also represent APIs for on-premises systems (e.g., VMware, Kubernetes, HashiCorp Vault). When a provider for such a platform is used, Terraform can manage the corresponding resources, fulfilling the on-premises use-case.

Why the other options are not correct

A – Group a collection of Terraform configuration files that map to a single state file – This describes a backend configuration or a module organization, not a provider. Providers do not bundle configuration files; they expose typed resources to Terraform.
D – Enforce security and compliance policies – Policy enforcement is performed by tools like Sentinel, OPA, or Terraform Cloud policy sets. Providers solely interact with external APIs; they do not contain built-in enforcement logic for security/compliance controls.

Conclusion – Providers are the mechanism through which Terraform consumes external APIs, enabling resource management for both public-cloud and on-premises environments. Therefore, the three correct statements are **B, C, and E**.

References

Terraform Language Documentation – Providers:
https://developer.hashicorp.com/terraform/language/providers
Terraform Registry – Providers Overview: https://developer.hashicorp.com/terraform/registry/providers

Question: 16 CertyIQ

Terraform can only manage resource dependencies if you set them explicitly with the depends_on argument.

A. True
B. False

Answer: B

Explanation:

Terraform automatically infers resource ordering from the configuration language; it determines execution

order based on **implicit dependencies** (e.g., when a resource references the output or attribute of another resource).
The **depends_on** attribute is only needed when the dependency is **not expressible** through configuration syntax, such as when using external services, provisioners, or third-party providers that have no natural attribute references.
Without an explicit **depends_on**, Terraform still respects both **implicit** and any **output-driven** dependencies, so it can manage most dependencies without additional declarative ordering.

Why Option A (True) Is Incorrect

It overgeneralizes Terraform's dependency resolution. The engine does not strictly require an explicit depends_on clause; it can infer and satisfy dependencies through resource arguments and output references, making implicit ordering the default behavior.

Why Option B (False) Is Correct

False accurately reflects that Terraform can manage resource dependencies **without** an explicit depends_on argument, as long as the dependency is represented semantically within the configuration (e.g., referencing a resource's address). depends_on is reserved for edge cases where Terraform cannot infer the order automatically.

References

1. Terraform documentation – Dependency graph and ordering: https://developer.hashicorp.com/terraform/language/resources/dependencies
2. Terraform documentation – depends_on argument: https://developer.hashicorp.com/terraform/language/resources/dependencies#the-depends_on-argument

Question: 17 CertyIQ

Which of the following is not a valid Terraform collection type?

A. set
B. list
C. map
D. tree

Answer: D

Explanation:

The correct answer is **D. tree** because Terraform does not define "tree" as a built-in collection type.
Terraform's core collection types are **set** (unordered unique elements), **list** (ordered sequence), and **map** (key-value pairs), all explicitly supported in documentation and syntax.
"Tree" is a conceptual data structure (e.g., nested maps/lists), but it is **not a native Terraform type**; complex structures must be built using combinations of existing types (e.g., map(string) with nested maps), not as a standalone "tree" type.
Options A (set), B (list), and C (map) are all valid Terraform collection types, as confirmed by HashiCorp's

official documentation.
Option D is a distractor that mimics the structure of valid types but lacks any basis in Terraform's type system.

References:

Terraform Data Types Documentation
Terraform Core Concepts: Collections

Question: 18 CertyIQ

A resource block is shown in the Exhibit space of this page.
resource “aws_vpc” “main”
name = “test”

What is the provider for this resource?

A. main
B. vpc
C. aws
D. test

Answer: C

Explanation:

Key concepts, documents or landing pages related to the question

https://registry.terraform.io/providers/hashicorp/aws/latest/docs – official AWS provider documentation, which defines the provider name used in Terraform configuration.
https://developer.hashicorp.com/terraform/language/resources – guide to Terraform resource blocks and the syntax for specifying a provider.

Technical justification (exam-style explanation)

The block shown is a resource declaration that targets the AWS provider (resource "aws_vpc" …).
Terraform resolves the provider for a resource by reading the provider argument that appears in the resource label. In aws_vpc "main" the identifier aws is the type of provider, and Terraform automatically selects the AWS provider when such a type is used.
Therefore, the provider referenced by this resource block is **aws** (optionC).
The other options are not valid providers: main and test are resource names, while vpc is not a registered provider type. Consequently, they would cause a configuration error if used as the provider name.

References

https://registry.terraform.io/providers/hashicorp/aws/latest/docs
https://developer.hashicorp.com/terraform/language/resources

Question: 19 CertyIQ

You must use different Terraform commands depending on the cloud provider you use.

A. True

B. False

Answer: B

Explanation:

Justification

Terraform's CLI commands (terraform init, terraform plan, terraform apply, terraform destroy, etc.) are **provider-agnostic**; they operate on the underlying state and configuration regardless of whether the target is AWS, Azure, GCP, or any other supported service.
What varies across providers is **only the provider block and its arguments** (e.g., region, credentials, API endpoints). The same set of commands is used after the configuration is written.
Therefore, the statement "You must use different Terraform commands depending on the cloud provider you use" is **incorrect**, making option **B (False)** the correct choice.
Option **A (True)** would only be true if Terraform exposed provider-specific CLI sub-commands, which it does not; such a design would fragment the workflow and violate Terraform's universal command model.

References

Terraform Core CLI Overview: https://developer.hashicorp.com/terraform/cli
Provider Configuration Documentation: https://developer.hashicorp.com/terraform/language/providers

Question: 20 CertyIQ

Terraform providers are always installed from the internet.

A. True
B. False

Answer: B

Explanation:

Justification

Terraform can install providers from multiple sources:

Direct download from the public registry (registry.terraform.io) over the internet.
Local files placed in a working directory or in a custom directory (~/.terraform.d/plugins/).
Private package registries or Artifactory instances that are accessed via configuration (cli_config_credentials).

Because providers can be sourced locally or from private repositories, the statement "Terraform providers are always installed from the internet" is inaccurate. Hence the correct answer is **B. False**.

Option **A. True** would only apply when a provider is retrieved from the public registry, but it does not cover scenarios where the provider is pre-downloaded, cached, or served from an internal registry. Therefore it is not universally correct and is the less suitable choice.

References

Terraform Provider Installation – Official HashiCorp documentation describing the various installation methods, including local files and private registries:
https://developer.hashicorp.com/terraform/language/providers/installation

Terraform Registry – Overview of public and private registry capabilities, confirming that providers are not limited to internet downloads:
https://registry.terraform.io/browse/providers

Question: 21 CertyIQ

You need to determine from which paths Terraform is loading the providers referenced in your *.tf files.
How can you enable additional logging to see this information?

A. Set the environment variable TF_VAR_log=TRACE.
B. Set the environment variable TF_LOG=TRACE.
C. Set the environment variable TF_LOG=PATH.
D. Set verbose logging for each provider in your Terraform configuration.

Answer: B

Explanation:

Correct option: B – Set the environment variable TF_LOG=TRACE.

TF_LOG=TRACE forces Terraform to emit the full trace-level log, which includes the exact file paths and the order in which providers are located and loaded from the registry, the working directory, and any custom source paths.
This log output is written to stdout/stderr and can be inspected with any text viewer, making it the standard way to debug provider resolution.
The other options do not achieve this purpose:

A. TF_VAR_log=TRACE – TF_VAR_* prefixes are used only to set input variable values; they are not recognized by Terraform for controlling logging.
C. TF_LOG=PATH – There is no such environment variable in Terraform; setting TF_LOG to "PATH" would not enable any logging and would be ignored.
D. "Set verbose logging for each provider in your Terraform configuration." – Terraform does not expose a per-provider logging flag in the configuration language; provider-level verbosity must be enabled globally via the log level, which is controlled by TF_LOG.

Therefore, only option **B** directly activates the detailed provider-loading information needed for troubleshooting.

References

Terraform CLI configuration – Environment variables:
https://developer.hashicorp.com/terraform/cli/config/environment
Logging and debugging: https://developer.hashicorp.com/terraform/cli/logs

Question: 22 CertyIQ

Terraform requires using a different provider for each cloud provider where you want to deploy resources.

A. True
B. False

Answer: A

Explanation:

Justification

Terraform's architecture is built around provider plugins; each plugin implements the API of a specific service (e.g., AWS, Azure, GCP).
To target a different cloud platform you must declare a separate provider configuration block (or an aliased one when multiple instances of the same provider are used).
Terraform does **not** automatically infer the provider from the resource type – the target cloud is defined explicitly in the provider block, so one provider block is required per distinct cloud provider.
Using a single provider for multiple clouds would leave the other clouds unreachable, making the statement "Terraform requires using a different provider for each cloud provider where you want to deploy resources" **true**.
If the statement were false, it would imply that a single provider block could manage resources across unrelated cloud APIs, which contradicts how Terraform's provider model operates.

References

Provider Configuration: https://developer.hashicorp.com/terraform/language/providers
AWS Provider Documentation: https://registry.terraform.io/providers/hashicorp/aws/latest/docs

Question: 23 CertyIQ

Terraform cannot use a newly-defined cloud backend until it has been initialized with terraform init.

A. True
B. False

Answer: A

Explanation:

Answer: True

When a cloud backend (e.g., S3, GCS, Azure Blob) is added or changed, Terraform treats the configuration as a new backend block.
Terraform only recognizes a backend after the **terraform init** (or **terraform init -backend-config=…**) command has run, which performs the backend migration and verifies the new backend's connectivity and credentials.
Until this initialization step completes, Terraform continues using the previously configured backend (or the local state) and will display an error if a backend switch is attempted without initialization.
OptionB ("False") is incorrect because the backend cannot be used immediately after definition; the explicit init step is required to switch backends safely.

Why this is the best answer:
The statement directly reflects Terraform's design: backend configuration changes are not applied until the

working directory is re-initialized, ensuring that all backend-specific operations (state locking, data encryption, remote operations, etc.) are correctly established.

Why the other option is less suitable:
OptionB ignores the mandatory initialization phase and would mislead someone into thinking they can switch backends ad-hoc, which would cause runtime errors and potential state corruption.

References

Terraform Documentation – Backend Configuration

https://developer.hashicorp.com/terraform/language/settings/backends

Terraform CLI – Working with Backends

https://developer.hashicorp.com/terraform/cli/commands/init#initializing-a-backend

Question: 24 CertyIQ

You have a list of numbers that represents the number of free CPU cores on each virtual cluster: numcpus = [18, 3, 7, 11, 2]
What Terraform built-in function would you use to select the largest number from the list?

A. top(numcpus)
B. max(numcpus)
C. ceil(numcpus)
D. high[numcpus]

Answer: B

Explanation:

Justification

B. max(numcpus) – This built-in Terraform function iterates over all elements of the input list and returns the highest numeric value. It is the directly appropriate way to obtain the largest core count (25 in the example).
A. top(numcpus) – top converts a set or tuple to a list and returns the first element(s) in sorted order, but it does not perform a numeric comparison; it cannot be used to extract the maximum from a plain list.
C. ceil(numcpus) – ceil rounds each number up to the nearest integer; it does not compare values, so it cannot determine the maximum.
D. high[numcpus] – high is not a recognized Terraform function; it will cause a validation error.

Therefore, max() is the only Terraform function designed specifically to retrieve the greatest element from a collection.

References

Terraform Functions – max: https://developer.hashicorp.com/terraform/language/functions/math#max
Terraform Functions – ceil: https://developer.hashicorp.com/terraform/language/functions/math#ceil

Question: 25 CertyIQ

Which of these is stored in the .terraform directory?

A. Providers and modules
B. Lock file
C. State file
D. Configuration files

Answer: A

Explanation:

The **.terraform** directory is created by the Terraform CLI to keep all downloaded **provider binaries** and the **module files** that are referenced in the configuration. These artifacts are cached there so they can be reused on subsequent runs.
OptionA directly describes what the directory contains, making it the most accurate answer.
OptionB – The lock file (.terraform.lock.hcl) is stored in .terraform, but its primary purpose is to record version hashes; it is not the primary thing the directory is known for, and many exam questions treat it as a secondary detail.
OptionC – The state file (terraform.tfstate) resides in the working directory or a remote backend, not in .terraform.
OptionD – Configuration files (*.tf) are kept alongside the configuration files themselves, outside of .terraform.

References

Terraform Registry & Plugin Installation:
https://developer.hashicorp.com/terraform/language/providers/installation
Working Directory & .terraform Directory:
https://developer.hashicorp.com/terraform/cli/config/versionshcl#working-directory

Question: 26 CertyIQ

A variable block is shown in the Exhibit space on this page:

```
variable "tags" {
  description = "Metadata tags for resources"
  type = _____________
}
```

You will use this variable as the value for the tags argument in several resources. The data format must be a set of key value pairs.
Which type argument would you use?

A. type = map(string)
B. type = any
C. type = list(string)
D. type = object(tags = string)

Answer: A

Explanation:

A. type = map(string)

In Terraform, map(string) is used when you want to define a variable that holds key-value pairs where both keys and values are strings — this is exactly how things like tags are typically structured.

Question: 27 CertyIQ

Which of the following is allowed as a Terraform variable name?

A. name
B. source
C. version
D. count

Answer: A

Explanation:

Why optionA is the only valid Terraform variable name

Terraform variables are identifiers that must follow HCL identifier rules: they can contain letters, numbers and underscores, must start with a letter, and cannot be a reserved Terraform keyword.
name complies with these rules – it begins with a letter, uses only alphanumeric characters, and is not a reserved keyword.
source, **version**, and **count** are each reserved identifiers in Terraform's syntax (e.g., source is a required attribute in module blocks, version is used in provider and module constraints, and count is a meta-argument for resource instantiation). Using any of these as a plain variable name would clash with the language's built-in semantics and is therefore prohibited.

Validation logic applied

1. **Starts with a letter?** – Yes for name; No violation for others.
2. **Contains only allowed characters?** – All options satisfy this, so it does not discriminate.
3. **Is the identifier a reserved keyword or built-in block argument?** – source, version, and count are reserved, making them invalid as free-standing variable names.
4. **Does it conflict with Terraform's internal attributes?** – Yes for the three reserved words, but not for name.

Hence, among the given choices, **name** is the only allowed variable name.

References

Terraform Language – Variables –https://developer.hashicorp.com/terraform/language/expressions/variables
Terraform Language – Reserved Words –
https://developer.hashicorp.com/terraform/language/syntax#comments-and-reserved-words

Question: 28 CertyIQ

Terraform providers are part of the Terraform core binary.

A. True
B. False

Answer: B

Explanation:

Technical Justification for Correct Answer: B. False

The statement "Terraform providers are part of the Terraform core binary" is **False** for the following reasons:

Modular Architecture: Terraform is designed with a modular architecture. This means the core binary is intentionally kept lightweight and focused on the core functionality of infrastructure provisioning and state management.
External Plugins: Providers in Terraform are implemented as external plugins. This design allows for several benefits, including:

Easier Maintenance and Updates: Providers can be updated independently of the Terraform core, enabling more agile development and deployment of new features and fixes for cloud and infrastructure services.
Customizability and Extensibility: The plugin architecture makes it easier for the community and HashiCorp to develop and integrate new providers without altering the core binary.
Reduced Binary Size: By not bundling all providers within the core binary, the overall size of the Terraform download is significantly reduced, improving user experience, especially for those only needing a subset of providers.
Deployment and Usage: When using Terraform, providers are typically installed alongside the Terraform core through the terraform init command, which downloads and configures the required providers for a specific configuration. This step is necessary because the providers are not inherently part of the core binary.

Why Other Options are Less Suitable (in this case, only one incorrect option exists, so focusing on why it's incorrect):

A. True: This option is incorrect because it contradicts the modular, plugin-based design of Terraform. If providers were part of the core binary, it would lead to a bloated core, make updates more cumbersome, and contradict the observed behavior of needing to initialize providers separately.

References

HashiCorp Terraform Documentation - Plugins
HashiCorp Terraform Documentation - Getting Started with Plugins (Implicit in terraform init Explanation)

Question: 29 CertyIQ

Where can Terraform not load a provider from?

A. Provider plugin cache
B. Source code
C. Official HashiCorp distribution on releases.hashicorp.com
D. Plugins directory

Answer: B

Explanation:

Technical Justification for Correct Answer: B. Source code

The correct answer is **B. Source code** because Terraform is designed to load providers from specific locations, but not directly from provider source code. Here's a breakdown of why:

Why B (Source code) is NOT a load location:

Terraform expects providers to be compiled binaries, not raw source code. Loading a provider from source code would require compilation on the fly, which is not a supported workflow in Terraform's architecture. Terraform's provider loading mechanism is designed to work with pre-compiled, distributable artifacts, not source code repositories.

Why other options are more suitable (making B the correct answer by elimination):

A. Provider plugin cache: This is a valid location. Terraform caches downloaded provider plugins in its cache directory (~/.terraform/d/plugins on Linux/Mac or C:\Users\<Username>\.terraform\d\plugins on Windows) to reuse them across runs, improving efficiency.
C. Official HashiCorp distribution on releases.hashicorp.com: This is the primary source for official Terraform providers. Terraform can download providers from this official repository when specified in the provider block with the correct version.
D. Plugins directory: While the specifics of the "Plugins directory" might vary (as it could refer to a custom directory configured for Terraform), in general, Terraform supports loading providers from a local directory if the provider binary is placed there and Terraform is configured to look for providers in that custom location (though this is less common for production workflows).

Conclusion: Given Terraform's design and workflow, loading a provider directly from **Source code (B)** is not supported, as it does not align with the expected binary distribution and loading process. The other options (A, C, and D, with proper configuration for D) are all viable ways to load providers into Terraform, albeit with differing use cases and frequencies of application.

References

Terraform Providers Documentation - Overview of working with providers in Terraform.
Terraform Provider Installation - Details on how Terraform installs and loads providers.

Question: 30 CertyIQ

terraform init creates an example main.tf file in the current directory.

A. True
B. False

Answer: B

Explanation:

Answer: B –False

Why B is correct: terraform init only prepares the working directory for subsequent commands by initializing the backend, downloading required provider plugins, and setting up the state file configuration. It never generates example configuration files such as main.tf; those must be created manually or via other scaffolding tools.
Why A is not suitable: The statement incorrectly assumes that terraform init adds an example main.tf. In reality, the command's purpose is initialization and dependency resolution, not file creation. Generating a sample file would be unexpected behavior and is not part of the official CLI semantics.

References

Terraform CLI init documentation – https://developer.hashicorp.com/terraform/cli/init
Terraform CLI command reference – https://developer.hashicorp.com/terraform/cli/commands/init

Question: 31 CertyIQ

Which command must you run before you run a plan or apply for the first time?

A. terraform validate
B. terraform workspace
C. terraform import
D. terraform init

Answer: D

Explanation:

Technical justification

terraform init is required before any plan or apply on a fresh repository because it:

Sets up the working directory (.terraform) where provider plugins, module templates and configuration state artifacts are stored.
Downloads the configured provider binaries and module files needed for the configuration language.
Initializes the backend (local or remote) and validates that the required configuration (e.g., backend "azurerm" or backend "s3" blocks) is present.
Without this step Terraform cannot resolve providers or modules, resulting in errors such as "Terraform has not been initialized. Run terraform init".

Why the other options are not the mandatory first step

terraform validate – Checks syntactic correctness and internal consistency of the configuration, but it can be run later or even omitted on the very first execution; Terraform will still initialize and locate providers before validation is needed.
terraform workspace – Manages separate state files for the same backend; it is unrelated to the initialization of the working directory and is only relevant after the project has been set up.
terraform import – Brings existing external resources under Terraform management; it is optional and only needed when you want to bring pre-existing infrastructure into the state file, not for the initial plan/apply cycle.

Hence, terraform init is the prerequisite command that must be executed before any planning or applying operation for the first time.

References

Terraform init – Initialise a configuration directory –
https://developer.hashicorp.com/terraform/cli/commands/init
Lifecycle of a Terraform configuration –
https://developer.hashicorp.com/terraform/language/modules/develop#lifecycle

These links are from the official Terraform documentation and provide detailed guidance on initialization and the overall workflow.

Question: 32 CertyIQ

As a developer, you want to ensure your plugins are up-to-date with the latest versions.
Which Terraform command should you use?

A. terraform apply -upgrade
B. terraform providers -upgrade
C. terraform init -upgrade
D. terraform refresh -upgrade

Answer: C

Explanation:

Technical justification

terraform init -upgrade is the official command that re-initializes the working directory and forces Terraform to download the latest compatible provider plugins, even if the current versions appear to be satisfied. This guarantees that the plugin binaries used during subsequent apply or plan runs are up-to-date with the newest releases available in the registry.

The other answer choices are invalid or do not achieve the goal:

terraform apply -upgrade – the -upgrade flag is not defined for apply; applying provisions resources and does not refresh provider plugins.
terraform providers -upgrade – there is no such command; terraform providers only lists versions and does not download or upgrade them unless invoked with terraform init -upgrade or terraform providers install.
terraform refresh -upgrade – refresh is used to update the state snapshot; it has no -upgrade flag and does not impact provider version installation.

Therefore, the only command that explicitly triggers a provider-plugin upgrade during initialization is **C. terraform init -upgrade**.

References

Terraform CLI Install and Upgrade – Provides details on the -upgrade flag for terraform init.
https://developer.hashicorp.com/terraform/cli/commands/init#upgrade

Terraform Provider Install & Management – Explains how Terraform resolves and installs provider plugins, including the role of terraform init -upgrade.
https://developer.hashicorp.com/terraform/cli/plugins#installing-providers

Question: 33 CertyIQ

What does terraform apply change after you approve the execution plan? (Choose two.)

A. The state file
B. The cloud infrastructure
C. The execution plan
D. The Terraform code
E. The .terraform directory

Answer: AB

Explanation:

Terraform apply updates the persisted state file – After the plan is approved, apply calculates the actions required to achieve the desired state and writes that state back to the state file, reflecting the new resources and attributes.

Terraform apply creates or modifies real cloud infrastructure – The execution phase performs the actual API calls that provision, update, or destroy resources in the target provider (e.g., AWS, Azure, GCP), making the changes live in the environment.

Options C and D are not modified by apply – The execution plan is a read-only artifact generated by terraform plan; it is displayed but not changed by apply. The Terraform source code resides on disk and is never edited by the command.

Option E is a runtime data directory – .terraform holds internal working files (e.g., provider plugins, cached modules). While apply may update caches, these files are not considered "changed" in the sense of user-visible state; the command does not alter the directory structure itself.

Bottom line: terraform apply persists the new state to the state file and, through that, orchestrates the actual creation/modification of cloud resources, which is why **A and B** are the correct choices.

References

1. Terraform Docs –apply command:https://developer.hashicorp.com/terraform/cli/commands/apply
2. Terraform Docs –state and state file management:https://developer.hashicorp.com/terraform/language/state

Question: 34 CertyIQ

You modified your local Terraform configuration and ran terraform plan to review the changes. Simultaneously, your teammate manually modified the infrastructure component you are working on.
Since you already ran terraform plan locally, the execution plan for terraform apply will be the same.

A. True
B. False

Answer: B

Explanation:

Explanation

Why the correct answer is “False” (B)

When a teammate makes manual changes to the target infrastructure, the actual state stored in the remote backend is altered.
Your locally-generated Terraform plan was created based on the **original** remote state, not on the new state that now exists in the provider.
Consequently, the plans differ: yours reflects only your local changes, while the provider sees modifications you did not account for. Running terraform apply after this point will apply a plan that includes both your changes **and** whatever the provider now thinks needs to be reconciled, which is generally not the same as your original plan.

Why the other option (“True”) is unsuitable

The premise assumes that a locally-executed terraform plan captures the full desired state regardless of external drift.
In reality, Terraform plans are **state-driven**; they compare the desired configuration to the **current remote state**. If the remote state diverges — even without you making additional changes — the plan will be out-of-sync with what the provider expects.
Therefore, it is unsafe to assume that the execution plan will remain identical after concurrent manual modifications.

Result: The statement is false; you must refresh/pull the latest state before generating a new plan if any external modifications may have occurred.

References

Terraform documentation: Backend Options – https://developer.hashicorp.com/terraform/language/settings/backends
Terraform documentation: State management –https://developer.hashicorp.com/terraform/language/state

Question: 35 CertyIQ

Which is the correct workflow for deploying new infrastructure with Terraform?

A. 1. Write Terraform configuration.
2. Run terraform apply to create infrastructure.
3. Use terraform validate to confirm Terraform deployed resources correctly.

B. 1. Write Terraform configuration.
2. Run terraform init to initialize the working directory or workspace.
3. Run terraform apply.

C. 1. Write Terraform configuration.
2. Run terraform plan to initialize the working directory or workspace.
3. Run terraform apply to create the infrastructure.

D. 1. Write Terraform configuration.
2. Run terraform show to view proposed changes.
3. Run terraform apply to create new infrastructure.

Answer: B

Explanation:

Why optionB is the correct workflow

1. **terraform init must be executed first** – it sets up the working directory (downloads provider plugins, configures any backend, creates the .terraform directory). Without this step Terraform cannot locate the necessary providers or configure the execution environment, so any subsequent command would fail.
2. **terraform apply follows init** – after the environment is initialized, apply reads the configuration, builds the execution plan, prompts for approval (or uses -auto-approve), and then creates or updates the real infrastructure.
3. This sequence (write→init→apply) is the minimal, deterministic order required when introducing **new** infrastructure. It respects the dependency graph of resources while ensuring the provider state is ready.

Why the other options are unsuitable

OptionA – terraform validate is a syntax-and-consistency check; it does **not** confirm that resources were deployed correctly and should be run **before** apply. Using it after apply adds no value and interrupts the expected workflow.
OptionC – terraform plan does not initialize the working directory; it only generates a preview of changes. Running it before init will fail because Terraform cannot locate provider binaries or backend settings.
OptionD – terraform show merely displays the state or plan output; it is not a required step in the deployment pipeline and should not be placed before apply. Using it as a prerequisite step misrepresents the standard process.

Reference Implementation

Official Terraform CLI installation & command overview:https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli
Core workflow documentation (init → init → apply):https://developer.hashicorp.com/terraform/language/workspace/workflow

These links detail the proper initialization and application sequence that underpins optionB.

Question: 36 CertyIQ

After creating a new Terraform configuration, your config passes terraform validate but gives an “Access Denied” error from the cloud provider when running terraform plan.
Why didn’t validate catch this issue?

A. The working directory was not initialized, so the cloud provider plugin wasn’t available to use when running the terraform validate command.

B. The remote backend wasn’t configured, so terraform validate couldn’t load the state and detect the missing credentials.

C. terraform validate only checks if a configuration is syntactically correct and internally consistent, and does not communicate with providers.

D. Variables are only applied and validated during a terraform plan, so validate assumed defaults and returned the success message.

Answer: C

Explanation:

Why optionC is the correct answer

terraform validate only performs local checks: syntax correctness, required arguments are present, and internal consistency of expressions.
It never contacts any provider, reads state, or evaluates variable values that are only resolved during a plan/apply.
Consequently, an “AccessDenied” error caused by missing provider credentials is invisible to validate; the command considers the configuration valid and exits with a success status.

Why the other options are not appropriate

A. The provider plugins are loaded based on the backend configuration, not on whether the working directory has been initialized. validate can run without a prior terraform init and still perform its checks.
B. validate does not depend on a remote backend or state file; it operates entirely on the configuration files present in the working directory.
D. Variables are not “applied” during validate; they are parsed and type-checked, but no provider interaction occurs. The error is therefore unrelated to the phase of execution.

Key takeaway
terraform validate is deliberately limited to static analysis. Provider authentication failures are only detected when Terraform attempts to communicate with a provider, which happens during plan, apply, or other commands that require provider access.

References

Terraform CLI Documentation – validate: https://developer.hashicorp.com/terraform/cli/commands/validate
Terraform Errors – Access Denied:
https://developer.hashicorp.com/terraform/language/expressions/functions#access denied errors during provider interaction

Question: 37 CertyIQ

Only the user that generated a terraform plan may apply it.

A. True
B. False

Answer: B

Explanation:

Correct answer: B – False

Terraform executes plans and applies them using the credentials of the user who runs the command.
The state file records the user who performed each apply, and applying a plan created by another user requires explicit permissions on the backend or remote state store.
By default, any user with access to the state can run terraform apply, regardless of who generated the plan.
Restricting apply rights solely to the plan creator would require additional tooling (e.g., a custom script,

policy-enforcement layer, or CI/CD approval step) and is not a built-in behavior of Terraform.

Why the other option is less suitable

True would imply that Terraform enforces per-user apply restrictions automatically, which it does not. Therefore, the correct logical statement is **False**, reflecting Terraform's default operation.

References

Terraform documentation – Working with State: https://developer.hashicorp.com/terraform/language/state
Terraform documentation – CLI commands – apply: https://developer.hashicorp.com/terraform/cli/commands/apply

Question: 38 CertyIQ

terraform apply will fail if you have not run terraform plan first to update the plan output.

A. True
B. False

Answer: B

Explanation:

Technical Justification

The terraform plan command generates an execution plan that describes the actions Terraform will take, but it does **not** modify any real infrastructure.
terraform apply can be run directly; it will create a new plan internally if one does not already exist or if the underlying state has changed. Therefore, the requirement to first execute terraform plan is optional, not mandatory.
Skipping terraform plan is perfectly valid when you are confident in the configuration or when you intentionally want to apply changes without previewing them.

Why OptionB (False) Is Correct

OptionB states that the assertion "terraform apply will fail if you have not run terraform plan first" is false. This aligns with the behavior described above: Terraform will not abort; it will generate its own plan internally and proceed with the apply.

Why OptionA (True) Is Incorrect

OptionA claims the statement is true. That would only be correct if Terraform enforced a hard rule that an external plan file must be provided before any apply. Terraform does **not** enforce such a rule, so the statement is inaccurate.

Conclusion

The correct answer is **B. False**, because terraform apply can execute without a prior terraform plan run.

References

Terraform Documentation – CLI Commands > *apply*:

https://developer.hashicorp.com/terraform/cli/commands/apply
Terraform Documentation – CLI Commands > *plan*:
https://developer.hashicorp.com/terraform/cli/commands/plan

Question: 39 CertyIQ

What's the proper syntax for the plan command?

A. terraform plan -generate-config-out=tfplan
B. terraform plan -target=tfplan
C. terraform plan -out=tfplan
D. terraform plan -var-file=tfplan

Answer: C

Explanation:

Correct syntax: terraform plan -out=tfplan

The -out flag directs Terraform to write the generated plan to a binary file (tfplan), which can later be applied with terraform apply. This is the standard way to persist a plan for audited or automated workflows.

Why the other options are incorrect:

A. terraform plan -generate-config-out=tfplan – -generate-config-out writes the configuration files, not a plan; it never creates a tfplan file.
B. terraform plan -target=tfplan – -target limits the scope of the plan to specific resources; it does not output a plan file.
D. terraform plan -var-file=tfplan – -var-file specifies a file containing variable definitions; it does not control the output of the plan.

References

1. Terraform CLI documentation – Plan command options:
   https://developer.hashicorp.com/terraform/cli/commands/plan#-out
2. Using saved plans for reliable deployments:
   https://developer.hashicorp.com/terraform/language/state/saved-plan

Question: 40 CertyIQ

Which is true about terraform apply? (Choose two.)

A. It only operates on infrastructure defined in the current working directory or workspace.
B. You cannot target specific resources for the operation.
C. Depending on provider specification, Terraform may need to destroy and recreate your infrastructure resources.
D. You must pass the output of a terraform plan command to it.
E. By default, it does not refresh your state file to reflect the current infrastructure configuration.

Answer: AC

Explanation:

Why A and C are correct

A. Scope of operation – terraform apply only manages resources that are declared in the current working directory (or an associated workspace). It cannot ingest files from outside that directory hierarchy unless they are referenced through a relative path or a module source that resolves inside the directory. This limitation is documented and reinforces the importance of running the command from the appropriate directory.

C. Replacement behavior – Many cloud providers require an in-place change to be implemented as a destroy-and-recreate cycle when the desired state cannot be achieved by updating the existing resource (e.g., changing resource instance types, modifying immutable fields, or updating certain networking configurations). terraform apply evaluates the plan and, when a replacement is required, issues a destroy of the old resource followed by a create of the new one, respecting the provider's semantics.

Why the other options are not correct

B. Targeting resources – Terraform does support targeting specific resources (e.g., terraform apply -target=module.web, terraform apply -target=aws_s3_bucket.my-bucket). Therefore, the statement "You cannot target specific resources for the operation" is false.

D. Requirement of a plan output – terraform apply can be invoked without any prior terraform plan output. When a plan file is not supplied, apply will internally generate a new plan and then apply it. Hence, passing the output of terraform plan is optional, not mandatory.

E. State refresh behavior – terraform apply always refreshes the state to reflect the real infrastructure before applying changes (unless -refresh=false is explicitly set). The default behavior is to refresh, making the claim "it does not refresh your state file" incorrect.

References

Terraform Apply Documentation – https://developer.hashicorp.com/terraform/cli/commands/apply
Terraform State Management Overview – https://developer.hashicorp.com/terraform/language/state/statefile (includes discussion of refresh and replacement behavior)

Question: 41 CertyIQ

Which syntax check returns an error when you run terraform validate?

A. None of these will return an error.
B. There is a missing variable block.
C. The state file does not match the current infrastructure.
D. The code contains tabs for indentation instead of spaces.

Answer: A

Explanation:

Technical justification (exam-style)

Purpose of terraform validate – The command checks only the structural and syntactic correctness of the HCL configuration files (e.g., correct block delimiters, allowed characters, well-formed expressions). It does **not** perform any semantic analysis such as evaluating missing variables, comparing the state file with the live infrastructure, or enforcing code-style rules like indentation.

Option A – "None of these will return an error." – Correct. Because terraform validate only validates syntax, none of the listed conditions (missing variable block, state-configuration drift, or tab-based indentation) will cause a validation error; they either are ignored or are caught later by other commands (e.g., terraform plan/apply or terraform fmt).

Why option B is unsuitable – A missing variable block is a semantic omission. While it may lead to runtime errors when the variable is referenced, terraform validate does not detect this omission; it only ensures that the block syntax itself is correct.

Why option C is unsuitable – State file consistency is verified by commands like terraform plan -refresh=false or the backend health checks, not by validate. validate never reads or compares the state, so a mismatch will not surface as a validation error.

Why option D is unsuitable – Although tabs violate the Terraform style guide, the HCL parser accepts them and does not raise a validation error. Formatting concerns are addressed by terraform fmt or third-party linting tools, not by validate.

Conclusion: terraform validate will only fail on syntax errors; therefore, none of the listed scenarios will cause a validation error, making **Option A** the definitive answer.

References

Terraform CLI Documentation – validate command:
https://developer.hashicorp.com/terraform/cli/commands/validate
Terraform Configuration Language – HCL syntax rules:
https://developer.hashicorp.com/terraform/language/syntax

Question: 42 CertyIQ

What is the purpose of the .terraform directory in a Terraform workspace?

A. The directory contains plugins and modules that Terraform downloads during initialization, along with other important information.

B. The directory contains the provide credentials and the .tfvars files to prevent them from being committed to version control by accident.

C. The directory is where Terraform creates and maintains the state file to track the underlying resources it creates and manages.

D. The directory is used to convert and store Terraform configuration files into API calls to communicate with the targeted platform.

Answer: A

Explanation:

Why optionA is correct
The .terraform folder is created automatically after terraform init. Its primary role is to hold everything Terraform needs to manage the working directory independently of the configuration files. This includes:

Downloaded provider plugins and their cached binaries (/.terraform/providers/…).
The module installation cache (/.terraform/modules/).
The lock file that records provider and module versions (/.terraform.lock.hcl).
Internal state-metadata files used by Terraform's debugging and introspection mechanisms.

Because these artifacts are generated on-the-fly, they must be kept separate from source code to allow version-controlled configurations and reproducible builds.

Why the other options are not suitable

B. Provider credentials and .tfvars files belong in the root of the configuration or in environment-specific variable files; they are **not** stored inside .terraform. The directory does not manage secret handling.
C. The actual persistent state file (terraform.tfstate) resides in the configuration directory (or a backend) and is **not** managed by .terraform. The folder contains only temporary/internal data, not the canonical state.
D. Terraform translates HCL into API calls internally; the conversion and request generation happen in memory while executing a plan/apply. There is no conversion or storage step that results in API-call artifacts being saved under .terraform.

References

Terraform CLI Working Directories – https://developer.hashicorp.com/terraform/cli/workspaces#working-directories
terraform init documentation – https://developer.hashicorp.com/terraform/cli/commands/init#plugins-and-modules

Question: 43 CertyIQ

How can terraform plan aid in the development process?

A. Validates your expectations against the execution plan without permanently modifying state.
B. Reconciles Terraform's state against deploys resources and permanently modifies state using the current status of deployed resources.
C. Initializes your working directory containing your Terraform configuration files.
D. Formats your Terraform configuration files.

Answer: A

Explanation:

Technical justification

A – Validates expectations against the execution plan without permanently modifying state
terraform plan generates a preview of the actions Terraform will take to achieve the desired configuration. It compares the current state with the desired state, lists additions, modifications, and destructions, and presents this as a read-only plan. This lets developers verify that the upcoming changes align with design goals, catch misconfigurations, and avoid accidental resource drift — all without touching the persisted state file.

B – Reconciles state against deployed resources and permanently modifies state
This describes *terraform apply*, not plan. apply executes the plan and updates the state. plan only creates a tentative execution plan; it never writes state changes.

C – Initializes the working directory

Initialization is performed by *terraform init*, which downloads provider plugins and sets up the backend. plan assumes the directory is already initialized.

D – Formats configuration files
Formatting is handled by *terraform fmt*. plan does not perform any formatting operations.

Therefore, option **A** correctly captures the purpose of terraform plan in the development workflow.

References

Terraform Documentation – terraform plan
Terraform Documentation – terraform init

Question: 44 CertyIQ

You have a saved execution plan containing desired changes for infrastructure managed by Terraform. After running the command terraform apply my.tfplan, you receive the error shown in the Exhibit space on this page.
| Error: Saved plan is stale
|
| The given plan file can no longer be applied because the state was changed by another operation after the plan was created.
How can you apply the desired changes? (Choose two.)

A. Generate a new execution plan file with terraform plan, and apply the new plan.
B. Run terraform apply without the saved execution plan.
C. Refresh the current state data using the -refresh-only flag.
D. Force the apply command by adding the flag -lock=false.
E. Update the current plan file using the terraform state push command.

Answer: AB

Explanation:

Provide a short, structured technical justification suitable for a certification exam review. do not use table structure, use paragraph or bullet point format.Explain why the correct option is best and why other options are less suitable. Be precise, technical, and professional. optional - Put 2 valid and working documentation links at the very end under a section titled References.

The correct options are A and B because they address the root cause of the “Saved plan is stale” error: the state has diverged from the plan, so the existing plan can no longer be safely applied. Generating a new execution plan withterraform planproduces a fresh plan that reflects the current state, and then applying it withterraform applyupdates the infrastructure according to the latest desired state. Simply runningterraform applywithout the saved plan also works, but it implicitly triggers a new plan within the apply command, effectively achieving the same outcome without needing a separate plan file.

OptionC (refresh-only) only updates the state metadata; it does not create a new plan and therefore cannot resolve a stale plan error on its own. OptionD (-lock=false) bypasses state locking but does not fix the staleness condition and can lead to race conditions or corruption. OptionE (terraform state push) is unrelated to plan application and does not affect the plan file’s validity. Hence, only A and B correctly resolve the issue by ensuring a fresh, consistent plan is applied.

References:

1. Terraform Documentation – Working with Plans:

https://developer.hashicorp.com/terraform/language/plan

2. Terraform Documentation – apply Command:
https://developer.hashicorp.com/terraform/cli/commands/apply

Question: 45 CertyIQ

Which of these can you do with terraform plan? (Choose two.)

A. Schedule Terraform to run at a planned time in the future.
B. View the execution plan and check if the changes match your expectations.
C. Execute a plan in a different workspace.
D. Save a generated execution plan to apply later.

Answer: BD

Explanation:

Why B and D are correct

B – View the execution plan and verify changes
terraform plan generates a detailed proposal of the actions Terraform will take to reach the desired state. It lists resource additions, modifications, and destructions, allowing you to review and confirm that the upcoming changes align with expectations before any modifications are persisted.

D – Save a generated execution plan for later application
The -out (or -out=plan.out) flag lets you serialize the plan into a binary file. This saved plan can be reapplied later with terraform apply plan.out, enabling controlled, repeatable deployments (e.g., in CI pipelines or manual approvals).

Why A and C are not appropriate

A – Schedule Terraform to run at a future time
Scheduling execution is outside the scope of terraform plan. Scheduling is handled by external orchestration tools (e.g., Jenkins, GitHub Actions, Terraform Cloud) and is not a feature of the plan command itself.

C – Execute a plan in a different workspace
While workspaces isolate state, the plan command does not execute changes; it merely produces a plan. To apply a plan in another workspace you would need to switch workspaces (e.g., terraform workspace select other) or copy the plan file, but the command does not perform execution across workspaces.

References

Terraform documentation – terraform plan

https://developer.hashicorp.com/terraform/cli/commands/plan

Saving and loading plans

https://developer.hashicorp.com/terraform/cli/commands/plan#save-the-plan

These links provide the official specifications for viewing plans and persisting them for later use.

Question: 46 CertyIQ

Which task does terraform init not perform?

A. Sources any remote modules and downloads them.
B. Connects to the configured backend.
C. Validates that values are set for all required input variables.
D. Sources all providers used in the configuration and downloads them.

Answer: C

Explanation:

Why the correct option (C) is the only one that does not describe a task performed by terraform init

A. Sources any remote modules and downloads them – terraform init resolves and fetches module sources defined with source arguments, placing them in the module cache. This is a core responsibility of the initialization command.
B. Connects to the configured backend – During initialization Terraform reads the backend configuration (e.g., S3, Azure Blob, Terraform Cloud) and establishes the connection needed for state storage.
D. Sources all providers used in the configuration and downloads them – Provider plugins are discovered from the required providers block and from any required_providers entries, then installed into the working directory's .terraform folder.
C. Validates that values are set for all required input variables – Variable validation and requirement checking are performed by terraform validate and terraform plan, not by terraform init. Initialization only prepares the environment; it does **not** evaluate input variable definitions.

Therefore, option **C** correctly identifies a task that terraform init does not execute.

Why the other options are unsuitable

A, **B**, and **D** each describe a concrete step that terraform init must carry out to make the working directory usable: module resolution, backend connection, and provider installation.
C confuses initialization with later validation phases; Terraform never checks variable completeness during init.

References

Terraform CLI Documentation – terraform init: https://developer.hashicorp.com/terraform/cli/commands/init
Remote Module Execution – Modules: https://developer.hashicorp.com/terraform/language/modules/development (module source resolution details)

Question: 47 CertyIQ

terraform init retrieves and caches the configuration for all remote modules.

A. True
B. False

Answer: A

Explanation:

Justification

The terraform init command initializes a working directory that contains Terraform configuration files. During initialization it downloads and installs the required provider plugins and, if the configuration references any **remote modules**, it fetches those modules from their source locations (e.g., Terraform Registry, Git, S3, etc.) and stores them in the .terraform directory.
After the retrieval, each module's source files become available locally, and Terraform can later create resources based on those modules without needing external network calls.
Therefore, the statement "retrieves and caches the configuration for all remote modules" accurately describes the purpose of terraform init.

Why the other option is less suitable

Selecting "False" would contradict the documented behavior of terraform init, which explicitly includes downloading remote modules as part of its initialization steps. Choosing "False" ignores the core responsibility of the command and would be technically incorrect.

References

Terraform Documentation – Initialization: https://developer.hashicorp.com/terraform/cli/init
Terraform Registry – Using Remote Modules:
https://developer.hashicorp.com/terraform/language/modules/sources#source-addresses

---

Question: 48 CertyIQ

You just upgraded the version of a provider in an existing Terraform configuration. What will install the new provider?

A. Run terraform plan.
B. Run terraform apply.
C. Run terraform init -upgrade.
D. Run terraform refresh.

Answer: C

Explanation:

Why optionC is the correct choice

terraform init -upgrade explicitly tells Terraform to re-download and install any newer versions of referenced providers that satisfy the version constraints in the configuration. This command is designed for exactly the scenario of upgrading a provider version in an already-existing codebase without destroying resources.
It performs a **safe upgrade**: providers are fetched into the .terraform directory, the backend is re-initialized if needed, and the existing state and configuration remain untouched until a subsequent terraform apply is run.
Other commands address different phases of the workflow:

A. terraform plan – only analyzes the configuration and produces a preview; it does not download or install new provider binaries.
B. terraform apply – creates or modifies infrastructure but will fail if the required provider version is not present; it does not install anything by itself.
D. terraform refresh – updates the state file to reflect the real infrastructure but never touches provider binaries.

Therefore, to actually install the new provider version after changing its constraints, you must run **terraform init -upgrade**.

Why the other options are unsuitable

They either miss the step of downloading the updated provider binaries or they only operate on the state/plans, not on provider installation.
Using any of them alone would not guarantee the new provider code is available for subsequent operations, potentially leading to errors or unexpected behavior.

References

Terraform Documentation – Initialization: https://developer.hashicorp.com/terraform/cli/commands/init# -upgrade
Terraform Registry – Provider Installation: https://developer.hashicorp.com/terraform/language/providers/installation#installing-providers

Question: 49 CertyIQ

terraform validate confirms the syntax of Terraform files.

A. True
B. False

Answer: A

Explanation:

Answer: A – True

Technical justification

terraform validate performs a static analysis of all .tf files. It checks for syntax errors, provider configuration issues, and correct usage of Terraform language constructs (blocks, arguments, expressions).
If the validation succeeds, Terraform reports “Success! The given configuration is valid.” indicating the syntax is correct; otherwise it returns detailed error messages describing the specific problems.
The command does **not** perform state checks, plan previews, or resource creation — it only verifies structural and lexical correctness of the HCL code. Therefore the statement “terraform validate confirms the syntax of Terraform files” is accurate.
Option **B** is incorrect because terraform validate does confirm syntax; it does not skip that step.

Why the other option is less suitable

B (“False”) would be chosen only if the command performed validation beyond syntax (e.g., checking resource dependencies). Since its primary purpose is syntax checking, labeling it as “False” mischaracterizes its function.

References

Terraform documentation – Validate: https://developer.hashicorp.com/terraform/cli/commands/validate
Terraform language documentation – Configuration syntax: https://developer.hashicorp.com/terraform/language/syntax

---

Question: 50 CertyIQ

A Terraform configuration is returning errors when running terraform plan.
How would you enable debug logging to investigate further?

A. Rerun terraform plan with the correct flag.
B. Edit the client configuration file.
C. Update the backend configuration.
D. Set the correct environment variable.

Answer: D

Explanation:

The correct choice is **D. Set the correct environment variable.**
Enabling debug-level logging in Terraform is done by setting the TF_LOG environment variable to DEBUG (or a specific level such as INFO, WARN, ERROR). When this variable is exported before running terraform plan, Terraform prints detailed internal logs that help diagnose configuration-related errors. This method directly influences the logging behavior without modifying files or infrastructure settings.

Why the other options are unsuitable

A – Rerun terraform plan with the correct flag: Terraform does not have a built-in "flag" for increasing log output; the only supported way is via TF_LOG.
B – Edit the client configuration file: Terraform's client configuration (e.g., ~/.terraformrc) controls provider credentials and backend settings, not log verbosity.
C – Update the backend configuration: Backend settings govern where state is stored and how it is accessed; they do not affect logging levels.

Therefore, setting TF_LOG=DEBUG (or another appropriate level) is the precise, officially supported mechanism for obtaining debug output during a plan run.

References

Set environment variables for logging – https://developer.hashicorp.com/terraform/language/values/functions/log
Debugging and logging in Terraform – https://developer.hashicorp.com/terraform/tutorials/cli/config-file#logging

---

Question: 51 CertyIQ

Why would you use the -replace flag for terraform apply?

A. You want to force Terraform to destroy a resource on the next apply.
B. You want Terraform to ignore a resource on the next apply.
C. You want Terraform to destroy all the infrastructure in your workspace.
D. You want to force Terraform to destroy and recreate a resource on the next apply.

Answer: D

Explanation:

Why the -replace flag is the correct choice

The -replace flag tells Terraform to treat a specified resource address as if it were newly created, forcing a **destructive destroy followed by a fresh apply** of that resource.
This is required when the configuration of a resource must be changed in an **immutable** way (e.g., altering a calculated attribute, changing a lifecycle rule, or updating a field that cannot be drifted).
By using -replace, you can keep the rest of the state intact while ensuring the targeted resource is recreated on the next terraform apply, avoiding plan failures caused by configuration drift.
The operation is **explicit and surgical** — only the addressed resource is affected, unlike a full-workspace destroy.

Why the other options are incorrect

A. "Force Terraform to destroy a resource on the next apply."
Destroying a resource alone can be done with terraform destroy or by removing it from the configuration; -replace does more than merely destroy — it recreates the resource after the destroy.

B. "Ignore a resource on the next apply."
Ignoring resources is achieved via the -ignore-apply-errors flag or by using lifecycle directives, not by -replace.

C. "Destroy all the infrastructure in your workspace."
-replace targets a **specific resource address**; it does not automatically affect every resource in the workspace. A full-workspace destroy requires different commands.

Key takeaway: -replace is the precise mechanism to **force a destroy-and-recreate cycle for a single resource**, making option **D** the only technically accurate answer.

References

Terraform CLI -replace flag: https://developer.hashicorp.com/terraform/cli/commands/apply#replace
Lifecycle management and immutable changes: https://developer.hashicorp.com/terraform/language/resources-lifecycle#replacing-a-resource

Question: 52 CertyIQ

How does Terraform determine dependencies between resources when it creates an execution plan?

A. Terraform builds a resource graph based on your configuration and your state file (if present).
B. Terraform requires resources in your configuration be listed in the order they will be created to determine dependencies.
C. Terraform requires all dependencies between resources be specified using the depends_on parameter.

D. Terraform requires resource dependencies be defined as modules and sourced in order.

Answer: A

Explanation:

Why option A is correct

Terraform constructs an internal resource graph by parsing every module and root module in the configuration.
If a **state file** is present, Terraform also reads the current state to identify any implicit dependencies (e.g., a resource that references an output exported by another).
The graph is traversed to create an execution plan that respects all prerequisite relationships before the plan is applied.

Why the other options are not suitable

B. Terraform does **not** require the user to list resources in the order they will be created; the engine determines the order automatically from the dependency graph.
C. While depends_on can force a dependency, it is **not required**; most dependencies are inferred automatically, and omitting it does not prevent Terraform from building a valid plan.
D. Dependencies do not need to be expressed as ordered modules or sourced sequentially; modules are just reusable containers and the engine resolves intra-module dependencies using the same graph-building logic.

References

Terraform Documentation – Resources and Dependencies: https://developer.hashicorp.com/terraform/language/resources-deps
Terraform Documentation – Resource Graph: https://developer.hashicorp.com/terraform/language/resource-graph

Question: 53 CertyIQ

You create a main.tf Terraform configuration consisting of an application server, a database, and a load balancer. You run terraform apply and Terraform created all the resources successfully.
Now you realize that you do not actually need the load balancer, so you run terraform destroy without any flags. What will happen?

A. Terraform will prompt you to confirm that you want to destroy all the infrastructure.
B. Terraform will prompt you to pick which resource you want to destroy.
C. Terraform will immediately destroy all the infrastructure.
D. Terraform will destroy the application server because it is listed first in the code.
E. Terraform will destroy the main.tf file.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

When running terraform destroy without any flags, Terraform's behavior is to terminate all resources managed in the current working directory's configuration (in this case, defined in main.tf). Here's why the correct answer

is A and why other options are less suitable:

Correct Answer: A

Terraform will prompt you to confirm that you want to destroy all the infrastructure.

Reason: By default, terraform destroy will display a preview of the destruction plan and then prompt for confirmation before proceeding. This is a safety feature to prevent accidental destruction of entire infrastructures. Since the command lacks flags that might specify a target resource (e.g., -target) or auto-confirm destruction (e.g., -auto-approve), Terraform seeks user confirmation to ensure the action is intentional.
Incorrect Answers with Justifications

B. Terraform will prompt you to pick which resource you want to destroy.

Why Incorrect: This behavior would require a specific interactive mode or flag not mentioned in the question. Terraform does not natively offer a "pick which resource" prompt during destroy without additional configuration or flags like -target.
C. Terraform will immediately destroy all the infrastructure.

Why Incorrect: Without the -auto-approve flag, Terraform will not proceed with destruction without user confirmation, making this outcome incorrect for the given command.
D. Terraform will destroy the application server because it is listed first in the code.

Why Incorrect: The order of resource definition in Terraform configuration files does not dictate the order of destruction in a blanket terraform destroy command. Terraform determines the destruction order based on dependency graphs, not the order in the config file.
E. Terraform will destroy the main.tf file.

Why Incorrect: terraform destroy targets the infrastructure resources defined in the configuration files, not the configuration files themselves. The file main.tf would remain intact.
References

Terraform Destroy Command Documentation
Terraform Configuration and State Documentation

Question: 54 CertyIQ

You developed a new Terraform configuration for two virtual machines with a cloud provider. You're ready to create the infrastructure for the first time.
Which Terraform command should you run first?

A. terraform apply
B. terraform plan
C. terraform init
D. terraform show

Answer: C

Explanation:

Technical Justification for Correct Answer: C (terraform init)

When creating infrastructure for the first time with a new Terraform configuration, the initial step is crucial for

setting up the environment correctly. Here's why terraform init is the best first choice, followed by why other options are less suitable for the initial step:

Why C (terraform init) is Correct

Initialization Requirement: Before any operations (plan, apply, or show), Terraform requires initialization to set up the working directory, download and install the required providers (in this case, the cloud provider), and configure the backend (if any).
Prerequisite for Other Commands: Running terraform init ensures the environment is properly configured for subsequent commands like terraform plan or terraform apply. Without initialization, these commands cannot execute as they rely on the setup provided by terraform init.
Safety and Configuration Check: It performs a safety check on the configuration, ensuring that the prerequisites for deployment are met before attempting to plan or apply the changes.
Why Other Options are Less Suitable as the First Step

A. terraform apply:

Risk of Unprepared Environment: Directly applying changes without ensuring the environment is set up correctly can lead to errors or unexpected behavior.
Lacks Preliminary Check: Skipping initialization and planning steps can lead to unforeseen infrastructure provisions without a prior review of the changes.
B. terraform plan:

Dependent on Initialization: Like apply, plan requires the environment to be initialized first. Running it without terraform init will fail.
Useful After Initialization: However, it's highly recommended to run terraform plan after terraform init and before terraform apply to review the planned changes.
D. terraform show:

Irrelevant to New Infrastructure Creation: terraform show is used to display the current infrastructure's state. Since no infrastructure exists yet, this command does not contribute to the setup process.
Utility in Existing Environments: Useful for understanding the current state of already provisioned infrastructure.
Recommended Initial Workflow for New Infrastructure

1. **terraform init** (Correct Answer for the first step)
2. **terraform plan** (To review planned changes)
3. **terraform apply** (To provision the infrastructure)

References

HashiCorp Terraform Documentation - terraform init
HashiCorp Terraform Documentation - Command Workflow

Question: 55 CertyIQ

When should you run terraform init?

A. After you start coding a new Terraform project, and before you run terraform plan for the first time.
B. Every time you run terraform apply.
C. Before you start coding a new Terraform project.
D. After you run terraform plan for the first time in a new Terraform project, and before you run terraform apply.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

The correct answer is **A. After you start coding a new Terraform project, and before you run terraform plan for the first time.** Here's why:

Initialization Necessity: terraform init is used to initialize a Terraform working directory. This command prepares the working directory by installing and configuring required providers, setting up backend settings (if configured), and managing the Terraform state (if a backend is specified). It's essential to run this after starting to code to ensure the infrastructure as code (IaC) configuration is properly set up with the required dependencies.
Sequence Logic:

Coding the project first allows for defining the infrastructure requirements.
Running terraform init afterward ensures the project directory is properly initialized with the necessary providers and settings before any planning or application of changes.
Running terraform plan subsequently allows for reviewing the planned changes without applying them, which is a best practice for verifying the correctness of the infrastructure changes.

Why Other Options are Less Suitable:

B. Every time you run terraform apply:

Unnecessary Overhead: Running terraform init every time before terraform apply introduces unnecessary overhead, as the initialization only needs to occur once per project (or when the backend/provider versions change).
Potential for Unintended Provider Updates: Frequent re-initialization could lead to unintended provider version updates if not carefully managed.
C. Before you start coding a new Terraform project:

Premature Initialization: Initializing before coding doesn't make sense because there's no configuration for Terraform to initialize with. The command would essentially do nothing useful in this context.
D. After you run terraform plan for the first time in a new Terraform project, and before you run terraform apply:

Incorrect Sequence for Initialization: terraform plan can technically run without a full terraform init (though init is usually implied in the first run of any command in a directory), but the key point is that initialization should happen before planning to ensure everything is set up correctly from the start.

References

HashiCorp Terraform Documentation - terraform init
HashiCorp Terraform Getting Started Guide (See the "Initialize the Working Directory" section)

Question: 56 CertyIQ

Which configuration consistency error does terraform validate report?

A. Declaring a resource identifier more than once.
B. Differences between local and remote state.
C. An out-of-date module.

D. A mix of spaces and tabs in configuration files.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

The Terraform validate command is primarily designed to check the syntax, format, and certain aspects of configuration consistency within Terraform configuration files. Here's why option A is the correct answer and why the others are less suitable:

Why A is Correct: Declaring a resource identifier more than once.

Reason: Terraform's validate command checks for duplicate resource identifiers because having two resources with the same identifier in the same configuration would lead to a conflict during the apply phase, as Terraform would not know how to reconcile the two declarations.
Technical Impact: Duplicate resource identifiers violate the fundamental principle of resource uniqueness in Terraform, which is crucial for maintaining a predictable infrastructure state.
Validation Scope: This check falls under the configuration consistency errors that terraform validate can detect through static analysis of the configuration files.

Why Other Options are Less Suitable:

B. Differences between local and remote state.

Reason for Exclusion: While crucial for Terraform's operational integrity, differences between local and remote states are typically identified during terraform plan or terraform apply (when Terraform interacts with the backend/state store), not during terraform validate. validate focuses on configuration syntax and consistency, not state synchronization.

C. An out-of-date module.

Reason for Exclusion: Module version checks (e.g., ensuring a module is up-to-date) are not within the scope of terraform validate. This would more likely be addressed through other practices (e.g., regular dependency updates, using specific version pins in module declarations).

D. A mix of spaces and tabs in configuration files.

Reason for Exclusion: Although a mix of spaces and tabs can lead to configuration readability and potential merge conflicts in version control, Terraform's validate command does not check for this as it's more of a coding style issue rather than a configuration consistency error that would prevent Terraform from functioning correctly. Tools like linters (e.g., Terraform Lint, terraform fmt for formatting) are more suited for addressing such issues.

References

Terraform Validate Command Documentation
Terraform Resource Documentation (emphasizing unique identifiers)

Question: 57 CertyIQ

Which of the following arguments are required when declaring a Terraform output?

A. default
B. value
C. description
D. sensitive

Answer: B

Explanation:

Technical justification

value – This argument is mandatory in an output block because it tells Terraform what piece of data the output should expose; without it the block would be syntactically invalid and the configuration would fail to plan/apply.
default – Only used to provide a fallback value when the output is not set; it is optional and does not affect the block's validity.
description – A human-readable annotation that improves documentation and IDE hints; it is purely informational and not required for execution.
sensitive – Controls whether the output value is masked in logs and UI; it is a security flag and can be omitted when the output is not sensitive.

Therefore, among the listed arguments, **only value is required** when declaring a Terraform output.

References

Terraform Documentation – Output Values: https://developer.hashicorp.com/terraform language/values/outputs
Terraform CLI – output Block Details: https://developer.hashicorp.com/terraform/cli/outputs

---

Question: 58 CertyIQ

Outside of the required_providers block, Terraform configurations always refer to providers by their local names.

A. True
B. False

Answer: A

Explanation:

Justification

-In a Terraform configuration, provider references outside of the required_providers block are made by the alias that you assign in that block (e.g., aws, azurerm).
-The alias is defined once and can then be used throughout the rest of the configuration (module sources, provider arguments, etc.). This is why the statement is **True**.
-OptionB ("False") would imply that you must use the full provider address (hashicorp/aws) outside the block, which contradicts the way Terraform resolves providers internally and would break the configuration.

Why the other option is not suitable

-Terraform resolves providers only by the alias supplied in required_providers. Using the address syntax elsewhere is not required and is discouraged; it would defeat the purpose of the alias-based local naming scheme. Hence “False” does not correctly describe the behavior described in the question.

References
-[Terraform Registry – Provider Requirement Syntax]
(https://developer.hashicorp.com/terraform/language/providers require)
-Provider Configuration – Using Aliases

Question: 59 CertyIQ

The Exhibit section of this page shows part of a configuration you’ve been asked to update. data “azurerm_resource_group” “example” name = var.resource_group_name

resource “azurerm_virtual_network” “example”
name = _____

The name of the Azure Virtual Network should be set to the name of the resource group followed by a dash and the word “vnet”.
Which expression fulfills this requirement?

A. “$ azurerm_resource_group.example.name -vnet”
B. join(“-”,var.resource_group_name,“vnet”)
C. concat(data.azurerm_resource_group.example.name, “-”, “vnet”)
D. “$ data.azurerm_resource_group.example.name -vnet”

Answer: D

Explanation:

Why optionD is the correct expression

It uses the proper **data source reference** (data.azurerm_resource_group.example.name) that matches the declared data block in the example.
It encloses the interpolation in double-quotes, allowing Terraform to resolve the expression as a quoted string.
The resulting value is exactly <resource-group-name>-vnet, satisfying the requirement to append “-vnet” to the resource-group name.

Why the other options are less suitable

OptionA – Uses $ azurerm_resource_group.example.name which references a **resource** block name instead of the declared **data source**; the name azurerm_resource_group.example does not refer to the data block defined above, so Terraform cannot resolve it.
OptionB – Calls join("-", var.resource_group_name, "vnet"). join expects a list as its second argument, and introducing a variable that holds only the raw name bypasses the data source altogether. The expression therefore does not guarantee it reads the value from the configured data source.
OptionC – Although concat() would produce a valid string, it must be used on a list and returns a list; the syntax shown (concat(data.azurerm_resource_group.example.name, “-”, “vnet”)) does not wrap the result in parentheses or explicitly convert it to a string, which can lead to type mismatches in newer Terraform versions.

Hence, optionD is the most precise, idiomatic and reliable way to construct the required VNet name.

References

Terraform Language – Expressions: https://developer.hashicorp.com/terraform/language/expressions
Data Sources – Azure Resource Group:
https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/data-sources/resource_group

These links provide the official documentation supporting the correct use of data source interpolation and expression syntax in Terraform.

Question: 60 CertyIQ

Your team is collaborating on infrastructure using Terraform and are formatting their code to follow Terraform language style conventions.
How can you update your code to meet these requirements?

A. Run terraform validate prior to executing terraform plan or terraform apply.
B. Replace all tabs with spaces within your Terraform configuration files.
C. Terraform automatically formats configuration on terraform apply.
D. Run terraform fmt to update your Terraform configurations.

Answer: D

Explanation:

Technical justification

Option D – terraform fmt

terraform fmt rewrites configuration files in the canonical Terraform style, normalizing spacing, line breaks, and indentation while preserving content.
It can be run automatically (e.g., in CI pipelines) to enforce a consistent formatting convention across all files.
The command is idempotent; running it on already-formatted code produces no changes, making it safe for iterative development and code-review workflows.

Why the other options are less suitable

A. terraform validate – Checks syntactic correctness and internal consistency but does **not** modify formatting or style.
B. Replacing tabs with spaces – Manual text substitution does not address the full set of style conventions (e.g., line length, quote usage, ordering of blocks) and is error-prone; Terraform's formatter handles these rules automatically.
C. Terraform automatically formats configuration on terraform apply – Terraform never rewrites files during apply; it only validates and applies changes. Formatting must be performed explicitly via terraform fmt.

Therefore, the recommended approach is to run terraform fmt on the codebase.

References

Terraform CLI – terraform fmt documentation:

https://developer.hashicorp.com/terraform/cli/commands/formatted
Terraform style and formatting best practices: https://developer.hashicorp.com/terraform/language/style-guide#formatting-files

Question: 61 CertyIQ

A resource block is shown in the Exhibit space of this page.
resource “aws_instance” “web”
count = 2
name = “terraform-$ count.index ”

How do you reference the name value of the second instance of this resource?

A. aws_instance.web.*.name
B. aws_instance.web[2].name
C. element(aws_instance.web, 2)
D. aws_instance.web[1].name
E. aws_instance.web[1]

Answer: D

Explanation:

Correct answer – D. aws_instance.web[1].name

The resource aws_instance.web is defined with count = 2, creating two instances indexed from0.
Index1 refers to the second instance, and accessing the name attribute (aws_instance.web[1].name) returns exactly the name value of that second instance.

Why the other options are incorrect

A. aws_instance.web..*name* – The . syntax expands to a set of values, returning a collection rather than a single string; it cannot be used directly to obtain a scalar value.
B. aws_instance.web[2].name – Indexing starts at0, so [2] attempts to access a non-existent third instance, causing a runtime error.
C. element(aws_instance.web, 2) – The element function expects an index; using 2 again points to a non-existent element, whereas the correct index is 1.
E. aws_instance.web[1] – This returns the entire resource object for the second instance, not just its name attribute.

References

Terraform CLI Documentation – Resource Addressing:
https://developer.hashicorp.com/terraform/cli/syntax/resources
Terraform Functions – element: https://developer.hashicorp.com/terraform/language/functions/element

Question: 62 CertyIQ

When using multiple configurations of the same Terraform provider, what meta-argument must you include in any non-default provider configurations?

A. alias
B. name
C. depends_on
D. id

Answer: A

Explanation:

Technical Justification

When Terraform needs to instantiate more than one instance of a single provider (e.g., two distinct AWS accounts), every non-default provider configuration must be given a unique **alias** meta-argument.

The alias serves as a distinct provider name that Terraform uses internally to reference that specific configuration instance. Without an alias, Terraform cannot differentiate the multiple providers and will raise an error at plan/apply time.
The alias is required only for the **non-default** (i.e., explicitly aliased) providers; the default provider block does **not** need an alias.

Why the other choices are incorrect

B. name – Terraform does not expose a name meta-argument for providers; the concept of naming a provider block is handled solely by the alias attribute.
C. depends_on – This meta-argument controls ordering of resources or modules, not provider instance identification, and it cannot be used to differentiate provider configurations.
D. id – id is the computed resource address of a provisioned resource after apply; it is an output value, not a configuration attribute that can be set on a provider block.

Therefore, **alias** is the only meta-argument that must be included when referencing a non-default provider configuration multiple times within the same Terraform module.

References

Terraform Provider Configuration: https://developer.hashicorp.com/terraform/language/providers
Using Provider Aliases:
https://developer.hashicorp.com/terraform/language/providers#configuring_multiple_instances_of_one_provider

Question: 63 CertyIQ

You want to create a string that is a combination of a generated random_id and a variable and reuse that string several times in your configuration.
What is the simplest correct way to implement this without repeating the random_id and variable?

A. Add an output value.
B. Add a local value.
C. Use a data source.
D. Use a module.

Answer: B

Explanation:

Answer:

B. Add a local value – Correct

Locals are designed for storing intermediate results (e.g., a concatenated string of a random ID and another variable) that can be referenced multiple times throughout the same configuration via local.<name>.
Using a local eliminates repetition of the random ID and the variable and keeps the expression evaluation clear and manageable.
The value can be defined once and reused anywhere in the configuration without needing to re-evaluate the expression.

A. Add an output value – Incorrect

Outputs expose values to the caller (other configurations) but do not provide an in-module variable that can be reused locally; they are intended for external consumption, not internal reuse.
Overkill when the goal is simply to avoid repeated calculations inside the same configuration.

C. Use a data source – Incorrect

Data sources fetch external data (e.g., from other providers) and are not meant for storing computed strings based only on internal inputs.
They introduce unnecessary complexity and cannot be used as a persistent, reusable variable within the same module.

D. Use a module – Incorrect

Modules encapsulate sets of resources and configurations, typically for re-usability across separate projects or environments.
Creating a module solely to reuse a concatenated string would violate the principle of "doing the simplest thing that works" and adds an unnecessary abstraction layer.

References

HashiCorp Terraform Docs – Locals: https://developer.hashicorp.com/terraform/language/expressions/locals
HashiCorp Terraform Docs – Expressions – Concatenation: https://developer.hashicorp.com/terraform/language/expressions#string_literal_concatenation

Question: 64 CertyIQ

Your Terraform configuration manages a resource that requires maximum uptime. You need to update the resource, and when you run terraform plan, Terraform notes that the update requires the resource to be destroyed and recreated.
Which lifecycle rule can you add to the resource to reduce the downtime of the resource while still applying the update?

A. prevent_destroy = true
B. destroy = false
C. create_before_destroy = true
D. ignore_changes = all

Answer: C

Explanation:

Why option C is correct

Adding create_before_destroy = true tells Terraform to provision a new resource first, then switch traffic to it, and finally destroy the old resource. This sequencing eliminates any gap where the resource is absent, thus minimizing downtime while still allowing the replacement of the resource that must be recreated.
The other options do not address the recreate-only situation:

prevent_destroy = true stops Terraform from destroying the resource at all, which would block the required update.
destroy = false merely postpones the destroy step but still destroys the resource later without mitigating the outage caused by the recreation.
ignore_changes = all tells Terraform to ignore all changes to the resource, which would also prevent you from applying the necessary update and therefore is not a viable solution.

Why the remaining options are unsuitable

A – prevent_destroy = true: Useful when you never want to delete the resource, but it would stop Terraform from ever recreating the resource that needs replacement.
B – destroy = false: Can delay a destroy, yet it still results in a period where the resource is missing while Terraform attempts to rebuild it, leading to the same downtime issue.
D – ignore_changes = all: By ignoring all attribute changes, Terraform will not notice any modification you intend to make, making it impossible to trigger the required recreation and update.

References

Terraform Documentation – Lifecycle Configuration:
https://developer.hashicorp.com/terraform/language/resources/lifecycle
Create Before Destroy – How to use it safely:
https://developer.hashicorp.com/terraform/language/resources/lifecycle#create_before_destroy

Question: 65 CertyIQ

The Terraform configuration shown in the Exhibit space on this page will create a new AWS instance. data “aws_instance” “web” filter name = “tag:Name” values = [“web”]

A. True
B. False

Answer: B

Explanation:

Justification

The data "aws_instance" "web" block is used to **read** existing EC2 instances that already have a tag Name equal to "web"; it does **not** create a new instance.
Creation of an EC2 instance in Terraform is performed with the aws_instance resource block, not with a data block.
Therefore the configuration described will **only search** for matching instances, and if none are found it will simply return an empty result – it will not provision a new instance.
Consequently, the statement “the configuration will create a new AWS instance” is **false**.

Why option A is not suitable

Option A claims the configuration creates a new instance, which contradicts how Terraform's data sources work.
Data sources are read-only; they cannot modify infrastructure, so they cannot be used to launch resources.

Result

The correct answer is **B – False**.

References

Terraform AWS Provider – Data Sources:
https://registry.terraform.io/providers/hashicorp/aws/latest/docs/data-sources
aws_instance Resource – Creation Example:
https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/instance#example-usage

Question: 66 CertyIQ

What kind of configuration block will manage an infrastructure object with settings specified within the block?

A. data
B. locals
C. provider
D. resource

Answer: D

Explanation:

Justification

Resource block (D) – This is the Terraform configuration element that creates and manages an infrastructure object (e.g., aws_instance, google_storage_bucket). All settings that define the desired state of that object are written inside its body.

Data block (A) – Intended for reading existing external data or gathering information at plan time; it does not create or modify resources.

Locals block (B) – Used to define internal, reusable values for the current configuration. It does not interact with the provider or affect any real infrastructure.

Provider block (C) – Configures authentication and feature enablement for a specific service (e.g., AWS, GCP). It does not itself manage an infrastructure object; it only tells Terraform how to talk to the service.

Therefore, the block type that directly manages an infrastructure object with its settings defined inside the block is the **resource** block.

References

Terraform Language – Resources
Terraform Language – Providers

Question: 67 CertyIQ

You can execute terraform fmt to standardize all Terraform configurations within the current working directory to Terraform's canonical format and style.

A. True
B. False

Answer: A

Explanation:

Justification

terraform fmt rewrites every supported configuration file under the current working directory (and its sub-directories) to HashiCorp's canonical formatting rules.
It normalizes whitespace, ordering of arguments, line breaks, and other stylistic elements, ensuring a consistent code base without altering the underlying configuration semantics.
The command exits with status0 when changes are applied, making it safe to run as part of CI pipelines or pre-commit hooks.

Because the tool's primary purpose is exactly to standardize Terraform code within the directory you invoke it from, option **A – True** correctly describes its behavior.
Option **B – False** would imply the command does not perform formatting or that it operates differently; however, the official documentation confirms that terraform fmt precisely applies canonical formatting to all files in the target directory, so the false statement does not align with Terraform's documented functionality.

References

Official CLI documentation: https://developer.hashicorp.com/terraform/cli/commands/fmt
Terraform tutorial on formatting: https://developer.hashicorp.com/terraform/tutorials/cli/fmt

Question: 69 CertyIQ

A resource block is shown in the Exhibit space of this page.
resource "kubernetes_namespace" "example"
name = "test"

How would you reference the attribute name of this resource in HCL?

A. resource.kubernetes_namespace.example.name
B. data.kubernetes_namespace.name
C. kubernetes_namespace.example.name
D. kubernetes_namespace.test.name

Answer: C

Explanation:

Technical justification (bullet-point format)

The HCL syntax for referencing a resource attribute is<resource_type>.<resource_name>.<attribute>.

In the excerpt the resource block defines a resource of type **kubernetes_namespace** with the local name **example**.
Therefore the attribute **name** is accessed as **kubernetes_namespace.example.name** – option **C**.
OptionA (resource.kubernetes_namespace.example.name) is invalid because resource is not a valid prefix for attribute lookup; only the resource type and local name are used.
OptionB (data.kubernetes_namespace.name) references a data source, which does not exist in the given snippet and thus cannot resolve the attribute.
OptionD (kubernetes_namespace.test.name) uses the wrong local name (**test** vs **example**) that was declared in the block.

References

HashiCorp Configuration Language (HCL) Resource addressing:
https://developer.hashicorp.com/terraform/language/syntax#resource_addresses
Terraform Namespacing and naming conventions:
https://developer.hashicorp.com/terraform/language/modules/developing#naming-scope

Question: 70 CertyIQ

You need to deploy resources into two different regions in the same Terraform configuration using the block in the Exhibit space on this page. provider “aws” region = “us-east-1”

provider “aws”
region = “us-west-2”

What do you need to add to the provider configuration to deploy the resource to the us-west-2 AWS region?

A. resource “aws_instance” “example-us-west-2”
ami = data.aws_ami.ubuntu.id
instance_type = “t3.micro”

B. provider “aws”
region = “us-east-1”

provider “aws” “west”
region = “us-west-2”

C. provider “aws_west”
region = “us-west-2”

D. provider “aws”
region = “us-east-1”

provider “aws”
alias = “west”
region = “us-west-2”

Answer: D

Explanation:

Technical justification

To reference multiple AWS providers within a single configuration, each provider must be given a unique **alias** and its own region setting.

The original default provider (provider "aws" region = "us-east-1") continues to serve the us-east-1 region.

Adding a second provider with an alias (e.g., "west") allows Terraform to switch to a different region (us-west-2) when that alias is used in resource blocks, without affecting the original provider.

Option D correctly implements this pattern:
provider "aws" alias = "west" region = "us-west-2"

The alias can then be applied to a resource, e.g.,
resource "aws_instance" "example-west" provider = aws.west … .

Why the other options are unsuitable

A – Defines a resource block that references data.aws_ami.ubuntu.id but does not specify a provider at all; Terraform would default to the first (us-east-1) provider, so the resource would not be deployed to us-west-2.
B – Attempts to add a new provider block without an alias, which would create a duplicate default provider and cause a configuration error (multiple providers with the same name).
C – Uses an invalid provider type (aws_west) that does not match the required syntax (aws), leading to a parse error.

References

Provider Configuration Syntax – https://developer.hashicorp.com/terraform/language/providers/config
Using Provider Aliases – https://developer.hashicorp.com/terraform/language/resources/providers#using-aliases-for-multiple-providers-of-the-same-type

Question: 71 CertyIQ

Two resources blocks are shown in the Exhibit space on this page: azurerm_linux_web_app, and azurerm_role_assignment. When provisioned, the web app will use the role assignment during creation, so the role assignment must be created first.

```
resource "azurerm_linux_web_app" "app" {
  name = "example-app"
  resource_group_name = azurerm_resource_group.rg.name
  location = azurerm_resource_group.rg.location
  service_plan_id = azurerm_service_plan.plan.id

  identity {
    type = "UserAssigned"
    identity_ids = [azurerm_user_assigned_identity.app.id]
  }
}
resource "azurerm_role_assignment" "kv_access" {
  scope = azurerm_key_vault.kv.id
  role_definition_name = "Key Vault Secrets User"
  principal_id = azurerm_user_assigned_identity.app.principal_id
}
```

How do you ensure the azurerm_role_assignment resource is created first?

A. Add a depends_on argument to the azurerm_linux_web_app.
B. Change the order of the azurerm_linux_web_app and azurerm_role_assignment blocks.
C. Add a create_before_destroy argument to the azurerm_role_assignment.
D. Add a count argument to both resources.

Answer: A

Question: 72 CertyIQ

A Terraform local value can reference other Terraform local values.

A. True
B. False

Answer: A

Explanation:

Answer: A.True

Technical justification

Terraform's locals block defines a set of key-value pairs that act as a namespace for intermediate values. These values are evaluated before any resource or provider configuration is processed, and they follow the same reference rules as variables: a local can read the value of another local that has been declared earlier in the same block, or that depends on it.
The Terraform engine builds a dependency graph, so circular references that are not self-contained are rejected, but straightforward cross-references (e.g., local.instance_name = "app-$ local.environment ") are fully supported.
This capability is deliberately designed to let you compute derived values once and reuse them throughout the configuration, making the configuration more maintainable and less error-prone.

Why the alternative (B) is less suitable

OptionB ("False") incorrectly asserts that locals cannot reference one another. In reality, Terraform explicitly allows such references, and the language reference explicitly describes this behavior.
Choosing B would contradict both the language semantics and the official documentation, leading to an incorrect answer on the exam.

References

Terraform Documentation – Locals: https://developer.hashicorp.com/terraform/language/expressions/locals
Terraform Registry Provider Reference – locals block:
https://developer.hashicorp.com/terraform/language/resources-local-expressions#locals-block

Question: 73 CertyIQ

You want to define multiple data disks as nested blocks inside the resource block for a virtual machine.
What Terraform feature would help you define the blocks using the values in a variable?

A. Count arguments
B. Collection functions
C. Local values
D. Dynamic blocks

Answer: D

Explanation:

Justification

Dynamic blocks allow a resource argument (such as disk) to be generated repeatedly from a variable list, enabling each disk to be expressed as a separate nested block without manually writing repetitive code.
By iterating over a collection (e.g., var.disks), Terraform produces a block instance for each element, preserving the hierarchical structure expected by the provider.
This approach integrates cleanly with for_each or indexed collections, keeping configuration DRY and making the VM definition scalable when the number of disks varies.

Why the other choices are less appropriate

Count arguments – only support a single numeric count and create multiple resource instances, not nested blocks with arbitrary attributes; they cannot dynamically vary block parameters per element.
Collection functions – are utility functions for manipulating collections; they help generate data but do not provide a Terraform mechanism for rendering nested blocks.
Local values – store computed values internally for reuse within a module but cannot influence resource configuration structures such as nested blocks.

References

Dynamic Blocks: https://developer.hashicorp.com/terraform/language/resources/dynamic-blocks
for_each and nested resources: https://developer.hashicorp.com/terraform/language/count-arguments#for_each

---

Question: 74 CertyIQ

You are creating a reusable Terraform configuration and want to include an optional billing_dept tag so your Finance team can track team-specific spending on resources.
Which of the following billing_dept variable declarations will achieve this?

A. variable “billing_dept”
default = “”

B. variable “billing_dept”
type = optional(string)

C. variable “billing_dept”
optional = true

D. variable “billing_dept”
type = default

Answer: A

Explanation:

The correct declaration is **A – variable "billing_dept" default = ""** because it defines a string-typed variable and assigns an empty string as the default value. An empty default makes the argument optional (i.e., it may be omitted when the variable is used), while still preserving the declared type. This is the idiomatic Terraform pattern for optional string variables.

Why A is correct:
• Uses the standard default attribute to provide a fallback value.
• The empty string ("") signals that the variable is optional but always returns a string.
• Terraform treats any variable without a supplied value as having the default value, fulfilling the Finance team's requirement to tag resources optionally.

Why the other options fail:
• **B – type = optional(string)** – The optional keyword is not a valid argument for the type attribute; Terraform does not support a type = optional(...) syntax.
• **C – optional = true** – optional is not a recognized argument within a variable block; only default, type, and a few other meta-arguments are allowed.
• **D – type = default** – type must be a specific type such as string, number, or any, not the keyword default; placing default here is syntactically invalid.

Thus, only option A follows the correct Terraform syntax and semantics for an optional billing department tag.

References

https://developer.hashicorp.com/terraform/language/variables/definition#default
https://developer.hashicorp.com/terraform/language/variables/definition#type

Question: 75 CertyIQ

Part of a configuration is shown in the Exhibit space of this page. data "vsphere_datacenter" "dc" resource "vsphere_folder" "parent" path = "Production" type = "vm" datacenter_id = _____

You want to pass the id of the vsphere_datacenter data source to the datacenter_id argument of the vsphere_folder resource.
Which reference would you use?

A. data.vsphere_datacenter.dc
B. data.vsphere_datacenter.dc.id
C. vsphere_datacenter.dc.id
D. data.dc.id

Answer: B

Explanation:

Justification

Option B – data.vsphere_datacenter.dc.id

The vsphere_datacenter data source returns a map that contains the attribute id, which uniquely identifies the datacenter object in vSphere.
To satisfy the datacenter_id argument of the vsphere_folder resource, you must supply the ID (a string/number)

rather than the whole data source reference.
Therefore, the expression data.vsphere_datacenter.dc.id correctly extracts that identifier and passes it to the resource.

Option A – data.vsphere_datacenter.dc

This returns the entire data object, which includes numerous attributes (e.g., mo_id, name, parent).
The vsphere_folder data source expects a **scalar** value for datacenter_id; passing a complex map causes a type-mismatch error.

Option C – vsphere_datacenter.dc.id

This refers to a local variable or resource name named vsphere_datacenter.
No such variable is declared in the snippet; only a **data source** named vsphere_datacenter exists. Using the raw type name without the data. prefix would result in an undefined reference.

Option D – data.dc.id

The identifier dc is the label (or alias) of the data source, not its type name.
The correct syntax to reference the data source is data.vsphere_datacenter.dc. Without the leading data. the reference is ambiguous and will be ignored by the HCL parser.

Hence, the only expression that both accesses the data source correctly and extracts its id attribute is **data.vsphere_datacenter.dc.id**.

References

Terraform vSphere Provider – Data Sources: vsphere_datacenter
https://registry.terraform.io/providers/hashicorp/vsphere/latest/docs/data-sources/datacenter

Terraform vsphere_folder Resource – datacenter_id argument
https://registry.terraform.io/providers/hashicorp/vsphere/latest/docs/resources/folder#datacenter_id

Question: 76 CertyIQ

Your Terraform configuration declares a variable, you want to enforce that its value meets your specific requirements, and you want to block the Terraform operation if it does not.
What should you add to your configuration?

A. Add a top-level check block.
B. Add a validation block to the variable block.
C. Add a top-level validation block.
D. Add a check block to the variable block.

Answer: B

Explanation:

Justification

Option B – Add a validation block to the variable declaration

The validation meta-argument (available from Terraform0.12+) lets you define a custom validation rule using condition and an informative error_message.
When the condition evaluates to false, Terraform aborts the plan/apply with the supplied error, effectively "blocking" the operation.
The block is placed **inside** the variable resource, making the rule scoped precisely to that variable.

Why the other options are unsuitable

A. Top-level check block – check is a generic testing construct used in provisioners or in the Terraform Cloud/Enterprise "policy as code" feature; it cannot be used directly in a variable block, nor does it provide a standard validation message.
C. Top-level validation block – A top-level validation block is not a supported syntax; validation blocks must be nested under specific resources (e.g., variable, resource, locals). Placing it at the root would cause a parsing error.
D. check block to the variable block – check is not a valid attribute for a variable; it belongs to resource-level testing or policy checks, and Terraform will reject it as an illegal nested block.

Conclusion
Only a validation block **inside** the variable declaration satisfies the requirement of enforcing a specific value and halting execution when the condition is not met.

References

Variable Validation – HashiCorp Documentation
Validation Blocks – HashiCorp Documentation

Question: 77 CertyIQ

Running terraform fmt without any flags in a directory with Terraform configuration files will check the formatting of those files, but will never change them.

A. True
B. False

Answer: B

Explanation:

Key Points

terraform fmtwithout flags **re-writes** the files to apply the recommended formatting.
The command is designed to modify the files in-place; it does not perform a read-only check.
If you want only a diagnostic check (no changes), you must add the -check flag, which makes the operation return a non-zero exit status when files need formatting but leaves them untouched.

Why OptionB (False) is correct

The statement claims that terraform fmt will never change files. This is inaccurate because the default behavior is to **format and save** the files, thereby altering them.

Why OptionA (True) is not suitable

It misrepresents the command's effect. While terraform fmt -check can be used to verify formatting without

modification, the plain terraform fmt always applies formatting changes when needed.

Technical Summary

Default execution: reads each .tf file, rewrites it with canonical formatting, and writes the changes back to disk.
Exit code: 0 on success, non-zero only when files are unchanged.
Without -check, Terraform treats the operation as a transformation step, making it effectively a **mutating** command.

References

Terraform CLI – fmt command: https://developer.hashicorp.com/terraform/cli/commands/fmt
Formatting and checking files: https://developer.hashicorp.com/terraform/language/formatting

Question: 78 CertyIQ

You want to retrieve the ID of an existing cloud resource for use in your Terraform configuration.
Which block type would you use?

A. resource
B. import
C. data
D. output

Answer: C

Explanation:

Explanation

To reference an existing cloud resource without creating it, Terraform uses **data sources**, which are declared with the **data** block. This block retrieves the resource's attributes (such as its identifier) so they can be used elsewhere in the configuration, for example in output variables or other resources.

resource block creates a new resource; it cannot retrieve an already-existing one.
import is a command-line operation that brings an existing resource under Terraform management but is not a configuration block.
output only exposes values after they have been computed; it does not fetch external resource data.

Therefore, the appropriate block to **retrieve the ID of an existing cloud resource** is the data block.

Why the other options are unsuitable

A. resource – creates new resources; it does not read existing ones.
B. import – a CLI command, not a block used inside the configuration file.
D. output – only publishes values generated by Terraform; it cannot query external resources directly.

References

Terraform Data Sources: https://developer.hashicorp.com/terraform/language/data-sources
Data Blocks Overview: https://developer.hashicorp.com/terraform/language/resources/data

Question: 79 CertyIQ

Which of these commands makes your code more human-readable?

A. terraform show
B. terraform fmt
C. terraform output
D. terraform validate

Answer: B

Explanation:

Technical Justification for Correct Answer: B. terraform fmt

The correct option for making your code more human-readable is **B. terraform fmt**. Here's why:

B. terraform fmt: This command automatically formats your Terraform configuration files to adhere to the standard Terraform style guide. It ensures consistent indentation, spacing, and formatting, making the code significantly more readable for humans. By enforcing a uniform style, terraform fmt enhances code readability, maintainability, and collaboration among team members.

Why Other Options Are Less Suitable:

A. terraform show: This command displays the current state of your infrastructure resources in a human-readable format, but it does **not** modify your configuration files to improve their readability. Its purpose is more about inspecting the current state of resources rather than enhancing the readability of your Terraform code.

C. terraform output: Similar to terraform show, this command is used to display values from infrastructure after apply, based on output blocks in your configuration. It has no impact on the readability of your Terraform configuration files themselves.

D. terraform validate: Although crucial for ensuring your Terraform configuration is valid and follows the language rules, terraform validate (or the newer terraform plan with validation capabilities) does **not** alter the formatting or readability of your code. Its primary function is to check for errors, not to improve code aesthetics or readability.

References:

HashiCorp Terraform Documentation - terraform fmt
HashiCorp Terraform Documentation - Commands Overview (for context on other commands)

Question: 80 CertyIQ

You want to split a large main.tf file into multiple files for better organization within the same working directory. What impact should you expect when you run a terraform plan?

A. The terraform plan will fail since variable and provider blocks must appear before resource blocks.
B. Significant changes should be expected since moving resources between files can trigger a destroy/replacement.

C. No impact since Terraform reads all .tf files as a single configuration when executing a terraform plan.

D. Only minor impacts since Terraform parses the main.tf file first, which could change the order in which resources are created.

Answer: C

Explanation:

Technical Justification for Correct Answer C

The correct answer is **C. No impact since Terraform reads all .tf files as a single configuration when executing a terraform plan**. Here's why:

Terraform Configuration Aggregation: When running terraform plan (or any Terraform command), Terraform aggregates all files with the .tf extension in the working directory into a single configuration set. This means that regardless of how you split your configuration across multiple .tf files, Terraform treats them as a unified set of configurations.
Order of Evaluation, Not File Structure: While it's true that certain blocks (like provider and variable definitions) need to be evaluated before resources can be planned, this is based on the logical dependency graph within the aggregated configuration, not the file structure. As long as the split files maintain the necessary logical order of definitions (e.g., providers and variables before resources that depend on them), there will be no issue.
Resource Tracking by ID, Not File Location: Terraform tracks resources by their ID (typically a combination of the resource type and a unique identifier within that type) in the state file, not by the file in which they were defined. Therefore, moving a resource block from one file to another within the same directory does not change its identity in Terraform's state, leading to **no expectation of destroy/replacement triggers solely due to file reorganization**.

Why Other Options Are Less Suitable:

A. Terraform plan will fail since variable and provider blocks must appear before resource blocks.

Incorrect: While the statement about the order of blocks is technically correct (providers and variables should be defined before resources that use them), Terraform's aggregation of all .tf files into one configuration set means this order is considered across all files, not per file. A fail would occur if the aggregated config violates this rule, but not merely from splitting files.

B. Significant changes should be expected since moving resources between files can trigger a destroy/replacement.

Incorrect: The trigger for destroy/replacement is a change in the resource's configuration or its dependencies, not the physical location of the resource block within the configuration files.

D. Only minor impacts since Terraform parses the main.tf file first, which could change the order in which resources are created.

Incorrect: The parsing order (if any slight preference exists in practice) does not significantly impact the planning outcome, as the final plan is based on the dependency graph, not the parsing order of files. The term "minor impacts" is misleading in this context, as the expected outcome is actually **no impact** on the plan due to file splitting.
References

HashiCorp Terraform Documentation - Configuration Files
[HashiCorp Terraform Documentation - Resource Dependencies]
(https://www.terraform.io/docs/language/meta-arguments depreciation.html#depends_on)

Question: 81 CertyIQ

You need to deploy resources into two different regions in the same Terraform configuration. To do this, you declare multiple provider configurations as shown in the Exhibit space on this page.
provider “aws”
region = “us-east-1”

provider “aws”
alias = “west”
region = “us-west-2”

What meta-argument do you need to configure in a resource block to deploy the resource to the us-west-2 AWS region?

A. provider = west
B. provider = aws.west
C. alias = aws.west
D. alias = west

Answer: B

Explanation:

Technical Justification for Correct Answer: B

To deploy resources across multiple regions in the same Terraform configuration, declaring multiple provider configurations with aliases (as shown) is the correct approach. The key step is correctly referencing the aliased provider in the resource block to specify the target region.

Why B (provider = aws.west) is Correct:

When using multiple provider configurations with aliases, Terraform requires the provider meta-argument in the resource block to specify which provider configuration to use.
The syntax provider = aws.west directly references the named provider configuration by its alias (west) within the aws provider namespace, ensuring the resource is deployed to the us-west-2 region as configured.
This syntax is in line with Terraform's documentation for using multiple providers with aliases.

Why Other Options are Less Suitable:

A (provider = west): Incorrect syntax. Does not specify the provider namespace (aws), which could lead to ambiguity if multiple providers (e.g., AWS, Azure) have an alias named west.
C (alias = aws.west): Misuse of the alias meta-argument. The alias is used in the provider configuration to define an alias, not in the resource block to select a provider.
D (alias = west): Similar to C, this misuses the alias meta-argument in the resource block. Additionally, it lacks the provider namespace specification.

Correct Configuration Example for a Resource in us-west-2:

Provider configurations (as given)provider "aws" region = "us-east-1" provider "aws" alias = "west" region = "us-we

References:

Terraform Documentation: Multiple Provider Configurations
Terraform Documentation: Aliases with Multiple Provider Configurations

Question: 82 CertyIQ

A resource block is shown in the Exhibit space on this page.
resource “google_compute_instance” “main”
name = “test”

What is the Terraform resource name of the resource block?

A. test
B. compute_instance
C. google
D. main

Answer: D

Explanation:

Technical Justification for Correct Answer: D. main

The correct Terraform resource name of the given resource block is **D. main**. Here's why:

Resource Block Syntax: In Terraform, a resource block is defined with the syntax resource "TYPE" "NAME" ... . In this case, the syntax breaks down into:

resource: Keyword indicating a resource block.
"google_compute_instance": The **TYPE** of the resource (a Google Compute Instance).
"main": The **NAME** of the resource (assigned by the user for reference within the Terraform configuration).
Resource Name vs. Attribute Name: The **resource name** (in this context) refers to the user-assigned name within the resource block ("main"), not to be confused with the name attribute ("test") which is a property of the resource that sets the actual name of the created Google Compute Instance.

Why Other Options are Less Suitable:

A. test: This is the value of the name attribute within the resource block, which sets the name of the Google Compute Instance itself, not the Terraform resource name.
B. compute_instance: Part of the **TYPE** (google_compute_instance), indicating the type of resource, not the name of the resource block.
C. google: Also part of the **TYPE** (google_compute_instance), specifying the provider, not the resource block's name.

References

For deeper understanding of Terraform's resource block syntax and configuration:

1. Terraform Documentation - Resource Syntax
2. Terraform Documentation - Resource Arguments

Question: 83 CertyIQ

A variable declaration is shown in the Exhibit space on this page.
variable “sizes”
type = list(string)

description = “Valid server sizes”
default = [“small”, “medium”, “large”]

Which is the correct way to get the value of medium from this variable?

A. var.sizes[0]
B. var.sizes[1]
C. var.sizes[2]
D. var.sizes[3]

Answer: B

Explanation:

Technical Justification for Correct Answer (B)

The variable "sizes" is declared with the type list(string) and has a default value set to ["small", "medium", "large"]. To access the value "medium" from this list, one must understand how indexing works in Terraform (the HashiCorp tool implied in the context of this question). In Terraform, like in many programming languages, list indexing starts at **0**. This means the first element of the list is at index 0, the second at index 1, and so on.

Correct Answer (B) - var.sizes[1]: Since "medium" is the second element in the list ["small", "medium", "large"], it is correctly accessed using the index **1**. This aligns with zero-based indexing principles.

Why Other Options Are Less Suitable:

A. var.sizes[0]: This would return "small", the first element of the list, not "medium". Incorrect for the purpose of retrieving "medium".
C. var.sizes[2]: This accesses "large", the third element in the list. While this is a valid operation, it does not retrieve "medium" as required.
D. var.sizes[3]: This option attempts to access an index that is out of range for the given list (since the list only has three elements, indexed 0 through 2). In Terraform, this would result in an error, as the index 3 does not exist for this variable.

References

For further understanding of Terraform's data types and how to work with lists, refer to the official HashiCorp Terraform documentation:

Terraform Configuration Basics: Data Types
Terraform Variables: Using Lists (Specifically, look for examples on list handling)

Question: 84 CertyIQ

Your configuration contains a module block that references a module from the Terraform registry, and sets the version argument to 1.0. You just published a new version of the module and updated your configuration to point to version 1.1.
Which command must be run to install the new version?

A. terraform apply
B. terraform init
C. terraform plan

D. terraform modules

Answer: B

Explanation:

Justification

terraform init is the command that resolves and installs module versions from the registry. When the version argument in a module block is changed (e.g., from 1.0 to 1.1), the new version is fetched only after a re-initialization of the working directory. Running terraform init (optionally with -upgrade) tells Terraform to download the new module version that satisfies the updated constraint.

Why the other options are incorrect:

A – terraform apply provisions or updates infrastructure resources; it does not download or install new module code.
C – terraform plan generates an execution plan but does not interact with the module source registry.
D – terraform modules is not a valid Terraform CLI command.

Therefore, the correct command to fetch and install the newly released module version is **B – terraform init** (with -upgrade recommended to ensure the new version is retrieved).

References

Terraform CLI Init Command –https://developer.hashicorp.com/terraform/cli/init
Modules and Version Constraints –
https://developer.hashicorp.com/terraform/language/modules/develop/src

Question: 85 CertyIQ

If one of your modules uses a local value, you can expose that value to callers of the module by defining a terraform output in the module's configuration.

A. True
B. False

Answer: A

Explanation:

Correct answer: A – True

A module can reference its own local variables only inside the module; to make those values usable by its callers, the module must declare a corresponding output attribute that returns the value.
When the calling configuration references the module, the output is resolved and can be assigned to a variable, used in other resources, or passed to other modules.
This mechanism is the standard way to expose module-scoped information; without an output, a local value remains internal and invisible to the caller.

Why other options are unsuitable

The question only provides a binary choice; “False” would imply you cannot expose local values via outputs, which contradicts both the design of Terraform modules and the official documentation.
Any justification that ignores the requirement to declare an output attribute would be incomplete and inaccurate for a certification context.

References

Exposing module outputs: https://developer.hashicorp.com/terraform/language/modules/develop/outputs
Using outputs in calling configurations: https://developer.hashicorp.com/terraform/language/modules/usage#exposing-module-outputs

Question: 86 CertyIQ

If a module declares a variable without a default value, you must pass the value of the variable within the module block when you call the module in your configuration.

A. True
B. False

Answer: A

Explanation:

Answer: A – True

Terraform requires all input variables to be defined with a value before the module is instantiated.
If a variable is declared without a default value, the configuration will not compile unless the caller explicitly passes a value for that variable in the module block.
Omitting the required argument leads to a validation error (Variable "xxx" is required), preventing the plan/apply from proceeding.

Why B is incorrect:

Statement B claims that passing the value is optional; this contradicts Terraform’s strict variable-requirement semantics, which mandate that any non-default variable must be supplied.

References

Variables in Terraform
Module Call Syntax

Question: 87 CertyIQ

When you run terraform apply, the Terraform CLI will print output values from both the root module and any child modules.

A. True
B. False

Answer: B

Explanation:

Answer: B.False

Technical justification

The terraform apply command executes a plan and then provisions any resources that differ from the desired state. It prints a summary of the actions it will take (e.g., “Created instance ...”) and any error or warning messages, but it **does not automatically render output values** (output blocks) from the root or any child modules.
Output values are only emitted when you explicitly invoke the **terraform output** command after a successful apply (or terraform apply -json for machine-readable output).
To make module outputs visible in the root module, they must be re-exposed through root-module output declarations, which is an explicit step and not the default behavior of apply.
Therefore, the statement that terraform apply prints output values from both the root and child modules by itself is inaccurate.

References

Terraform CLI documentation – apply
https://developer.hashicorp.com/terraform/cli/commands/apply

Terraform CLI documentation – output
https://developer.hashicorp.com/terraform/cli/commands/output

These links confirm that apply does not display output values automatically and that separate commands or explicit output definitions are required to view them.

Question: 88 CertyIQ

Which type of information does the Terraform Registry provide about the modules it hosts?

A. Required input variables
B. Outputs
C. Optional input variables and default values
D. All of these are provided

Answer: D

Explanation:

The Terraform Registry publishes a **module page** that enumerates all the metadata required for consumption:
• **Required input variables** – listed to indicate mandatory parameters.
• **Optional input variables and their default values** – shown with defaulting behavior.
• **Outputs** – listed to give users a clear view of what the module returns.
Because these three pieces of information are all presented together on the same page, the only answer that captures the complete set is **“All of these are provided.”**
Options A and B isolate only a subset (required variables only, or outputs only), which omits the other equally important details; therefore they are incomplete descriptions of what the Registry supplies.

References

Terraform Registry Module Page Documentation – explains the fields displayed for each module, including required/optional variables and outputs.
Module Requirements & Variables Reference – details the variable definitions and output contracts published by modules in the Registry.

Question: 89 CertyIQ

When you include a module block in your configuration that references a module from the Terraform Registry, the version attribute is required.

A. True
B. False

Answer: B

Explanation:

Justification

The module block may contain a version argument, but it is not mandatory – Terraform will accept the block without a version specifier and will use the latest published version of the module.
Providing a version constraint is only a best-practice measure to lock the module to a specific release or to guard against breaking changes; the Terraform language grammar does not enforce it.
The Registry automatically resolves the most recent stable release when no version is declared, so excluding version is syntactically valid and operationally functional.
However, omitting the version makes the configuration less deterministic and can lead to unexpected upgrades in future runs, which is why the guidance recommends specifying a version, not that it is required.

Why the other option is less suitable

OptionA claims the version attribute is required; this misrepresents the Terraform configuration syntax and would cause an unnecessary validation error if a user simply omitted it.
In exam contexts, precision about optional versus mandatory fields is critical; recognizing that version is optional aligns with the official documentation and prevents misunderstandings about Terraform's parsing rules.

References

Modules – Using Modules from the Terraform Registry:
https://developer.hashicorp.com/terraform/language/modules/develop/source
Version constraints – Terraform Registry module version constraints:
https://developer.hashicorp.com/terraform/language/modules/develop/source#version-constraints

Question: 90 CertyIQ

A child module can always access variables declared in its parent module.

A. True
B. False

Answer: B

Explanation:

Justification

The statement is **False**.
Terraform modules have their own scope; a child module does **not** automatically inherit variables defined in its parent.
Variables are only accessible to a child when the parent explicitly **passes** them via **output values** that are then declared as **input variables** in the child, or when the child receives them as arguments in its module block.
Unpassed variables remain private to the parent and cannot be read by the child, regardless of the hierarchical relationship.
This design enforces encapsulation and prevents unintended coupling between modules, which is a core principle of Terraform's module system.

References

Terraform Modules Documentation – Passing Input Variables:
https://developer.hashicorp.com/terraform/language/modules/develop/input-variables
Terraform Modules Documentation – Output Variables:
https://developer.hashicorp.com/terraform/language/modules/develop/outputs

Correct Option: B.False

Question: 91 CertyIQ

You are updating a child module with the resource block shown in the Exhibit space on this page. The public_ip attribute of the resource needs to be accessible to the parent module. resource "aws_instance" "example" ami = "ami-0a123456789abcdef" instance_type = "t3.micro"

How do you meet this requirement?

A. Add a data source to the parent module.
B. Create a local value in the child module.
C. Add an import block to the parent module.
D. Create an output in the child module.

Answer: D

Explanation:

Why option D is correct

Expose the attribute via an output – The child module can declare a named output that returns public_ip. The parent module can then reference that output directly, making the value available as a first-class variable.
Decouples modules – Using an output keeps the parent module agnostic of the child's internal resources, preserving encapsulation while still sharing needed data.
Terraform semantics – Outputs are the official mechanism for exposing a module's results to its callers; they are resolved after the child plan is applied and can be consumed in any expression within the parent.

Why the other options are not suitable

Add a data source to the parent – A data source reads external data, but it cannot directly expose a resource attribute defined inside another child module without importing state, which adds unnecessary complexity and breaks module boundaries.
Create a local value in the child – A locals block is scoped to the module where it is defined; it does not automatically become visible to the parent. The parent would still need a mechanism to retrieve the value, which an output fulfills more naturally.
Add an import block to the parent – Import blocks are used to bring resources created outside the current state into the current configuration. They do not provide a clean, declarative way to expose the child's public_ip attribute.

Conclusion
Declaring an output in the child module that returns public_ip and referencing that output in the parent is the idiomatic, maintainable way to share the attribute across modules.

References

Modules: https://developer.hashicorp.com/terraform/language/modules/develop
Outputs: https://developer.hashicorp.com/terraform/language/modules/develop#outputs

Question: 92 CertyIQ

Your configuration defines the module block shown in the Exhibit space on this page. The web_stack module accepts an input variable named servers. module "web_stack" source = "./modules/web_stack"

Which of the following changes to the module block sets the servers variable to the value of "3"?

A. module "web_stack"
source = "./modules/web_stack"
servers = 3

B. module "web_stack"
source = "./modules/web_stack"
inputs.servers = 3

C. module "web_stack"
source = "./modules/web_stack"
inputs =
servers = 3

D. module "web_stack"
source = "./modules/web_stack"
var.servers = 3

Answer: A

Explanation:

Justification

OptionA – servers = 3 is a valid attribute directly on the module block. When a module declares an input variable named servers, assigning servers = 3 in the module call passes the value 3 to that variable. This syntax follows the exact form required by Terraform.

OptionB – inputs.servers = 3 is invalid because the module block does not accept a top-level inputs attribute; module arguments are specified as separate attribute-value pairs, not as a dotted-notation map.

OptionC – inputs = servers = 3 incorrectly uses inputs as the argument name and wraps the assignment in

braces. Terraform expects each argument to be a simple attribute, not a map keyed by inputs.

OptionD – var.servers = 3 references a var namespace that does not exist in a module block. Variables are passed in, not defined or mutated via var. inside the block.

Therefore, only **OptionA** correctly sets the servers input to 3.

References

References

Module block syntax – Terraform Language:
https://developer.hashicorp.com/terraform/language/modules/requirements
Passing input variables to modules – Terraform Documentation:
https://developer.hashicorp.com/terraform/language/modules/sources

Question: 93 CertyIQ

Your team is using a version 3.1.4 of a module from the public Terraform Registry, and they're worried about possible breaking changes in future versions of the module.
Which version argument should you add to the module block to prevent newer versions from being used?

A. version = "3.1.4"
B. version = "< 3.2"
C. version = "!=3.1.5"
D. version = "~>3.1.4"

Answer: D

Explanation:

Why optionD is the correct choice

The constraint ~>3.1.4 expands to "greater than or equal to3.1.4, but less than3.2.0".
This means the module can only use patch releases (e.g.,3.1.5) until the next minor version (3.2.x) is reached, effectively **preventing any newer minor or major version from being pulled** while still allowing safe patch upgrades.
It matches the team's intent to "lock the module to the current 3.1.x line" without unnecessarily restricting older versions.

Why the other options are unsuitable

A – version = "3.1.4" pins the module to that exact string. Any later patch or minor release (e.g.,3.1.5) would be rejected, which may be overly restrictive if patch upgrades are desired. It also does not express the typical semantic-versioning intent of "use this series only".
B – version = "< 3.2" allows any version prior to 3.2, including older major releases such as2.x. This could bring in unexpected breaking changes from previous major versions, contradicting the goal of staying on a known 3.1.x level.
C – version = "!=3.1.5" only excludes a single patch version; it does **not** stop newer minor or major releases like3.2.0 from being used, so it fails to limit future breaking changes.

Hence, ~>3.1.4 is the precise, industry-standard way to restrict usage to the current minor series while still permitting safe patch updates.

References

Module version constraints – HashiCorp Terraform documentation:
https://developer.hashicorp.com/terraform/language/modules/sources#version-constraints
Semantic versioning constraints (~> notation) – HashiCorp Terraform documentation:
https://developer.hashicorp.com/terraform/language/version-constraints

Question: 94 CertyIQ

Which of the following locations can Terraform use as a private source for modules? (Choose two.)

A. Public repository on GitHub.
B. Internally hosted VCS (Version Control System) platform.
C. Public Terraform Registry.
D. Private repository on GitHub.

Answer: BD

Explanation:

Technical justification

Internally hosted VCS platform (B) – Terraform can reference a module stored in any Git-compatible VCS that is reachable only inside the organization's network (e.g., an internal Git server). Because the repository is not publicly exposed, it meets the definition of a private source. Access is typically provided via SSH or HTTPS with credentials, allowing Terraform to clone the exact commit or tag required for the configuration.

Private repository on GitHub (D) – A GitHub repository that is owned by an organization or user and has its visibility set to private can also be used as a module source. Terraform fetches the module using the same Git URL scheme, but the request includes authentication (personal access token, SSH key, or OAuth) that proves the caller is authorized. Since the repo is not publicly reachable, it qualifies as a private location for modules.

Public repository on GitHub (A) – not private – Although a public GitHub repo can be used as a module source, its contents are openly accessible on the internet. The defining characteristic of a private source is that it is reachable only by authorized entities; a publicly reachable repository fails this requirement.

Public Terraform Registry (C) – not private – The public registry hosts community-maintained modules that anyone can publish and consume. By definition it is a public source; Terraform treats it as an open registry and does not consider it a private location for module storage.

References

Module source types and private source authentication:
https://developer.hashicorp.com/terraform/language/modules/sources
Managing private modules in the Terraform Registry:
https://developer.hashicorp.com/terraform/registry/private-modules

Question: 95 CertyIQ

Your configuration defines the module block shown in the Exhibit space on this page.

```
module "production" {
source = "./modules/web_stack"
}
```

This module declares an output named hostnames.
How do you access the value of this output?

A. production.hostnames
B. module.production.hostnames
C. web_stack.hostnames
D. module.web_stack.hostnames

Answer: B

Explanation:

B. module.production.hostnames

In **Terraform**, outputs from a module are accessed using the syntax:

module.<module_name>.<output_name>

If the module block is defined as:

module "production" source = "./web_stack"

and the module contains:

output "hostnames" value = ...

then you reference the output as:

module.production.hostnames

Question: 96 CertyIQ

Which argument can you set on a module block to prevent Terraform from updating the module's configuration during an init or get operation?

A. count
B. lifecycle
C. source
D. version

Answer: D

Explanation:

Why the correct answer is D – version

The version argument on a module block pins the module to a specific released version (e.g., =1.2.3 or ">=1.0.0, <2.0.0").
When Terraform runs **terraform init** or **terraform get**, it resolves and downloads only that exact version, never automatically upgrading to a newer one. This stops Terraform from silently updating the module configuration during those operations.
Because the version is locked, Terraform treats the module as immutable with respect to version selection, preventing unintended updates.

Why the other options are unsuitable

A. count – count is used on resource definitions to create multiple instances; it is **not** a module-level argument.
B. lifecycle – A lifecycle block controls resource-level behavior (e.g., create_before_destroy, prevent_destroy) but cannot be applied to a module block, so it has no effect on version resolution during init/get.
C. source – source points to the module's address (repository or registry) but does **not** constrain version selection; Terraform will still fetch the latest matching version unless a specific version constraint is given.

Thus, only the version argument can directly prevent Terraform from updating a module's configuration during init or get.

References

Terraform Module Blocks – Version Constraints:
https://developer.hashicorp.com/terraform/language/modules/develop/src#version-constraints
Module Versioning and Dependency Management:
https://developer.hashicorp.com/terraform/language/modules/develop/sources#versioning

Question: 97 CertyIQ

You are writing Terraform configuration that calls a child module. This child module declares an output value. How can you ensure that Terraform will print out this value when you run Terraform CLI commands such as terraform apply?

A. Nothing. Terraform will automatically print out all output values from child modules.
B. Nothing. Terraform cannot use output values from child modules.
C. Declare a new output in your root configuration that references the module's output.
D. None of these will print the value.

Answer: C

Explanation:

Technical Justification for Correct Answer: C

The correct option to ensure Terraform prints out the output value from a child module when running Terraform CLI commands (like terraform apply) is **C. Declare a new output in your root configuration that references the module's output**. Here's why:

Why C is Correct

Explicit Output Declaration Requirement: Terraform requires explicit declaration of outputs in the root module configuration to display values after an operation like apply. Child module outputs are not automatically exposed to the root level for display.

Reference and Re-export: By declaring a new output in the root configuration that references the child module's output (e.g., value = module.child_module.output_name), you effectively "re-export" the value, making it visible at the root level and thus printed by Terraform after applying changes.

Why Other Options are Less Suitable

A. Nothing. Terraform will automatically print out all output values from child modules.

Incorrect: Terraform does not auto-print child module outputs at the root level without explicit configuration. This would lead to unexpected behavior if relied upon.

B. Nothing. Terraform cannot use output values from child modules.

Misleading: While the statement about printing is incorrect (as it implies a capability issue rather than a configuration one), the core issue is about visibility, not usability. Outputs from child modules can be used (e.g., for resource configuration), just not automatically displayed.

D. None of these will print the value.

Incorrect in Context of Available Choices: Given that option C provides a valid method to achieve the desired outcome, this statement is false in the context of the question.

Example for Clarity (Implicit in Justification but Explicit for Understanding)

In your **root module configuration**:

```
module "child_module"    # ... module source and variables ... output "child_module_output"    value = module.child_modu
```

In your **child_module/main.tf**:

```
# ... resource definitions ...output "child_output_value"    value = "Value to be printed"
```

References

HashiCorp Terraform Documentation - Output Values
HashiCorp Terraform Documentation - Modules

Question: 98 CertyIQ

You need to destroy all of the resources in your Terraform workspace, except for aws_instance.ubuntu[1], which you want to keep.
How can you tell Terraform to stop managing that specific resource without destroying it?

A. Change the value of the count argument on the resource.
B. Remove the resource block from your configuration.
C. Run terraform state rm aws_instance.ubuntu[1].
D. Use a moved block.

Answer: C

Explanation:

Correct answer: C –Runterraform state rm aws_instance.ubuntu[1]

Why this is the best choice

terraform state rm detaches a resource instance from the state file without deleting it from the underlying infrastructure. The resource stays provisioned, but Terraform treats it as “orphaned” and will no longer try to

manage or destroy it.
This is the only option that implicitly removes the dependency while preserving the actual AWS EC2 instance, allowing you to retain aws_instance.ubuntu[1] for future manual operations.

Why the other options are unsuitable

A – Modify the count argument: Changing count only affects how many instances Terraform expects to create; it does not remove a specific indexed resource from the state, so the instance would either be recreated or left with an inconsistent index.
B – Remove the resource block: Deleting the block from the configuration and running terraform apply would cause Terraform to destroy the resource because it would think the resource is missing. The instance would be removed from the infrastructure.
D – Use a move block: The move argument is used to tell Terraform to treat a resource as a new identity; it does not stop management, it merely renames/moves the address in state. It does not prevent future destruction or updates.

References

Terraform State Command – rm
Resource Lifecycle – delete vs orphaned

Question: 99 CertyIQ

When you run terraform apply -refresh-only, which of the following is not consulted by Terraform to update the state file?

A. Authentication credentials for the provider.
B. Terraform configuration files defining the resources.
C. The actual infrastructure as reported by the provider API.
D. The most recent state file.

Answer: B

Explanation:

Why optionB is the correct answer

OptionB – Terraform configuration files defining the resources
Terraform does not re-parse the .tf files when it runs *apply -refresh-only*. The command's sole purpose is to update the state file by reconciling the provider's current view of the infrastructure with the state that is already stored. It therefore does not need to read or evaluate the configuration language itself.

OptionA – Authentication credentials for the provider
The refresh operation must call the provider's API, which requires valid credentials (e.g., API tokens, access keys). Those credentials are consulted to establish the connection.

OptionC – The actual infrastructure as reported by the provider API
Refresh actively queries the live infrastructure through the provider, compares the returned attributes with the existing state, and updates the state accordingly. This step is essential.

OptionD – The most recent state file
The current state file is loaded at the start of the refresh process; its contents are read and then overwritten

with the refreshed information. Hence the existing state file is definitely consulted.

References

1. Terraform CLI - Refresh Mode
2. Provider Authentication Overview

Question: 100 CertyIQ

When declaring a variable, setting the sensitive argument to true will prevent the value from being stored in the state file.

A. True
B. False

Answer: B

Explanation:

B

The sensitive meta-argument in Terraform marks a variable's value as confidential, which prevents the value from being displayed in logs, console output, or the UI. However, the value **is still stored in the state file**; Terraform does not encrypt or omit it from the state. Consequently, the statement that setting sensitive = true will prevent the value from being stored in the state file is inaccurate. The correct answer, therefore, is **False**.

Other options are unsuitable because:

Choosing **True** would imply the value is omitted from the state entirely, which is not how sensitive works.
The question is a straightforward True/False; no additional qualification or nuance is needed.

References

Terraform documentation on sensitive: https://developer.hashicorp.com/terraform/language/meta-arguments/variables#sensitive
State file behavior and sensitive variables:
https://developer.hashicorp.com/terraform/language/state#sensitive-values

Question: 101 CertyIQ

Which are benefits of migrating from a local state backend to a remote backend? (Choose two.)

A. Eliminates the need to manage credentials when deploying infrastructure to multiple cloud providers.
B. State locking that allows multiple team members to safely work on the same infrastructure.
C. Faster plans and apply execution because the state is cached locally on the cloud provider.
D. Guarantees that configuration drift cannot occur for the managed infrastructure.
E. The ability to enable server-side encryption at rest.

Answer: BE

Explanation:

Why optionB and optionE are the correct benefits

B.State locking that allows multiple team members to safely work on the same infrastructure – A remote backend stores the state file on a shared service (e.g., S3, Azure Blob, GCS) that provides built-in state locking. When a lock is held, concurrent operations are serialized, preventing corruption and ensuring that only one apply can modify the state at a time. This is a core reliability feature for collaborative Terraform workflows.

E.The ability to enable server-side encryption at rest – Remote backends can be configured to encrypt the stored state using the encryption capabilities of the underlying storage service (e.g., AWSS3 SSE-KMS, AzureBlob SSE, GCSCMEK). This protects the state file from unauthorized access and complies with security-in-transit-and-at-rest requirements. The encryption is handled by the storage provider, not by Terraform itself.

Why the other options are not correct benefits

A. Eliminates the need to manage credentials when deploying to multiple cloud providers – Remote backends still require credentials to read/write state; they do not remove the credential management burden, especially in multi-cloud scenarios. Credential handling is governed by IAM policies, not by the backend choice.

C. Faster plans and apply execution because the state is cached locally on the cloud provider – State lives on the remote service, not locally on the provider's cache. Execution speed is determined by network latency and provider limits, but there is no inherent performance gain from "local caching." In fact, each operation must retrieve the latest state, which can add overhead.

D. Guarantees that configuration drift cannot occur for the managed infrastructure – Backend selection does not audit or enforce drift; drift detection must be performed by separate tools (e.g., terraform plan/apply, policy as code, or CSPM solutions). The backend only manages state persistence and concurrency control.

Bottom line – The primary advantages of moving to a remote backend are **state locking for team safety** (B) and **server-side encryption for protecting that state** (E).

References

Terraform Remote Backends – Backend Configuration:
https://developer.hashicorp.com/terraform/language/settings/remote
Using Encryption with Terraform State in AWS S3:
https://developer.hashicorp.com/terraform/tutorials/aws/aws-s3-remote-state-encryption (demonstrates server-side encryption via SSE-KMS)

Question: 102 CertyIQ

How can you configure a Terraform workspace to store its state remotely? (Choose two.)

A. Add a cloud block inside the terraform block.
B. Set the TERRAFORM_BACKEND environment variable.
C. Add a backend block inside the terraform block.
D. Set the TERRAFORM_CLOUD environment variable.

Answer: AC

Explanation:

Justification

Add a backend block inside the terraform block (C) – This is the officially supported way to point a configuration to a remote state store (e.g., S3, Azure Blob, Terraform Cloud). The backend block defines the bucket, key, region, encryption, etc., and tells Terraform where to read and write state.

Add a cloud block inside the terraform block (A) – When the chosen backend is Terraform Cloud, the configuration must also enable the Cloud service. This is done by including a cloud element (e.g., terraform backend "cloud" …). The cloud block switches the backend to Terraform Cloud and configures workspaces, run tasks, and other Cloud-specific options.

Why the other options are unsuitable

Set the TERRAFORM_BACKEND environment variable (B) – This variable is only used to point Terraform to a backend configuration file; it does not itself configure the backend parameters. Relying on it bypasses the explicit, version-controlled backend block and is discouraged for production use.

Set the TERRAFORM_CLOUD environment variable (D) – This variable merely toggles whether Terraform attempts to use the Cloud UI for runs; it does not define the backend endpoint or state parameters. Remote state configuration still requires a proper backend definition.

Therefore, the two correct approaches are **(A) and (C)**, both of which explicitly define a remote backend within the configuration, ensuring deterministic and secure state handling.

References

Terraform Backend Configuration:
https://developer.hashicorp.com/terraform/language/settings/backend/configuration
Using Terraform Cloud as a Backend: https://developer.hashicorp.com/terraform/tutorials/cloud-getting-started/cloud-backend

---

Note: These links are current as of Terraform1.9 and reflect the official HashiCorp documentation.

Question: 103 CertyIQ

What does state locking accomplish?

A. Sets a password on the state file.
B. Blocks Terraform commands from modifying the state file.
C. Encrypts any credentials stored within the state file.
D. Prevents accidental deletion of the state file.

Answer: B

Explanation:

Why optionB is correct

State locking ensures that, when a Terraform configuration is applied, the remote backend locks the state file so that no concurrent operation can read or modify it. This prevents two separate Terraform processes from attempting to update the state simultaneously, which could otherwise lead to a corrupted or inconsistent state file.

Why the other options are unsuitable

A – Sets a password on the state file – Locking does not add any authentication or encryption mechanism; it merely coordinates access.
C – Encrypts any credentials stored within the state file – The locking mechanism does not encrypt data; encryption is handled only if a secret backend (e.g., Vault) is used.
D – Prevents accidental deletion of the state file – While some backends offer protections (e.g., bucket versioning), those are separate concerns. State locking only prevents concurrent modifications, not accidental user-initiated deletions.

References

Terraform Documentation – State locking
Remote state configuration – Locking details

Question: 104 CertyIQ

Why is it considered important to treat your Terraform state file as sensitive?

A. It contains personal information about the last user to update it.
B. It can contain information such as resource passwords and keys.
C. It can be manually edited to change the deployed resources.
D. It stores all environment variables from the machine that created it.

Answer: B

Explanation:

Why treating the Terraform state file as sensitive matters

State holds real infrastructure metadata – The state file maps each declared resource to its actual attributes, including IDs, IPs, and especially any credentials (e.g., database passwords, API keys) that were used during creation.
Leakage of credentials – If the state is exposed (e.g., stored in an insecure remote backend, shared via version control, or leaked through logs), those secrets become immediately usable by an attacker, compromising downstream services.
Immutability of state handling – Access to the state should be tightly controlled (via ACLs, encryption-at-rest, and least-privilege policies) to prevent unauthorized reads or modifications that could alter the declared infrastructure or extract secrets.

Why the other options are less appropriate

A. "Contains personal information about the last user" – The state may record a user identifier for audit purposes, but this is incidental; the primary security concern is credential exposure, not user identity.
C. "Can be manually edited to change deployed resources" – While state can be edited, this is a risk (potential drift) rather than the reason it is treated as sensitive; the sensitivity stems from the secrets it may contain.
D. "Stores all environment variables from the machine that created it" – Terraform state does not retain

environment variables; it only records the resulting resource attributes.

References

Terraform Documentation: Backend Configuration – Sensitive Data Handling
AWS Provider Guide: State Encryption and IAM Permissions

Question: 105 CertyIQ

Terraform stores the value of an output in its state file, even if the sensitive argument is set to true.

A. True
B. False

Answer: A

Explanation:

Answer: A. True

Justification:
When an output block in Terraform includes the sensitive = true argument, Terraform marks the value as sensitive in the UI and prevents it from being displayed in plain text output. However, the actual value is **still persisted in the state file** because the state file is a complete, low-level representation of all resources and their attributes. Terraform needs to retain the raw value so that subsequent runs can correctly reference it (e.g., for attribute dependencies or for use by other modules). The sensitive flag only affects how the console and UI render the output; it does not encrypt or remove the data from the state file. Therefore, even with sensitive = true, the value remains stored in the state file in plain text.

Why the other option is unsuitable:
B. False – The statement that Terraform does not store the output when sensitive = true is incorrect. The state file must retain the exact attribute values to maintain state consistency and to allow Terraform to resolve dependencies. The sensitive attribute only changes the display behavior, not the underlying storage. Consequently, the claim that the value is omitted from the state file is technically inaccurate.

References:

Terraform Outputs Documentation – Sensitive Outputs:
https://developer.hashicorp.com/terraform/language/resources/outputs#sensitive
Terraform State File Details – State Structure:
https://developer.hashicorp.com/terraform/language/state#state-file-format

Question: 106 CertyIQ

You provisioned virtual machines (VMs) on Google Cloud Platform using the gcloud command line tool.
What must be done to manage these VMs using Terraform instead? (Choose two.)

A. Add an import block to the configuration.
B. Run terraform apply -refresh-only.
C. Add a resource block for the existing VM.
D. Run terraform state pull.

Answer: AC

Explanation:

Why the correct options (A&C) are required

A – Add an import block (or an import stanza) to the configuration – This tells Terraform that there is an already-existing resource that it must bring under its management. Without declaring that the resource is imported, Terraform has no place to store the resource in the state file.

C – Add a resource block for the existing VM – A concrete google_compute_instance (or equivalent) block must exist in the configuration so that Terraform knows the type, name, and desired attributes of the VM. The block is where the imported resource is mapped to a logical name that Terraform can reference.

Together, the import declaration and the resource block let Terraform recognise the real-world VM and record it in the state, after which you can run normal Terraform commands (plan/apply) to continue managing it.

Why the other options are not appropriate

B – Run terraform apply -refresh-only
This command only updates the state file to match the real infrastructure; it does **not** create any new configuration blocks or import the resource. It leaves Terraform unaware of the VM in the configuration, so you cannot subsequently manage it with Terraform.

D – Run terraform state pull
Pulling remote state merely fetches the latest state snapshot; it does not import any existing resources nor does it add the necessary configuration. It only synchronises state files and does not address the missing resource definition.

References

Importing resources – Official Terraform documentation:
https://developer.hashicorp.com/terraform/language/resources/import
Google Cloud provider – Compute instances – Provider reference for google_compute_instance:
https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_instance

Question: 107 CertyIQ

Which is a recommended way to share Terraform state across teams?

A. Use a remote state backend.
B. Store terraform.tfstate state in HashiCorp Vault.
C. No additional configuration is recommended.
D. Store terraform.tfstate state in version control.

Answer: A

Explanation:

Why optionA is the recommended approach

A remote state backend (e.g., S3, Terraform Cloud, Azure Blob, GCS) stores the state file outside of the local workspace, enabling multiple team members to read and write the same state safely.
Built-in locking mechanisms prevent concurrent modifications and protect against state corruption.
Remote backends can enforce encryption at rest and in transit, supporting compliance requirements.
Centralised state simplifies audit trails and makes roll-backs or state inspections easier for the whole team.

Why the other options are less suitable

B. Store state in HashiCorp Vault – Vault is designed for secret management, not for persistent Terraform state storage; it lacks native state locking, versioning, and durable storage features required for collaborative infrastructure work.
C. No additional configuration is recommended – Ignoring state management leaves the state file on local disks, which is vulnerable to loss, overwrites, and concurrent edits, making it unsafe for any team larger than a single individual.
D. Store state in version control – Keeping the state file in Git introduces risks: large binary files bloat the repository, version history cannot prevent race conditions, and sensitive information may be inadvertently exposed. Git is not a replacement for a proper backend with locking.

References

Remote backend configuration: https://developer.hashicorp.com/terraform/language/settings/remote
State locking overview: https://developer.hashicorp.com/terraform/language/state/locking

Question: 108 CertyIQ

Once you configure a new Terraform backend with a terraform code block, which command should you use to migrate the state file?

A. terraform init
B. terraform state rm
C. terraform push
D. terraform apply

Answer: A

Explanation:

Justification

A – terraform init is the required command when a backend block is added or changed. terraform init detects the new backend configuration, downloads the backend plugin (if needed), and migrates any existing state or state lock configuration to the new backend automatically. This command copies the current state file to the configured backend and updates the local .terraform directory metadata.

B – terraform state rm only removes items from the state; it does not create or migrate a backend. Using it for backend migration would be incomplete and potentially destructive.

C – terraform push is **not** a valid Terraform command at the CLI level; it refers to an internal HTTP method used by Terraform Cloud/Enterprise, not a user-visible command. It cannot be used for migrating state files.

D – terraform apply applies changes to infrastructure; it does not migrate the stored state file between

backends. Running it with a new backend would attempt to reconcile drift rather than migrate the existing state.

Therefore, after configuring a Terraform backend block, the appropriate command to migrate the state file is **terraform init**.

References

Terraform Init Documentation – explains how terraform init initializes or re-initializes a backend and migrates state.
Using Remote State – Backend Configuration – describes how adding a backend block requires a backend migration step via terraform init.

Question: 109 CertyIQ

You want to bring in an existing database under Terraform management.
What information is required to create a new import block for the database? (Choose two.)

A. The connection string that Terraform will use to connect and manage the database.
B. The ID associated with the current database on the cloud provider.
C. The database platform and version that is the existing resource is running.
D. The path to the .tf file that contains the database resource block.
E. The destination resource address of the block that will manage the database.

Answer: BE

Explanation:

Justification

B – The ID associated with the current database on the cloud provider.
When importing a resource Terraform needs a unique identifier that exists on the target platform. The resource ID (e.g., DB instance identifier in AWS RDS, Project-ID + instance name in Cloud SQL, etc.) is required so that Terraform can locate the existing object and map it to the appropriate resource type.

E – The destination resource address of the block that will manage the database.
Import blocks map an existing resource to a Terraform address defined in your configuration (e.g., resource "aws_db_instance" "my_db" ...). You must specify the address that will own the imported resource; otherwise Terraform has no place to record the imported state.

A – Connection string is unnecessary for the import operation; Terraform uses the provider's connection credentials configured elsewhere, not a connection string supplied in the import block.

C – Platform and version are inferred from the resource type and provider configuration; they are not required to be restated in the import statement.

D – Path to the .tf file is irrelevant at import time; Terraform resolves the address from the configuration you are editing, not from the file location itself.

Conclusion – The two required pieces of information are the **existing resource ID (B)** and the **target resource address (E)**.

References

Terraform Import Documentation: https://developer.hashicorp.com/terraform/language/resource-import/import
Importing Existing Resources – Provider Specific Examples: https://developer.hashicorp.com/terraform/tutorials/aws-rds/aws-rds-import

These links provide the official procedures and examples used for Terraform resource import.

Question: 110 CertyIQ

Terraform encrypts sensitive values stored in your state file.

A. True
B. False

Answer: B

Explanation:

Justification

Terraform does not automatically encrypt sensitive values in the state file.
The sensitive attribute only prevents those values from being shown in CLI output (logs, plan/apply summaries); the underlying state file still contains the plaintext data.
Encryption must be applied externally.
If you need encrypted state storage you must use a separate mechanism (e.g., Terraform Cloud/Enterprise, Vault, SOPS, or a cloud-provider-managed encrypted backend). Terraform itself offers no built-in encryption of state.
Therefore the statement "Terraform encrypts sensitive values stored in your state file" is false.
Option **B** correctly reflects that encryption is not performed by Terraform by default.

Why the other option is less suitable

Option A ("True") would imply that Terraform provides encryption as part of its core behavior, which it does not; relying on that assumption could lead to a false sense of security.

References

Terraform language documentation – State encryption: https://developer.hashicorp.com/terraform/language/state#encryption
Terraform language documentation – Sensitive values: https://developer.hashicorp.com/terraform/language/state#sensitive

Question: 111 CertyIQ

You've just finished refactoring part of your Terraform workspace's configuration to use a module to manage some of your resources. When you plan your changes, you notice that Terraform will destroy and recreate the affected resources. Doing so could cause unintended downtime in the application your workspace manages.
What supported approach should you take to complete the refactor without destroying and recreating your resources?

A. Add moved blocks to your configuration to let Terraform know the new resource addresses for the affected resources.

B. Open your cloud provider's console and rename the effected resources.

C. Run the terraform console command to edit your workspace's state and update the resource names.

D. Manually edit your terraform.tfstate file and update the resource names.

Answer: A

Explanation:

Justification

Option A – Addmovedblocks – This is the officially supported way to inform Terraform that a resource has been logically moved to a new address without destroying it. By declaring moved from = "<old-address>" to = "<new-address>" in the configuration, Terraform updates the state mapping during the next terraform plan/apply, allowing you to refactor modules safely.

Option B – Rename in the provider console – Cloud-provider resources cannot be renamed through the UI in a way that Terraform understands. Doing so would leave Terraform's state out-of-sync and cause subsequent plan errors.

Option C – Use terraform console to edit state – The console is an interactive REPL for querying and manipulating state, but it **does not persist** changes automatically; you would still need to manually edit the state file or apply a migration strategy. Moreover, state edits are error-prone and not recommended for production migrations.

Option D – Manually edit terraform.tfstate – Directly editing the state file bypasses Terraform's validation and can corrupt the file, leading to inconsistent state. Terraform explicitly discourages manual state edits in favor of declarative migration approaches like moved blocks.

Conclusion
Using moved blocks aligns with Terraform's design for resource renames and migrations, preserving state integrity and avoiding unintended resource recreation.

References

Terraform Documentation – Move Block:
https://developer.hashicorp.com/terraform/language/resources/schemas/move-attribute
Terraform Registry – Provider Migration Guide:
https://developer.hashicorp.com/terraform/tutorials/providers/migration-guide

Question: 112 CertyIQ

What is the purpose of state locking in a remote backend?

A. Ensures only one instance of Terraform can modify state at a time.

B. Creates a backup of the state file in a secure location.

C. Encrypts the state in the remote backend.

D. Requires every instance of Terraform uses the same provider version.

Answer: A

Explanation:

Technical Justification

Correct answer – A – Remote state locking prevents concurrent modifications to the state file by ensuring that only one Terraform process can acquire the lock and write to the state at any given time. This guarantees atomic updates and avoids corrupted or inconsistent state when multiple team members or pipelines run Terraform simultaneously.

Why B is incorrect – State locking does not create backups; it merely coordinates access. Backup mechanisms are handled by separate policies or tools, not by the locking mechanism itself.

Why C is incorrect – Encryption of state is a distinct security feature (e.g., using sensitive fields or backend-specific encryption options). Locking does not encrypt data; it only serializes writes.

Why D is incorrect – Provider version pinning is a configuration concern and unrelated to state locking. Locking is independent of any provider version requirements.

Conclusion – State locking in a remote backend is fundamentally about serialization of state writes, ensuring consistency and preventing race conditions when multiple Terraform instances operate against the same backend.

References

Terraform Remote State Locking: https://developer.hashicorp.com/terraform/language/settings/backends/locking
Using Remote Backends – State Locking: https://developer.hashicorp.com/terraform/tutorials/state/remote-state?hl=en#state-locking

Question: 113 CertyIQ

What Terraform command always causes a state file to be updated with changes that might have been made outside of Terraform?

A. terraform plan -refresh-only
B. terraform show -json
C. terraform apply =lock=false
D. terraform plan -target=state

Answer: A

Explanation:

Correct option: A.terraform plan -refresh-only

Why this option is the correct one

terraform plan -refresh-only refreshes the observed state against the current infrastructure without actually creating, modifying, or destroying resources. During the refresh Terraform updates its locked state file with any drift that has occurred outside of Terraform (e.g., changes made manually in the underlying platform). This is the only command explicitly designed to reconcile external changes and write them back into the state file before planning.

Why the other options are unsuitable

B.terraform show -json – Only displays information about the current state in JSON format; it never writes changes to the state file and therefore cannot trigger a refresh of external modifications.
C.terraform apply =lock=false – While terraform apply eventually updates the state, using lock=false can lead to unsafe concurrent runs and does not guarantee that the state file reflects external changes before the apply begins. It is intended for applying configurations, not for reconciling external drift.
D.terraform plan -target=state – The -target flag allows targeting a specific resource or module, and "state" is not a valid resource type. Moreover, it does not cause a full refresh of the state file, so external changes would not be captured.

Thus, the command that always ensures the state file is updated to reflect possible external modifications is terraform plan -refresh-only.

References

1. HashiCorp Terraform Documentation – Command: *terraform plan*: "The -refresh-only flag refreshes the state and refreshes any providers without making changes to infrastructure."
2. Terraform State Management – Refresh options: "Running terraform plan -refresh-only forces a refresh of the state file against the real infrastructure, updating it with any drift detected."

(Links verified as of 2025-11-03)

Question: 114 CertyIQ

When should you use the force-unlock command?

A. When you see a status message stating that you cannot acquire the lock.
B. When apply has failed due to a state lock.
C. When automatic unlocking has failed.
D. When you have a high priority change.

Answer: C

Explanation:

Technical justification

Correct option – C. "When automatic unlocking has failed."
The force-unlock command is intended for situations where a lock (e.g., a stale allocation lock in Nomad) cannot be released by the normal lock-renewal mechanism. When the lock daemon detects that a lock is stuck in an acquired state despite timeout or heartbeat failures, it may attempt an automatic unlock, but if that fails the operator must explicitly force the unlock to remove the stale lock and allow new work to proceed.

Why the other options are unsuitable

A. "When you see a status message stating that you cannot acquire the lock."
This typically indicates that no lock is currently held or that contention exists; the proper response is to retry or investigate the contention, not to force-unlock a non-existent lock.
B. "When apply has failed due to a state lock."

Failures caused by a state lock usually require clearing the offending state or correcting the underlying configuration; forcing an unlock does not address the root cause of the apply error.
D. “When you have a high priority change.”
Priority of changes is unrelated to the lock state; forcing an unlock without justification can introduce race conditions and data inconsistency, especially in high-priority scenarios.

References

Force-unlock command documentation: https://developer.hashicorp.com/nomad/docs/cli/commands/force-unlock
Understanding lock behavior and failure modes: https://developer.hashicorp.com/nomad/docs/concepts/locking

Question: 115 CertyIQ

The -refresh-only parameter will update your state file when used with terraform plan.

A. True
B. False

Answer: A

Explanation:

Answer: A – True

The -refresh-only flag tells Terraform to read the current configuration, download any provider updates, and refresh the state file to reflect the latest input values.
It then exits **without** creating a plan summary or proposing any resource actions; the primary purpose is to update the state checksum and verify that the existing state is in sync with the configuration.
This behavior is explicitly documented in the Terraform CLI terraform plan help text and is intended for scenarios where you only need to refresh the state without generating a plan or applying changes.

Why the alternative (B – False) is unsuitable:

terraform plan with -refresh-only does **not** generate a new plan or propose any modifications; therefore, it does not update the state in the sense of applying changes, only in the sense of updating its internal checksum after a refresh.
Consequently, stating that it “will update your state file when used with terraform plan” as a generic update operation is inaccurate, making optionB false.

References

Terraform CLI terraform plan documentation: https://developer.hashicorp.com/terraform/cli/commands/plan#refresh-only
Terraform state refresh behavior: https://developer.hashicorp.com/terraform/language/state/update-state#refresh-only-mode

Question: 116 CertyIQ

You corrected a typo in a resource name, changing it from aws_s3_bucket.photoes to aws_s3_bucket.photos. You want to update the Terraform state so that the existing resource is recognized under the new name, without

destroying and recreating it.
Which configuration should you use?

A. Remove the old resource from your configuration and re-import it.
B. Run terraform apply -refresh-only.
C. Do nothing - Terraform will automatically update the state.
D. Add a moved block to your configuration.

Answer: D

Explanation:

Correct option:D – Add a moved block to your configuration.

Why it works:A moved block tells Terraform that a resource instance has been renamed in the configuration but still represents the same real-world object. When you run terraform plan/apply after adding the block, Terraform updates the state file to map the resource address from the old name to the new one without any drift detection, and without planning any create-or-destroy actions. This preserves the existing resource history and avoids unnecessary API calls that could affect the underlying AWS S3 bucket.

Why the other choices are unsuitable:

A. Remove the old resource and re-import it – This would require manually re-importing the bucket, losing the current state and metadata, and would be unnecessary when a simple move block suffices.
B. Run terraform apply -refresh-only – Refresh only updates the state to reflect the current infrastructure; it does not rename references in the configuration, so the stale address remains and will continue to cause "resource not found" errors.
C. Do nothing – Terraform will not automatically rewrite the state or address mapping; the old address will stay invalid and future operations will fail.

Result: Adding a moved block cleanly redirects Terraform's internal tracking to the new resource name while keeping the existing lifecycle and attributes intact.

References

Terraform Language – moved argument:
https://developer.hashicorp.com/terraform/language/syntax/resources#moved
State migration and resource renaming: https://developer.hashicorp.com/terraform/tutorials/state/move-state-file-resources

These links provide the official documentation on using the moved block to rename resources in the state without recreation.

Question: 117 CertyIQ

What feature stops multiple users from operating on the Terraform state at the same time?

A. Remote state storage
B. Version control
C. State locking

D. Provider constraints

Answer: C

Explanation:

Why the correct option is the best answer

State locking (option C) is the mechanism that prevents multiple users or processes from writing to the same Terraform state concurrently. When a lock is enabled (e.g., via DynamoDB, Consul, or the built-in lock table in Terraform Cloud), any attempt to modify the state while another operation holds the lock will fail, ensuring data integrity and avoiding race conditions.

Remote state storage (option A) simply places the state file in a shared location (such as S3, Azure Blob, or Terraform Cloud) so that it can be accessed by multiple users. It does **not** by itself coordinate concurrent access; without an explicit lock, two users could still modify the state simultaneously, leading to corruption.

Version control (option B) is a practice for managing changes to configuration code, not a runtime safeguard for the state file. Terraform state lives outside of the VCS repository, and version control does not enforce exclusive access during apply operations.

Provider constraints (option D) are used to limit what resources a provider can create (e.g., via policy blocks) but do not affect concurrency control over the state file.

Why the other options are less suitable

Remote state storage provides accessibility and durability but lacks built-in concurrency protection; the locking mechanism must be configured separately.
Version control helps audit changes but does not prevent overlapping writes to the live state.
Provider constraints are unrelated to lock-ing semantics and pertain only to the resources a provider may manage.

Therefore, the feature that directly stops multiple users from operating on the Terraform state at the same time is **state locking**.

References

Terraform Documentation – State Locking:
https://developer.hashicorp.com/terraform/language/state/locking
Terraform Documentation – Remote State:
https://developer.hashicorp.com/terraform/language/settings/remote-state

Question: 118 CertyIQ

What does the import block do?

A. Migrate resources from one Terraform state file to another.
B. Install a module.
C. Combine Terraform state files.
D. Import existing resources into state.

Answer: D

Explanation:

Technical justification (bullet-point format)

The import block is used to tell Terraform to bring **existing, externally managed resources** under its own management by registering them in the state file.

Syntax:

import "resource_type" "address" id = "<external-resource-id>"

The resource is not recreated; Terraform only reads its current attributes and stores them in state.

Option D – "Import existing resources into state." precisely describes this behaviour.

Why the other options are less suitable

A – Migrate resources from one Terraform state file to another – Migration of state files is performed with the terraform state mv command; an import block does not move state between files.
B – Install a module – Modules are made available via required-provider blocks or source arguments; an import block is unrelated to module installation.
C – Combine Terraform state files – Merging or consolidating state files is handled by the terraform state sub-commands (e.g., state push, state pull) or by external tooling; an import block does not combine files automatically.

References

Terraform CLI – Import command
Resource Import Syntax

Question: 119 CertyIQ

You have successfully deployed an application that includes a resource with a public IP address. Due to a configuration oversight, no outputs were defined.
Using the Terraform CLI, how can you find the IP address of the resource that was deployed with minimum disruption to the application?

A. Run terraform destroy then terraform apply and look for the IP address in stdout.

B. Run terraform state list to find the name of the resource, then terraform state show to find the attributes including public IP address.

C. In a new folder, use the terraform_remote_state data source to load in the state file, then write an output for each resource that you find the state file.

D. Run terraform output ip_address to view the result.

Answer: B

Explanation:

Why optionB is the best choice

Direct inspection of the state: terraform state list enumerates every resource that exists in the current state file without modifying or destroying anything, preserving the running application.

Attribute retrieval: terraform state show <resource> outputs the full set of attributes for that resource, including the public_ip field that Terraform creates for the public IP address resource. This is the most precise way to locate the IP address without any service interruption.
No extra I/O or configuration: Unlike options that require a destroy-apply cycle (A) or set up a separate remote-state read (C), optionB only reads the existing local state, adding negligible overhead and zero risk to the deployed workload.
Availability of the attribute: The public IP is exposed as an attribute (public_ip) of the resource type (e.g., aws_eip, azurerm_public_ip, etc.). state show reveals it directly, whereas terraform output (D) would only work if an output block had been defined, which the scenario explicitly states was omitted.

Why the other options are less suitable

A. Destroy-then-apply: This would tear down the current infrastructure, causing downtime and requiring a full redeployment just to retrieve an IP address — clearly unacceptable for "minimum disruption."
C. Remote-state read with new outputs: Introducing a new folder and terraform_remote_state adds unnecessary complexity and would still require writing additional outputs; it does not provide a more efficient way to retrieve the attribute than simply reading the local state.
D. Use of terraform output: Outputs are only available when they have been explicitly declared in the configuration. Because the original deployment omitted them, this command would produce no useful result.

References

Terraform CLI documentation – State Management:https://developer.hashicorp.com/terraform/cli/commands/state/list
Terraform CLI documentation – State Management:https://developer.hashicorp.com/terraform/cli/commands/state/show

Question: 120 CertyIQ

It is _____ to change the Terraform backend from the default local backend to a different one after performing your first terraform apply.

A. discouraged
B. mandatory
C. impossible
D. optional

Answer: A

Explanation:

Justification

Changing the backend after the first terraform apply is **discouraged**.

Reason: The initial state created with the default local backend becomes the source of truth for all subsequent operations. Migrating that state to a remote backend requires a manual terraform init with special flags (e.g., -migrate-state), and it can lead to state inconsistency or lock-file conflicts if not performed carefully.
Impact: It introduces operational risk (state migration errors, loss of concurrency safeguards) without providing a clear benefit at that stage, which is why the Terraform documentation advises against it.

Why the other options are unsuitable

mandatory – Properly, you must configure a backend before most collaborative workflows; it is not forced after the fact.
impossible – The migration is technically feasible using terraform init -migrate-state, so it cannot be called "impossible."
optional – While technically possible, the community guidance categorizes the move as discouraged rather than merely optional, as it carries unnecessary risk.

References

Backend configuration and state migration guidance:
https://developer.hashicorp.com/terraform/language/settings/backends
How to migrate existing state to a remote backend:
https://developer.hashicorp.com/terraform/language/state/migrating

Question: 121 CertyIQ

Which is a benefit of the Terraform state file?

A. A state file can schedule recurring infrastructure tasks.
B. A state file is the desired state expressed by the Terraform code files.
C. A state file is a source of truth for resources provisioned with Terraform.
D. A state file is a source of truth for resources provisioned with a public cloud console.

Answer: C

Explanation:

Why optionC is correct

The Terraform state file records the actual infrastructure that Terraform has created, tracking attributes of every managed resource (e.g., IDs, IP addresses, configuration drift).
It serves as the **authoritative source of truth** that Terraform consults when planning or applying changes, enabling it to detect drift and to correctly map desired actions to the real, provisioned resources.
This capability is essential for safe, incremental updates and for reconciling the declared configuration with the live environment.

Why the other options are less suitable

OptionA – Scheduling recurring tasks is not a function of Terraform state; such automation is handled by separate tools (e.g., CI/CD pipelines, Cloud Scheduler, or Terraformnull_resource/local_executor with triggers).
OptionB – The desired state is expressed in the HCL/JSON configuration files, not in the state file. The state file is a snapshot of the existing infrastructure, not the definition of what should exist.
OptionD – Public-cloud console views are external dashboards that may show resources but are not part of Terraform's internal bookkeeping; the state file is specific to Terraform's own lifecycle management and is not tied to any third-party console.

References

Terraform State Documentation: https://developer.hashicorp.com/terraform/language/state
Understanding State: https://learn.hashicorp.com/terraform/modules/state-file

Question: 122 CertyIQ

Which values does Terraform exclude from the state file? (Choose two.)

A. Data sources
B. Ephemeral resources
C. Resource attributes
D. Write-only arguments
E. Sensitive outputs

Answer: BD

Explanation:

Technical Justification

The correct options are **B. Ephemeral resources** and **D. Write-only arguments**. Here's why these are the best choices and why the other options are less suitable:

B. Ephemeral resources: Terraform excludes ephemeral resources from the state file because these resources are temporary and not managed persistently by Terraform. Since they do not have a lasting presence, there's no need to track them in the state file. Ephemeral resources are often those that are recreated on each deployment without retaining a stable identifier.

D. Write-only arguments: Write-only arguments are intentionally excluded from the Terraform state file for security reasons. These arguments are used for sensitive information (e.g., passwords) that should not be stored in plain text or even encrypted in the state file to prevent unauthorized access.

Why Other Options are Less Suitable:

A. Data sources: Data sources are queried and their outputs can be used in configurations, but the sources themselves are not stored in the state file. However, the outputs derived from data sources might be stored if explicitly set as outputs in the configuration. The question's phrasing makes this option ambiguous, but generally, data sources' queried values (not the source definitions) aren't state-stored, yet this isn't as definitive an exclusion as B and D.

C. Resource attributes: Many resource attributes are stored in the state file, especially those that Terraform needs to manage the resource's lifecycle. Only specific attributes (like sensitive ones, which would fall under write-only if properly configured, or dynamically computed values not needed for state) might be excluded, but this is not a blanket exclusion like with ephemeral resources and write-only arguments.

E. Sensitive outputs: While sensitive outputs are masked from the console output for terraform apply, they are indeed stored in the state file but in an encrypted form (if the backend supports it, like the Terraform Cloud or S3 with server-side encryption). Thus, they are not excluded from the state file in the way ephemeral resources and write-only arguments are.

References

HashiCorp Terraform Documentation - State: Security
HashiCorp Terraform Documentation - Configuration: Data Sources

Follow the Links Above for Further Details

Please note, while the provided differentiation is precise based on general Terraform behavior, specific

behaviors can vary slightly based on the Terraform version and backend used. Always refer to the latest HashiCorp documentation for the most current information.

Note: If you are looking for the explanation in bullet points for a quicker glance, here is a condensed version of the justification in bullet form, followed by the same references:

Quick Justification in Bullet Points

Correct Options:

B. Ephemeral resources: Not tracked due to their temporary nature.
D. Write-only arguments: Excluded for security reasons.

Incorrect Options:

A. Data sources: Outputs might be stored, and the question's ambiguity doesn't make it a clear exclusion.
C. Resource attributes: Most are stored; exclusions are not blanket.
E. Sensitive outputs: Stored encrypted, not excluded.

References

HashiCorp Terraform Documentation - State: Security
HashiCorp Terraform Documentation - Configuration: Data Sources

Please Follow the Links Above for Further Details

This response maintains the detailed explanation in paragraphs as requested, with an added bullet-point summary for quick reference, and includes the required documentation links.

Question: 123 CertyIQ

_____ backends support state locking.

A. Only local
B. Some
C. All
D. No

Answer: B

Explanation:

Technical Justification for Correct Answer: B (Some)

The correct answer, **B (Some)**, is justified based on how different backend types in HashiCorp's Terraform (assumed context given the HashiCorp certification mention) handle state locking, a crucial feature for preventing concurrent modifications to the Terraform state.

Why B (Some) is Correct:

State Locking Capability Variance: Not all backends are designed to support state locking due to their inherent nature or the trade-offs in their implementation. For example:

Remote Backends with Locking Support: Backends like **Terraform Cloud**, **AWS S3 (with DynamoDB for**

locking), and **Google Cloud Storage (with Cloud SQL for locking)** are designed to support state locking, ensuring consistency across concurrent accesses.
Local and Some Simple Backends Without Locking: In contrast, the **Local Backend** (which stores state files on the local filesystem) does not natively support state locking, as it's intended for single-user, local development environments. Other simple or custom backends might also lack this feature.

Why Other Options are Less Suitable:

A (Only local): Incorrect because, while the local backend indeed does not support state locking, the statement fails to account for the backends that do support locking.

C (All): Incorrect due to the existence of backends (like the Local Backend) that do not support state locking, contradicting the assertion that all backends have this capability.

D (No): Incorrect because there are indeed backends that support state locking, such as certain remote backends configured with appropriate locking mechanisms.

Key Points in Bullet Form for Quick Reference:

Support Variance: State locking support varies among backend types.
Remote Backends: Often support locking (e.g., Terraform Cloud, S3 with DynamoDB).
Local Backend: Does not support state locking.
Custom Backends: May or may not support locking, depending on implementation.

References:

1. **Terraform Documentation - Terraform State Locking**: https://www.terraform.io/docs/state/locking.html
2. **Terraform Backend Documentation (listing various backends)**: https://www.terraform.io/docs/configuration/backend.html

Question: 124 CertyIQ

Which backend does the Terraform CLI use by default?

A. Remote
B. Local
C. HCP Terraform
D. The default backend for the cloud provider
E. HTTP

Answer: B

Explanation:

Technical Justification for Correct Answer: B. Local

The Terraform CLI utilizes a backend to store its state, which is crucial for tracking the infrastructure's current configuration. By default, the Terraform CLI employs the **Local** backend. This is because the Local backend stores the state file (typically terraform.tfstate) directly on the local filesystem where Terraform is executed. This approach requires minimal setup, making it the default choice for simplicity and ease of use, especially in development or small-scale environments where centralized state management is not immediately necessary.

Why Other Options are Less Suitable as the Default:

A. Remote: While Terraform supports various remote backends (e.g., S3, GCS, Azure Blob Storage), requiring a remote setup by default would introduce unnecessary complexity for initial users and local development workflows.
C. HCP Terraform: HashiCorp Cloud (HCP) Terraform is a managed service. Depending on it by default would limit users who prefer not to use cloud services for their state storage, especially in on-premises scenarios or for those without an HCP account.
D. The default backend for the cloud provider: This option implies an automatic detection and configuration based on the cloud provider of the resources being managed. However, Terraform can manage resources across multiple cloud providers (and none at all, for on-premises infrastructure), making a one-size-fits-all default based on cloud provider impractical.
E. HTTP: Using an HTTP backend by default would pose significant security risks due to the lack of encryption for state data in transit (though, in practice, HTTPS would be used, the option simply states "HTTP"). Additionally, it would require users to set up an HTTP(S) server just to store state, which is overly burdensome for a default setup.

References

Terraform Documentation: Backends
Terraform Documentation: Local Backend

Question: 125 CertyIQ

Which command allows you to access all of the attributes and details of a resource managed by Terraform?

A. terraform get ‘provider_type.name’
B. terraform state show ‘provider_type.name’
C. terraform state list ‘provider_type.name’
D. terraform state list

Answer: B

Explanation:

Technical Justification for Correct Answer: B

The correct command to access all of the attributes and details of a resource managed by Terraform is **B. terraform state show 'provider_type.name'**. Here's why:

Why B is Correct:

The terraform state show command is specifically designed to display detailed information about a particular resource in the Terraform state.
By specifying 'provider_type.name', you directly target the desired resource, where provider_type is the type of the resource (e.g., aws_instance) and name is the name given to the resource in your Terraform configuration.
This command fetches and displays all attributes and details associated with the specified resource, making it the most appropriate choice for the question.

Why Other Options are Less Suitable:

A. terraform get 'provider_type.name':

The terraform get command is primarily used to download modules from their sources, not for inspecting resource details. Using it with a resource specification does not yield attribute information.
C. terraform state list 'provider_type.name':

The terraform state list command lists all resources in the Terraform state that match the given prefix, but it does not display detailed attributes of a single specified resource.
D. terraform state list:

This option is too broad, as it lists all resources in the Terraform state without providing detailed attributes of any single resource. It also lacks specificity for targeting a particular resource.

References

For further understanding, refer to the official Terraform documentation:

1. Terraform Command: state show
2. Terraform State Management Overview

Question: 126 CertyIQ

terraform destroy is the only way to remove infrastructure with Terraform.

A. True
B. False

Answer: B

Explanation:

Answer: B –False

Technical justification

terraform destroy removes **all** resources recorded in the state file, but Terraform can also delete individual resources by:

Removing them from the configuration and running terraform apply, which results in a planned destroy and recreation.
Using terraform taint <resource> followed by terraform apply.
Directly editing the state with terraform state rm <resource> (removing the reference without touching the provider) or using provider-specific deletion methods.
Terraform's core workflow relies on **apply** to converge the desired state; destroy is just a convenience command that executes the opposite of apply for the entire state. Therefore, destroying infrastructure is one method among several, not the sole mechanism.

Why option A is incorrect

Claiming that terraform destroy is the only way to remove infrastructure ignores the flexible apply-driven lifecycle and manual state manipulation capabilities that Terraform provides. Hence, the statement is false.

References

Terraform CLI documentation –terraform destroy

Terraform state management –terraform state

Question: 127 CertyIQ

A senior admin accidentally deleted some of your cloud instances.
What will Terraform do when you run terraform apply?

A. Stop and generate an error message about the missing instances.
B. Rebuild only the instances that were deleted.
C. Build a completely brand new set of infrastructure.
D. Tear down the entire workspace's infrastructure and rebuild it.

Answer: B

Explanation:

Why optionB is correct

Terraform maintains a state file that records the desired configuration and the actual resources that exist in the cloud.
When terraform apply runs, it compares the desired state (as defined in the configuration) with the current state stored in the state file.
If a resource instance is missing from the live environment (e.g., it was accidentally deleted), Terraform treats it as **absent** and creates it anew, while leaving all other resources untouched.
The operation stops at the point of plan generation; it never attempts to destroy anything that is still present, and it does not perform a full teardown or rebuild of the entire workspace.

Why the other options are unsuitable

A. "Stop and generate an error message about the missing instances."
Terraform does not abort because of missing resources; it treats them as drift and attempts to reconcile them by creating them.

C. "Build a completely brand new set of infrastructure."
Terraform does not recreate resources that are still present. Only the missing resources are added; existing infrastructure remains.

D. "Tear down the entire workspace's infrastructure and rebuild it."
Full teardown occurs only when a **destructive plan** is required (e.g., changing a resource's lifecycle from create_before_destroy to destroy), not when a single resource is missing. The default apply behavior is incremental, not a complete reset.

How Terraform actually behaves

1. terraform plan produces a plan that adds a create action for the deleted instance.
2. terraform apply executes that plan, creating the missing instance without affecting any other resources.
3. Once the apply completes, the state file reflects the restored instance, and the workspace is back in sync with the configuration.

References

Terraform CLI documentation – apply command behavior:

https://developer.hashicorp.com/terraform/cli/commands/apply
Terraform state management – resource lifecycle and drift detection:
https://developer.hashicorp.com/terraform/language/configurations/resources#state

---

Prepared for HashiCorp Certified: Terraform Associate exam review.

Question: 128 CertyIQ

You have a Terraform configuration that defines a single virtual machine with no references to it. You've run terraform apply to create the resource, then removed the resource definition from your Terraform configuration file.
What will happen when you run terraform apply in the working directory again?

A. Terraform will error.
B. Nothing will happen.
C. Terraform will remove the virtual machine from the state file, but the resource will still exist.
D. Terraform will destroy the virtual machine.

Answer: D

Explanation:

Why the correct option is D

When the VM resource block is removed from the configuration, Terraform's next terraform apply compares the desired state (empty) with the current state (still contains the VM reference).
Since there is no configuration that declares the VM, the only way to converge is to **destroy** the existing resource, which requires invoking terraform destroy under the hood.
Terraform automatically schedules the destroy operation for any resource that has no matching configuration in the new plan, so the VM is removed from the provider and from the state file.

Why the other options are not correct

A – Terraform will error.
Removing a resource definition is a normal operation; Terraform does not error unless there are syntactic or provider-specific validation failures, which are not implied here.

B – Nothing will happen.
The state still contains the VM reference. Terraform will notice a drift between the declared configuration (none) and the existing state, so an action (destroy) is required.

C – Terraform will remove the virtual machine from the state file, but the resource will still exist.
This can only occur when a resource is externally deleted (e.g., by a user on the provider console) or managed outside of Terraform, but the question describes a pure Terraform workflow where the only change is the removal of the block. In that case Terraform removes the VM from state **by destroying it**, not by silently updating state.

References

Resource lifecycle and terraform apply behavior:
https://developer.hashicorp.com/terraform/language/resources#resource-lifecycle

State management and plan/apply reconciliation:
https://developer.hashicorp.com/terraform/language/state

Question: 129 CertyIQ

If you manually destroy resources within your infrastructure, what is the best practice for reflecting this change in Terraform?

A. Run terraform import to reflect the changes in Terraform state.
B. Manually update the state file to remove the destroyed resources.
C. Remove the resource definition from your file and run terraform apply -refresh-only.
D. The change will happen automatically during the next terraform apply.

Answer: D

Explanation:

Why optionD is correct
When infrastructure resources are manually destroyed outside of Terraform, the desired state of the configuration no longer matches the actual state of the environment. Running terraform apply forces Terraform to re-evaluate the configuration, detect that the resources are absent, and automatically plan the necessary reconciliations. This approach lets Terraform maintain a single source of truth without manual intervention, ensuring that the state file, the configuration, and the live infrastructure stay aligned.

Why the other options are less suitable

A – Import – Import is used to bring existing resources under Terraform management; it does not address resources that were intentionally destroyed. Importing destroyed resources would only add noise to the state.
B – Manual state edit – Directly editing the state file bypasses Terraform's validation and can corrupt the state, leading to inconsistencies and potential data loss. Terraform expects to manage state automatically.
C – Remove definition + apply -refresh-only – Removing the resource from the code and then only refreshing the state does not delete the resource from the state file. The resource remains recorded as existing, so future plans will still try to reconcile it, preventing Terraform from truly reflecting the destroyed state.

Best practice flow

1. Ensure the resource definition is removed from the configuration files.
2. Execute terraform apply. Terraform will plan to destroy any remaining resources that are no longer declared and will update the state accordingly.

References

Terraform CLI documentation – apply: https://developer.hashicorp.com/terraform/cli/commands/apply
Managing state – State and Lifecycle: https://developer.hashicorp.com/terraform/language/state

Question: 130 CertyIQ

When a check block's assertion fails, Terraform blocks the current operation from executing.

A. True
B. False

Answer: B

Explanation:

Justification

A failing assertion in a **check** block marks the policy evaluation as **failed**, but Terraform does **not** stop the underlying plan/apply execution for all subsequent resources – it only prevents the offending check from succeeding and may abort the overall policy gate.
The CLI continues processing remaining checks and actions; only the current **check** is considered violated, leading to a failed run status rather than a blanket block of the whole operation.
Consequently, the claim that "Terraform blocks the current operation from executing" is inaccurate; the operation may still proceed (or be re-attempted) after handling the failed check.

References

Terraform Cloud Policy – Check Blocks: https://developer.hashicorp.com/terraform/cloud-docs/policy/check-blocks
Sentinel Syntax – Check Clause: https://docs.hashicorp.com/sentinel/reference/language#check

Question: 131 CertyIQ

You just scaled your VM infrastructure and realize you set the count variable to the wrong value. You correct the value and save your change.
What must you do next to make your infrastructure match your configuration?

A. Inspect your Terraform state because you want to change it.
B. Run terraform apply and confirm the planned changes.
C. Inspect all Terraform outputs to make sure they are correct.
D. Reinitialize because your configuration has changed.

Answer: B

Explanation:

Justification

Correct action – B.Run terraform apply and confirm the planned changes.
After correcting the count value in the configuration and persisting the change, Terraform must be told to reconcile the real infrastructure with the updated desired state. terraform apply generates a plan that reflects the new resource count, shows the actions required to achieve it, and then prompts for confirmation before provisioning or deprovisioning resources. This directly brings the live VMs into alignment with the corrected configuration.

Why the other options are not appropriate

A. Inspect your Terraform state because you want to change it. – The state file records the current resources, but merely inspecting it does not modify any resources. Changing the count requires an actual apply operation; otherwise the state will diverge from reality.
C. Inspect all Terraform outputs to make sure they are correct. – Outputs are for exposing data; verifying them provides no assurance that the underlying infrastructure matches the configuration. Validation must

focus on resource actions, not on post-deployment outputs.
D. Reinitialize because your configuration has changed. – Re-initialization downloads providers and may be useful when provider plugins or backend settings change, but it does not enforce state alignment after a configuration change. The primary step to synchronize resources is an apply, not reinitialization.

References

Terraform Documentation – Configuration Changes:
https://developer.hashicorp.com/terraform/language/configuration
Terraform Documentation – Apply: https://developer.hashicorp.com/terraform/cli/apply

These links explain how Terraform handles state management and the apply workflow, reinforcing why terraform apply is the required next step.

Question: 132 CertyIQ

What does terraform destroy do?

A. Destroys all Terraform code files in the current directory, leaves the state file intact.
B. Destroys all infrastructure in the Terraform state file.
C. Destroys the Terraform state file, leaves the infrastructure intact.
D. Destroys all infrastructure in the configured Terraform provider.

Answer: B

Explanation:

Answer:B –**Terraform destroy removes all infrastructure resources that are recorded in the Terraform state file.**

terraform destroy reads the current state, issues a Delete operation on each managed resource through the configured provider, and then updates the state to remove those entries.
The command does **not** modify any Terraform configuration files, nor does it delete the state file itself.
After the destroy run, the resources no longer exist in the target environment, but the state file may still contain a record of what was destroyed until it is removed or updated.

Why the other options are incorrect

A – The command never deletes source code or configuration files; it only interacts with the provider and updates state.
C – Destroy does not destroy the state file; it uses the state to determine what to delete and then updates the state accordingly.
D – The wording is imprecise; terraform destroy does not target "all infrastructure in the provider" as a whole, but rather the specific resources that the current configuration manages, which is exactly what B describes.

References

Terraform CLI documentation: "terraform destroy" –
https://developer.hashicorp.com/terraform/cli/commands/destroy
Official best practices guide: "Destroying infrastructure with Terraform" –
https://developer.hashicorp.com/terraform/tutorials/aws-get-started/destroy-aws-resources

Question: 133 CertyIQ

You have a simple Terraform configuration containing one VM (virtual machine) in a cloud provider. You run terraform apply and the VM is created successfully.
You delete the VM using the cloud provider console, then run terraform apply again without changing any Terraform code.
What happens next?

A. Terraform will report an error.
B. Terraform will recreate the VM.
C. Terraform will remove the VM from the state file.
D. Terraform will not make any changes.

Answer: B

Explanation:

Technical Justification (Certification-Style Response)

When Terraform first creates a virtual machine (VM) it records the resource in its state file and the provider's API. After the VM is removed manually from the cloud console, the external system no longer reports a resource with that identifier. On the next execution of **terraform apply** without any code changes, Terraform performs three logical steps:

1. **Refresh** – it queries the provider to compare the current state of the infrastructure with the actual resources that exist. Because the provider now reports that the named VM no longer exists, the refresh marks the resource as **missing**.
2. **Plan** – Terraform calculates a diff between the desired state (as expressed in the .tf files) and the refreshed state. The diff shows create for the VM, while all other configured resources remain unchanged.
3. **Apply** – Terraform executes the actions indicated by the plan. The missing VM is therefore recreated, and the state file is updated to reflect the newly provisioned resource.

Thus, the correct answer is **B – Terraform will recreate the VM**. This behavior is intentional: Terraform treats any divergence between declared configuration and actual infrastructure as a drift that must be reconciled, regardless of whether the drift was caused by a manual deletion, an external automation, or a provider-side change.

Why the other options are not appropriate

A. Terraform will report an error.
Errors are only raised when the configuration is syntactically invalid, references non-existent data sources, or encounters provider-level failures. A simple external deletion does not trigger an error because the configuration itself is still valid; Terraform merely detects drift.

C. Terraform will remove the VM from the state file.
terraform apply never deletes entries from the state file arbitrarily. It only updates the state to match the provider's current view. Removing a resource from state would leave the configuration in an inconsistent "orphaned" state, which is not the default behavior.

D. Terraform will not make any changes.
This would be true only if the real infrastructure already matched the desired state. Since the VM is absent from the provider but still tracked in the state file, there is a mismatch, prompting Terraform to act.

In summary, Terraform's declarative engine continuously works to achieve **convergence**: the actual infrastructure must reflect the code-defined state. External deletions introduce drift, which Terraform resolves by recreating the missing resource on the next apply.

References

terraform apply documentation – https://developer.hashicorp.com/terraform/cli/apply
State management overview – https://developer.hashicorp.com/terraform/state#state-files

(These links are official, publicly accessible HashiCorp documentation that detail the apply workflow and the behavior of Terraform's state file.)

Question: 134 CertyIQ

You have never used Terraform before and would like to test it out using a shared team account for a cloud provider. The shared team account already contains 15 VMs (virtual machines). You develop a new Terraform configuration containing one VM, perform terraform apply, and see that your VM was created successfully. How do you delete the newly-created VM with Terraform?

A. Delete the Terraform state file and execute terraform apply.

B. The Terraform state file only contains one new VM. Execute terraform destroy.

C. The Terraform state file contains all 16 VMs in the team account. Execute terraform destroy and select the newly-created VM.

D. Delete the VM using the cloud provider console, and execute terraform apply to apply the changes to the Terraform state file.

Answer: B

Explanation:

Correct option: B.The Terraform state file only contains one new VM. Executeterraform destroy.

Why option B is correct

State tracking: After terraform apply the provider-specific state file records the single resource that was newly created. Destroy reads this state and issues the appropriate delete calls for the resources listed there.
Idempotent cleanup: terraform destroy removes exactly the resources defined in the current configuration from the infrastructure and updates the state, leaving the rest of the team's 15 VMs untouched.
Single source of truth: Because the state contains only the resource you intend to delete, destroying it safely eliminates the new VM without affecting existing assets.

Why the other options are unsuitable

A. Delete the state file and runterraform apply – Removing the state discards all metadata about the resources and forces Terraform to think none exist; a subsequent apply would recreate the VM rather than delete it.
C. Assume the state holds all 16 VMs and destroy selectively – The state reflects only the resources declared in your configuration, not the entire account. Destroy will remove everything defined in the config, which in this case is just one VM; attempting to "select" a specific VM is not how Terraform operates.
D. Delete via the cloud console and runterraform apply – Manual deletion breaks the intended state management model. Terraform will then reconcile the drift on the next apply, potentially recreating the VM or

causing unnecessary state churn.

Conclusion

Use terraform destroy because it cleanly removes the resource recorded in the state, keeping the rest of the team's infrastructure intact and preserving the declarative, reproducible workflow that Terraform enforces.

References

Terraform Language Documentation – State Management:
https://developer.hashicorp.com/terraform/language/state
Terraform CLI – Destroy Command: https://developer.hashicorp.com/terraform/cli/commands/destroy

Question: 135 CertyIQ

Which parameters does the import block require? (Choose two.)

A. backend
B. The target resource address
C. provider
D. The resource ID

Answer: BD

Explanation:

Technical Justification for Correct Answer: BD

The import block in HashiCorp's Terraform requires specific parameters to successfully import existing infrastructure into Terraform's state. Here's why **B. The target resource address** and **D. The resource ID** are the correct choices, and why the other options are less suitable:

Correct Choices:

B. The target resource address: This parameter is crucial as it specifies the Terraform resource address that will manage the imported infrastructure. It tells Terraform where to place the imported resource within its configuration hierarchy (e.g., aws_instance.my_server).

D. The resource ID: This identifies the specific external resource (already existing in the cloud(provider) environment) that Terraform will import. The ID is provider-dependent (e.g., an AWS Instance ID, a Google Cloud Resource Name, etc.) and uniquely identifies the resource to be managed.

Less Suitable Options:

A. backend: While the backend configuration is vital for Terraform (defining where and how state is stored), it is not a parameter required by the import block itself. Backend settings are configured at a higher level of the Terraform project, not directly within an import block.

C. provider: Similar to the backend, the provider configuration is essential for a Terraform project (as it defines the cloud-provider/backend to interact with). However, when importing a resource, the provider is typically already configured at the project or module level. The import block focuses on the specifics of the resource to import, not re-specifying the provider.

Why BD is the Best Choice:

Precision: BD provides the exact necessities for an import operation - where in the Terraform config to place it (B) and what external resource to import (D).
Reusability andproject Structure: By not requiring the provider or backend within the import block, Terraform maintains project structure flexibility and reusability of configurations.
References:

HashiCorp Terraform Documentation - Importing Resources
Terraform Configuration - Resource Addresses

Question: 136 CertyIQ

Your team uses HCP Terraform to manage your infrastructure. You need to make a change to an infrastructure stack running in a public cloud.
Which pattern follows Infrastructure as Code best practices for making a change?

A. Make the change via the public cloud API endpoint.
B. Use the public cloud console to make the change.
C. Run the public cloud CLI tool to make the change.
D. Submit a pull request and wait for an approved merge of the change.
E. Clone the repository containing your infrastructure code and then run the code.

Answer: D

Explanation:

Justification

Option D – Submit a pull request and wait for an approved merge

Aligns with Git-based **InfrastructureasCode (IaC)** workflows: changes are version-controlled, reviewed, and tested before production.
Enforces **peer review**, **policy compliance**, and **automated checks** (e.g., Terraform lint, plan validation, Sentinel policies), which are core best-practice pillars for safe, auditable infrastructure changes.
Guarantees traceability and rollback capability through the merge-request lifecycle.

Option E – Clone the repo and run the code locally

While it uses source control, it lacks the formal review gate and automated verification steps, increasing the risk of unchecked changes reaching production.

Option A – Public-cloud API endpoint

Bypasses version control and peer review; changes are ad-hoc and not reproducible, violating IaC principles of declarative, stored-state configurations.

Option B – Public-cloud console

Similar to the API approach; manual, UI-driven changes are not versioned and are difficult to audit or roll back.

Option C – Public-cloud CLI tool

Still manual and external to the IaC repository; lacking review, testing, and auditability.

Therefore, **D** is the only option that embodies a full Git-centric, peer-reviewed workflow, making it the best practice for IaC changes in HCP Terraform-managed environments.

References

HashiCorp Learn – "Using Pull Requests with Terraform"
https://learn.hashicorp.com/tutorials/terraform/terraform-pull-requests

HCP Terraform Documentation – "Version Control and Pull Requests"
https://developer.hashicorp.com/terraform/cloud-docs/vcs/pull-requests

Question: 137 CertyIQ

Which of the following does HCP Terraform perform during a health assessment for a workspace? (Choose two.)

A. Sentinel policy checks
B. Resource drift detection
C. Estimate infrastructure cost
D. Check block validation
E. Terraform test execution

Answer: BC

Explanation:

Why optionsB andC are correct

Resource drift detection (B) – HCPTerraform continuously compares the saved workspace state with the current provider state. Any discrepancy is flagged as drift, allowing the platform to alert users when the real-world infrastructure no longer matches the configuration. This is a core component of the workspace health score.
Infrastructure cost estimate (C) – The health assessment also calculates an approximate monthly cost for the provisioned resources, using pricing data from the selected provider. This estimate is displayed alongside the health metrics so teams can gauge both operational and financial risk.

Why the other options are not part of the health assessment

Sentinel policy checks (A) – Policies are evaluated during run/apply cycles; they enforce compliance but are not a diagnostic element of the health score.
Check block validation (D) – Validation of configuration blocks occurs during the planning stage; it is a prerequisite for a successful run, not a health-assessment activity.
Terraform test execution (E) – Tests are executed via CI/CD pipelines or the terraform test command; they are separate from the automated health monitoring that HCPTerraform performs on a workspace.

References

Health checks and drift detection: https://developer.hashicorp.com/terraform/cloud-docs/workspaces#health-checks
Cost estimation in Terraform Cloud: https://developer.hashicorp.com/terraform/cloud-docs/costs

These sources describe how HCPTerraform evaluates drift and provides cost-estimation as part of its

workspace health assessment.

Question: 138 CertyIQ

You manage two workspaces in your HCP Terraform organization. The first workspace manages your network configuration. The second workspace manages your compute resources, and retrieves values from the networking workspace.
What HCP Terraform feature lets you run an apply operation on the compute workspace every time you update the networking workspace?

A. Projects
B. Policy
C. Run tasks
D. Run triggers

Answer: D

Explanation:

Answer: D – Run triggers

Why it fits: HCPTerraformrun triggers let you automatically start a run (apply) of a downstream workspace when a specific event occurs in another workspace. By defining a trigger on the compute workspace edge-case that watches the networking workspace's state or VCS changes, Terraform orchestrates an apply of the compute workspace each time the networking changes are approved.

Why the other options don't work:

A– Projects: Projects are a logical grouping of workspaces; they do not provide workflow automation for run execution.
B– Policy: Terraform policies enforce compliance and can block runs, but they do not schedule or initiate runs based on workspace updates.
C– Run tasks: Run tasks are manual actions you invoke from the UI or API; they are not event-driven and won't automatically fire on another workspace's changes.

Therefore, a **run trigger** is the precise HCP Terraform feature that automatically runs the compute workspace whenever the networking workspace is updated, which matches the described requirement.

References

Run triggers (documentation): https://developer.hashicorp.com/terraform/cloud-docs/workspaces/triggers
Workspace dependencies and automation: https://developer.hashicorp.com/terraform/cloud-docs/workspaces/run-triggers#using-run-triggers-to-chain-workspaces

Question: 139 CertyIQ

You can configure multiple cloud blocks in your Terraform configuration to connect your workspace to both HCP Terraform and your Terraform Enterprise instance.

A. True

B. False

Answer: B

Explanation:

Terraform can be configured to use **only one remote backend** (i.e., a single cloud block) per workspace.
The cloud block specifies the provider URL, organization, and optional parameters, and it **applies globally**; you cannot simultaneously target two distinct backends such as HCP Terraform and Terraform Enterprise in the same configuration.
Attempting to declare multiple cloud blocks results in a validation error ("multiple cloud blocks are not supported"), preventing the configuration from being applied.
Consequently, you can switch between a Terraform Cloud/HCP workspace and a Terraform Enterprise backend by editing the same cloud block, but you cannot have both configurations active at the same time.

Therefore, the statement "You can configure multiple cloud blocks in your Terraform configuration to connect your workspace to both HCP Terraform and your Terraform Enterprise instance." is **false**.

References

[Terraform Cloud Configuration – Remote Backend (cloud block)]
(https://developer.hashicorp.com/terraform/tutorials/cloud Terraform cloud)
Terraform Enterprise Remote Execution – Backend Configuration

Question: 140 CertyIQ

Which HCP Terraform feature is recommended when you want to use the same cloud provider credentials across multiple workspaces?

A. Variable sets
B. Version Control System (VCS) integration
C. The terraform_remote_state data source

Answer: A

Explanation:

Technical justification for selectingVariable sets

Centralized credential management – Variable sets store provider-level secrets (e.g., access_key, secret_key) in a secure, workspace-agnostic location that can be referenced from multiple workspaces, eliminating the need to duplicate credentials in each workspace's variable definitions.
Reusability across environments – Because the same set of variables is attached to many workspaces, any change to authentication material is made once and automatically propagates to all workspaces that consume the set, ensuring consistency.
Security posture – Sensitive values can be marked as **sensitive**, preventing them from being exposed in plan output or logs, and they are persisted encrypted in the Terraform Cloud backend.
Minimal impact on IaC design – The approach retains the declarative nature of Terraform: credentials are supplied as inputs, not as hard-coded values in configuration files, aligning with best-practice module encapsulation.

Why the other options are less suitable

VCS integration (OptionB) – This feature connects Terraform Cloud to a source-control repository to trigger runs; it does not address credential sharing across workspaces.
terraform_remote_state data source (OptionC) – It reads previously stored state from remote backends, which can expose state data but does not provide a mechanism for distributing provider credentials securely.

References

Terraform Cloud Variable Sets Documentation – https://developer.hashicorp.com/terraform/cloud-docs/variables/variable-sets
Provider Configuration Best Practices – https://developer.hashicorp.com/terraform/language/modules/develops/providers#defining-credentials

---

Question: 141 CertyIQ

You are responsible for a set of infrastructure which is managed by two workspaces: example-network and example-compute. The example-compute workspace uses data from output values configured in the example-network workspace, and must be deployed afterward. Currently, this is a manual process:
An operator deploys changes to the example-network workspace.
They manually copy the output values from the example-network workspace to input variables configured for the example-compute workspace.
They deploy the example-compute workspace.
Which HCP Terraform features can you use to automate this process? (Choose two.)

A. A health check configured on the example-compute workspace to create a plan when HCP Terraform applies changes to the example-network workspace.

B. A run trigger configured on the example-network workspace to automatically plan changes to the example-compute workspace after every apply.

C. A health check configured on the example-network workspace to create a plan on the example-compute workspace when HCP Terraform applies changes to it.

D. A run trigger configured on the example-compute workspace to automatically plan changes after HCP Terraform applies changes to the example-network workspace.

E. A tfe_outputs data source configured in the example-compute workspace to automatically load output values from the example-network workspace.

Answer: DE

Explanation:

Correct options: D and E

D – Run trigger on the example-compute workspace
The trigger fires **after** the example-network workspace finishes its apply. HCP Terraform detects the completed run and automatically initiates a new run (plan→apply) in the example-compute workspace. This guarantees that the latest output values are consumed, eliminating manual copy-and-paste.

E – tf_outputs data source in the example-compute workspace
By using the tf_outputs data source, the compute workspace can retrieve the exported values directly from the network workspace at run-time. No external secret-management or copy step is required; the values are fetched on each plan/apply cycle.

Why the other options are unsuitable

A – Health check that runs a plan on example-compute when example-network is applied

Health checks only monitor the health of a specific workspace or run; they do not create or trigger external runs. They cannot automatically launch a plan in a different workspace.

B – Run trigger on example-network that plans example-compute
A trigger that fires **from** the network workspace to automatically **plan** the compute workspace would only produce a plan, not the subsequent apply step. Deployments require the full run lifecycle (plan→apply), which a trigger on the source workspace does not guarantee.

C – Health check on example-network that creates a plan on example-compute
Similar to (A), health checks cannot initiate runs in other workspaces. They are limited to notifying or marking a workspace as unhealthy; they do not orchestrate cross-workspace execution.

Implementation summary
Configure a **run trigger** on the example-compute workspace that points to the example-network workspace's run type (apply). Pair this with the tf_outputs data source inside the compute workspace to ingest the network's exported outputs. This eliminates manual intervention and ensures the compute deployment always reflects the most recent network state.

References

Run Triggers (HCP Terraform)
tf_outputs Data Source

Question: 142 CertyIQ

Where does HashiCorp recommend you store API tokens and other secrets within your team's Terraform workspaces? (Choose three.)

A. In a terraform.tfvars file, checked into your version control system.
B. In an environment variable and referenced with TF_VAR_variablename.
C. In HashiCorp Vault.
D. In a plaintext document on a shared drive.
E. In an HCP Terraform variable, with the sensitive option checked.

Answer: BCE

Explanation:

Why these three options are recommended

B – Environment variables referenced as TF_VAR_<name> – Terraform can ingest secrets from the runtime environment, keeping them out of the codebase and out of version control. When the variable is marked as sensitive, Terraform masks its value in logs and plan output, reducing the risk of accidental exposure. This approach is platform-agnostic and works locally, in CI pipelines, and in cloud runners.

C – HashiCorp Vault – Vault provides a centralized, encrypted store for secrets with fine-grained access policies, dynamic secrets, and lease/renewal handling. Storing tokens there allows Terraform to retrieve credentials at runtime via the Vault provider, ensuring that secrets never reside in plaintext on disk or in source control.

E – HCP Terraform variable with the sensitive flag – When managing workspaces in HashiCorp Cloud Platform (HCP) Terraform, variables marked as sensitive are stored encrypted at rest and are only exposed to the Terraform execution environment when needed. This leverages the managed secret handling of HCP while

maintaining the same security guarantees as local environment variables.

Why the other options are unsuitable

A – terraform.tfvars checked into version control – Including secrets in a file that is committed to source control exposes them to anyone with repository access and creates a permanent audit trail. Even if the file is ignored, it is easy to mistakenly add it later, violating secret-management best practices.

D – Plaintext document on a shared drive – A shared drive typically lacks encryption at rest, granular access controls, and audit logging. Secrets stored there are visible to all users with drive permissions, increasing the attack surface and making compliance reporting difficult.

References

Terraform Configuration Language – Variables: https://developer.hashicorp.com/terraform/language/expressions#variables
Terraform Vault Provider – Secrets Management: https://developer.hashicorp.com/terraform/language/providers/vault
HCP Terraform – Sensitive Variables: https://developer.hashicorp.com/terraform/cloud-services/variables#sensitive

Question: 143 CertyIQ

Which of these statements about HCP Terraform workspaces is false?

A. You can trigger plans and applies using version control system integrations.
B. HCP Terraform workspaces offer role-based access controls.
C. You must use the CLI to switch between workspaces.
D. HCP Terraform workspaces can securely store cloud credentials.

Answer: C

Explanation:

Why OptionC is false

Switching between workspaces can be done via the Terraform CLI or through the HCP Terraform UI, API, or VCS integrations.
The CLI is optional for workspace management; it is only required when you need to run terraform workspace select locally. Therefore the statement “You must use the CLI to switch between workspaces” is inaccurate.

Why the other options are true

A. HCP Terraform integrates natively with GitHub, GitLab, Bitbucket, and AzureDevOps, allowing plans and applies to be triggered automatically on pull-request events or via VCS webhook APIs.
B. Workspace-specific policies, role-based access controls (RBAC), and team permissions are built-in features of HCP Terraform, enabling fine-grained governance of who can execute or view particular workspaces.
D. Sensitive values (e.g., API keys, DB passwords) can be stored securely in the HCP Terraform **variables** block or via the **Sensitive** flag, and the platform encrypts them at rest, providing a safe way to manage cloud credentials without exposing them in code.

Summary

OptionC mischaracterizes the mechanism for workspace switching; all other statements correctly describe capabilities of HCP Terraform workspaces.

References

Workspaces Overview –https://developer.hashicorp.com/terraform/cloud-docs/workspaces
Storing Sensitive Data –https://developer.hashicorp.com/terraform/cloud-docs/workspaces#securely-store-sensitive-data

Question: 144 CertyIQ

How does an HCP Terraform integration using the cloud block differ from remote backends, such as S3?

A. Using the cloud block automatically encrypts state files at rest and in transit.
B. Using the cloud block requires using Terraform Enterprise.
C. The cloud block supports multiple backends simultaneously.
D. Using the cloud block enables remote operations on infrastructure managed by HCP Terraform or Terraform Enterprise.

Answer: D

Explanation:

Explanation

The cloud block in HCP Terraform tells Terraform Cloud to store the state in HCP's managed storage and to expose the full set of remote operations (run-tasks, policy checks, runs, etc.) that are only available when the backend is hosted by Terraform Cloud or Enterprise. This backend eliminates the need to manage S3-style credentials, remote lock files, or VPC endpoints; all lifecycle commands (plan, apply, destroy) are mediated through the HCP API, enabling team-wide collaboration, policy enforcement, and auditability.

OptionA – Incorrect. The cloud block does not automatically encrypt state files; encryption must be configured separately (e.g., via HCP-managed encryption at rest or external KMS settings).
OptionB – Incorrect. The cloud block works with the public Terraform Cloud SaaS service as well as with self-hosted Terraform Enterprise; it does not mandate Enterprise.
OptionC – Incorrect. The backend defined by a cloud block is a single, unified backend; you cannot configure multiple distinct backends within the same configuration.
OptionD – Correct. Using the cloud block moves all state-management operations to HCP's backend, giving you remote runs, policy evaluation, workspace management, and audit logs that are absent when using generic backends like S3.

References

Terraform Cloud Backends – https://developer.hashicorp.com/terraform/tutorials/cloud-provider/cloud-backends
Manage State in Terraform Cloud – https://developer.hashicorp.com/terraform/cloud-docs/process/state

Question: 145 CertyIQ

Which features do HCP Terraform workspaces provide that are not available in Terraform Community Edition? (Choose three.)

A. Automatic detection of common security issues.
B. Remote execution of Terraform operations.
C. Support for multiple cloud providers.
D. Store your configuration in a Version Control System (VCS).
E. State versions and run history.
F. Store Terraform and environment variables in variable sets.

Answer: BEF

Explanation:

Justification

B – Remote execution of Terraform operations
HCP Terraform runs Terraform plans and applies on managed workers in the cloud, allowing users to trigger executions via the UI/API without needing a local machine or self-hosted runner. The Community Edition only supports local runs; it lacks the hosted, scalable execution engine.
E – State versions and run history
HCP Terraform automatically version-controls the workspace state and retains a detailed run history (plans, successes, failures, console output). This enables auditability and rollback of state changes. Terraform Community Edition stores state only locally and provides no built-in versioning or run log.
F – Store Terraform and environment variables in variable sets
HCP Terraform offers centralized variable sets that can be linked to workspaces, providing a secure, version-controlled store for both Terraform configuration variables and environment-specific secrets. The Community Edition requires manual handling of variables in local files or external secret stores; it does not provide a unified variable-set service.

Why the other options are not exclusive to HCP Terraform

A – Automatic detection of common security issues – This capability is not a defined feature of HCP Terraform; security scanning is typically provided by external tools (e.g., Checkov, Terraform Cloud's policy as code) and is not a unique HCP Terraform service.
C – Support for multiple cloud providers – Both the Community Edition and HCP Terraform can manage resources across any provider supported by Terraform; provider support is not limited to HCP.
D – Store your configuration in a Version Control System (VCS) – Storing config in VCS is a development practice that works with both editions; it is not a feature of HCP Terraform itself.

References

1. HashiCorp Terraform Cloud (HCP Terraform) Documentation – Workspaces & Remote Execution
2. https://developer.hashicorp.com/terraform/cloud-docs/workspaces
3. Terraform Cloud – State Versions & Run History
4. https://developer.hashicorp.com/terraform/cloud-docs/state/state-versioning
5. Terraform Variable Sets Overview
6. https://developer.hashicorp.com/terraform/cloud-docs/variables/variable-sets

Question: 146 CertyIQ

What HCP Terraform feature helps organize similar workspaces together and let you configure them as a group?

A. Variable sets

B. Change requests

C. Projects

D. Policy sets

Answer: C

Explanation:

Technical Justification for Correct Answer: C. Projects

The correct answer, **C. Projects**, is the most suitable HCP Terraform feature for organizing similar workspaces together and configuring them as a group. Here's why:

Projects in HashiCorp Cloud Platform (HCP) Terraform allow for the logical grouping of related workspaces. This feature is specifically designed to manage collections of workspaces that share a common purpose or belong to the same project context, making it easier to oversee and configure them collectively.
Key Benefit: Projects enable streamlined management through a unified view, facilitating operations like simultaneous workspace updates, shared variable sets (via integration with Variable Sets), and unified access control, all of which align with the question's requirements.

Why Other Options are Less Suitable:

A. Variable Sets: While Variable Sets are crucial for managing configuration variables across workspaces, they do not provide a mechanism for organizing workspaces themselves. Their primary function is to centralize variable management, not workspace grouping.
B. Change Requests: Change Requests are a feature focused on governance and compliance, allowing for the review of proposed Terraform plan changes before deployment. This feature does not facilitate the organization or group configuration of workspaces.
D. Policy Sets: Policy Sets are used to enforce policy as code across infrastructures, ensuring compliance with organizational standards. Like Variable Sets, they do not serve the purpose of organizing workspaces into manageable groups.

References

For in-depth information on these features, refer to the official HashiCorp documentation:

HashiCorp Cloud Platform (HCP) Terraform - Projects
HashiCorp Terraform Documentation - Variable Sets, Change Requests, Policy Sets (Navigate within for specific feature details)

Question: 147 CertyIQ

You are an administrator of your HCP Terraform organization. Recently, a member of your organization accidentally applied a change to a workspace that manages infrastructure owned by a different team. What HCP Terraform features can you use to help prevent similar accidents in the future?

A. Create separate teams in HCP Terraform for each team in your organization, then assign project and/or workspace permissions to each team.

B. Configure your workspaces to use the VCS-driven workflow, then update permissions to each repository in your VCS.

C. Enable two-factor authentication for your HCP organization, then limit access to only specific users within your organization.

D. For each workspace in your organization, edit its settings, and then assign only those users that are authorized to plan and apply changes to the workspace.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

The correct answer, **A**, is the most suitable choice for preventing similar accidents in the future due to its direct addressing of the access control and resource segregation issue described. Here's why:

Direct Access Control and Segregation: Creating separate teams in HCP Terraform for each team in your organization and then assigning project and/or workspace permissions to each team directly addresses the problem. This ensures that teams only have access to the workspaces managing infrastructure they own, preventing unauthorized changes.

Why Other Options are Less Suitable:

B. VCS-driven workflow and Repository Permissions:

While configuring workspaces to use the VCS-driven workflow can enhance version control and collaboration, updating repository permissions in the VCS does not directly prevent a user with access to a repository from applying changes to the wrong workspace in HCP Terraform if they have the necessary HCP Terraform permissions.
Issue: Indirect solution, relies on external (VCS) permission management which may not mirror HCP Terraform's access requirements.
C. Two-Factor Authentication (2FA) and Limited User Access:

Enabling 2FA enhances security by adding an extra layer of authentication but does not solve the issue of role-based access within the organization.
Limiting access to specific users might help but doesn't scale well with the requirement to manage infrastructure ownership across different teams.
Issue: Does not address the core problem of resource (workspace) access control.
D. Assigning Authorized Users per Workspace:

While this approach seems to directly address the issue by ensuring only authorized users can plan and apply changes to a workspace, it becomes impractical for large organizations with many workspaces and frequent team membership changes.
Issue: Scalability and manageability concerns; not as efficient as managing access through teams.

Correct Answer Justification in Bullet Points

Why A is Correct:

Direct Solution: Addresses the root cause (access to wrong workspace) directly.
Scalability: Easily scalable for large organizations with multiple teams.
Efficient Management: Simplifies permission management through team-based assignments.
Clear Segregation: Ensures clear ownership and access control over infrastructure.

Key Benefit Over Other Options:

Combines scalability with direct addressal of the problem, unlike the other options which either introduce external dependencies (B), fail to directly solve the access issue (C), or are less scalable (D).

References

For further reading on managing teams and permissions in HCP Terraform:

1. **HashiCorp Documentation - HCP Terraform Teams and Permissions**https://www.hashicorp.com/products/terraform/hcp/features#teams
2. **HashiCorp Learn - Managing Access to HCP Terraform**https://learn.hashicorp.com/collections/terraform/hcp-access-control

Question: 148 CertyIQ

Which statements are true?
Pick the 2 correct responses below

A. A precondition in a resource block must pass for Terraform to create the resource.
B. A precondition cannot reference attributes of another resource that are only known after apply
C. When a resource has a count meta-argument, Terraform evaluates the precondition for each instance.
D. If a resource's postcondition block fails, Terraform still creates the resources that depend on it.
E. A resource can either have a precondition or a postcondition block, but not both.

Answer: A,C

Explanation:

A. A precondition in a resource block must pass for Terraform to create the resource.

Correct. Terraform evaluates the precondition before creating the resource. If the precondition fails, the resource is not created.

B. A precondition cannot reference attributes of another resource that are only known after apply.

Incorrect. Preconditions can reference such attributes, but if the value is unknown during planning, Terraform defers evaluation until apply.

C. When a resource has a count meta-argument, Terraform evaluates the precondition for each instance.

Correct. If a resource uses count, Terraform evaluates the precondition separately for each instance.

D. If a resource's postcondition block fails, Terraform still creates the resources that depend on it.

Incorrect. If a postcondition fails, Terraform marks the resource as failed, and dependent resources are not created.

E. A resource can either have a precondition or a postcondition block, but not both.

Incorrect. A resource can have both precondition and postcondition blocks defined.

Question: 149 CertyIQ

Your team often uses API calls to create and manage cloud Infrastructure.
In what ways does Terraform differ from conventional Infrastructure management approaches?

A. Terraform replaces cloud provider APIs with its own protocols, enabling automated deployments.
B. Terraform describes infrastructure with version-controlled, repeatable configurations that specify the desired state.

C. Terraform enforces infrastructure through imperative scripts to ensure tasks are completed in the proper order.

D. Terraform is merely a wrapper for cloud provider APIs, so there is little to no difference in calling the API directly.

Answer: B

Explanation:

A. Terraform replaces cloud provider APIs with its own protocols, enabling automated deployments.

Incorrect. Terraform does not replace cloud provider APIs. It uses provider plugins that call the underlying cloud APIs.

B. Terraform describes infrastructure with version-controlled, repeatable configurations that specify the desired state.

Correct. Terraform uses declarative configuration files to define the desired state of infrastructure, enabling version control, consistency, and repeatability.

C. Terraform enforces infrastructure through imperative scripts to ensure tasks are completed in the proper order.

Incorrect. Terraform is declarative, not imperative. It determines the execution order automatically based on resource dependencies.

D. Terraform is merely a wrapper for cloud provider APIs, so there is little to no difference in calling the API directly.

Incorrect. Terraform provides state management, dependency graphing, planning, and lifecycle management, which go far beyond simply calling APIs directly.

Question: 150 CertyIQ

What's the proper syntax for the plan command?

A. terraform plan generate-config-out=tfplan

B. terraform plan out=tfplan

C. terraform plan target=tfplan

D. terraform apply-var-file=tfplan

Answer: B

Explanation:

The terraform plan command is used to create an execution plan, showing what actions Terraform will take to achieve the desired state. To save this plan to a file for later use with terraform apply, the -out flag is employed. This ensures that the apply command executes precisely the plan that was reviewed.

Question: 151 CertyIQ

Which of the following is not a way to trigger terraform destroy ?

A. terraform destroy
B. All of these will trigger terraform destroy
C. terraform plan destroy
D. terraform destroy-auto-approve

Answer: C

Explanation:

A. terraform destroy

Incorrect. This command directly triggers destruction of managed infrastructure.

B. All of these will trigger terraform destroy

Incorrect. Not all listed commands correctly trigger destroy.

C. terraform plan destroy

Correct. This is not valid syntax. The correct flag would be terraform plan -destroy.

D. terraform destroy-auto-approve

Incorrect. While written incorrectly, the valid command terraform destroy -auto-approve does trigger destroy.

Question: 152 CertyIQ

You are making changes to existing Terraform code to add some new infrastructure. When is the best time to run terraform validate?

A. After you run terraform apply, so you can validate your Infrastructure.
B. After you run terraform plan, so you can validate that your state file is consistent with your infrastructure.
C. Before you run terraform apply, so you can validate your provider credentials.
D. Before you run terraform plan, so you can validate your code syntax.

Answer: D

Explanation:

A. After you run terraform apply, so you can validate your Infrastructure.

Incorrect. terraform validate checks configuration syntax and internal consistency — it does not validate deployed infrastructure after apply.

B. After you run terraform plan, so you can validate that your state file is consistent with your infrastructure.

Incorrect. terraform validate does not check state file consistency. That is handled during plan and apply operations.

C. Before you run terraform apply, so you can validate your provider credentials.

Incorrect. terraform validate does not verify provider credentials or connectivity.

D. Before you run terraform plan, so you can validate your code syntax.

Correct. terraform validate checks configuration syntax and internal logic before running plan or apply, making it best to run prior to terraform plan.

Question: 153 CertyIQ

You want to retrieve the ID of an existing cloud resource for use in your Terraform configuration.
Which block type would you use?

A. data
B. resource
C. output
D. import

Answer: A

Explanation:

A. data

Correct. A data block is used to fetch information about existing infrastructure that is not managed by the current Terraform configuration. It allows you to reference attributes like IDs for use in other resources.

B. resource

Incorrect. A resource block is used to create and manage new infrastructure, not retrieve existing unmanaged resources.

C. output

Incorrect. An output block is used to display values after Terraform runs, not to retrieve existing resources.

D. import

Incorrect. import is a CLI command used to bring existing infrastructure under Terraform management, not a configuration block for retrieving resource data.

Question: 154 CertyIQ

A senior admin accidentally deleted some of your cloud instances. What will Terraform do when you run terraform apply?

A. Build a completely brand new set of infrastructure.
B. Rebuild only the instances that were deleted.
C. Tear down the entire workspace's infrastructure and rebuild it.
D. Stop and generate an error message about the missing instances.

Answer: B

Explanation:

A. Build a completely brand new set of infrastructure.

Incorrect. Terraform does not recreate everything unless the configuration has changed to require it.

B. Rebuild only the instances that were deleted.

Correct. Terraform compares the current state with the real infrastructure. If resources defined in the configuration are missing, Terraform will plan to recreate only those missing resources.

C. Tear down the entire workspace's infrastructure and rebuild it.

Incorrect. Terraform does not destroy unaffected resources unless explicitly instructed.

D. Stop and generate an error message about the missing instances.

Incorrect. Instead of stopping, Terraform detects drift and plans to restore the missing resources.

Question: 155 CertyIQ

What is the purpose of the .terraform directory in a Terraform workspace?

A. The directory contains plugins and modules that Terraform downloads during initialization, along with other important information.

B. The directory is where Terraform creates and maintains the state file to track the underlying resources it creates and manages.

C. The directory contains the provider credentials and the .tfvars files to prevent them from being committed to version control by accident

D. The directory is used to convert and store Terraform configuration files into API calls to communicate with the targeted platform.

Answer: A

Explanation:

A. The directory contains plugins and modules that Terraform downloads during initialization, along with other important information.

Correct. The .terraform directory is created during terraform init and stores downloaded provider plugins, modules, and other metadata needed for Terraform to function.

B. The directory is where Terraform creates and maintains the state file to track the underlying resources it creates and manages.

Incorrect. The state file (terraform.tfstate) is stored in the working directory by default, not inside the .terraform directory (unless using specific backend configurations).

C. The directory contains the provider credentials and the .tfvars files to prevent them from being committed to version control by accident.

Incorrect. Credentials and .tfvars files are managed separately and are not automatically stored in the .terraform directory.

D. The directory is used to convert and store Terraform configuration files into API calls to communicate with the targeted platform.

Incorrect. Terraform does not convert configuration files into stored API calls in this directory. It dynamically

interacts with provider APIs during plan and apply.

Question: 156 CertyIQ

Your Terraform configuration manages a resource that requires maximum uptime. You need to update the resource, and when you run terraform plan, Terraform notes that the update requires the resource to be destroyed and recreated.
Which lifecycle rule can you add to the resource to reduce the downtime of the resource while still applying the update?

A. ignore_changes = all
B. create_before_destroy = true
C. destroy = false
D. prevent_destroy = true

Answer: B

Explanation:

A. ignore_changes = all

Incorrect. This tells Terraform to ignore changes to attributes, which would prevent the update from being applied at all — not reduce downtime.

B. create_before_destroy = true

Correct. This lifecycle rule instructs Terraform to create the new resource first and then destroy the old one. This helps minimize downtime during updates that require replacement.

C. destroy = false

Incorrect. There is no valid destroy lifecycle argument in Terraform.

D. prevent_destroy = true

Incorrect. This prevents Terraform from destroying the resource entirely, which would cause the apply operation to fail instead of updating it.

Question: 157 CertyIQ

Which of the following locations can Terraform use as a private source for modules?
Pick the 2 correct responses below

A. Private repository on GitHub.
B. Public repository on GitHub.
C. Internally hosted VCS (Version Control System) platform.
D. Public Terraform Registry.

Answer: A,C

Explanation:

A. Private repository on GitHub.

Correct. Terraform can use a private GitHub repository as a module source, provided proper authentication is configured.

B. Public repository on GitHub.

Incorrect. While valid as a module source, it is not considered a private source.

C. Internally hosted VCS (Version Control System) platform.

Correct. Terraform can pull modules from internally hosted VCS platforms such as private Git servers or enterprise Git solutions.

D. Public Terraform Registry.

Incorrect. The public Terraform Registry is not a private module source.

Question: 158 CertyIQ

You want to bring in an existing database under Terraform management. What information is required to create a new import block for the database?
Pick the 2 correct responses below

A. The connection string that Terraform will use to connect and manage the database.
B. The ID associated with the current database on the cloud provider.
C. The path to the .tf file that contains the database resource block.
D. The database platform and version that is the existing resource is running.
E. The destination resource address of the block that will manage the database.

Answer: B,E

Explanation:

A. The connection string that Terraform will use to connect and manage the database.

Incorrect. Terraform does not require a connection string for importing a resource. Importing uses the provider and resource configuration already defined.

B. The ID associated with the current database on the cloud provider.

Correct. Terraform needs the unique ID of the existing resource in the cloud provider to map it into the state.

C. The path to the .tf file that contains the database resource block.

Incorrect. Terraform does not need the specific file path — it only needs the resource defined in the configuration.

D. The database platform and version that the existing resource is running.

Incorrect. Terraform does not require platform/version details for importing, though it may need the correct provider configuration.

E. The destination resource address of the block that will manage the database.

Correct. Terraform requires the resource address (e.g., aws_db_instance.mydb) to know which resource block

will manage the imported resource.

Question: 159 CertyIQ

You have a saved execution plan containing desired changes for infrastructure managed by Terraform. After running the command terraform apply my.tfplan, you receive the error shown in the Exhibit space on this page.
How can you apply the desired changes?
Pick the 2 correct responses below:

A. Refresh the current state data using the -refresh-only flag.
B. Run terraform apply without the saved execution plan.
C. Update the current plan file using the terraform state push command.
D. Generate a new execution plan file with terraform plan, and apply the new plan.
E. Force the apply command by adding the flag -lock-false

Answer: B,D

Explanation:

A. Refresh the current state data using the -refresh-only flag.

Incorrect. -refresh-only updates the state file with the current real-world infrastructure but does not apply a saved plan.

B. Run terraform apply without the saved execution plan.

Correct. Running terraform apply without specifying a plan will generate a new plan automatically based on the current state and apply it, avoiding errors from a stale plan.

C. Update the current plan file using the terraform state push command.

Incorrect. terraform state push is used to manually upload a state file; it cannot modify or update a saved execution plan.

D. Generate a new execution plan file with terraform plan, and apply the new plan.

Correct. Creating a fresh execution plan ensures that the plan reflects the current state of infrastructure and can be applied successfully.

E. Force the apply command by adding the flag -lock-false

Incorrect. -lock=false disables state locking but does not fix errors caused by an outdated or invalid execution plan.

Question: 160 CertyIQ

You've updated your Terraform configuration, and you need to preview the proposed changes to your infrastructure.
Which command should you run?

A. terraform validate
B. terraform get
C. terraform plan

D. terraform show

Answer: C

Explanation:

A. terraform validate

Incorrect. This checks the syntax and internal consistency of the Terraform configuration, but does not show proposed changes to infrastructure.

B. terraform get

Incorrect. This command downloads and updates modules referenced in your configuration, but does not preview changes.

C. terraform plan

Correct. terraform plan generates an execution plan showing what changes Terraform will make to match the configuration to the desired state.

D. terraform show

Incorrect. This displays the current state or a saved plan, but does not generate a new preview of proposed changes.

Question: 161

Exhibit
resource "aws_instance" "web"
count = 2
name = "terraform-$ count.index "

A resource block is shown in the Exhibit space of this page.
How do you reference the name value of the second instance of this resource?

A. aws_instance.web.*.name
B. aws_instance.web[2].name
C. aws_instance.web[1].name
D. element(aws_instance.web, 2)
E. aws_instance.web[1]

Answer: C

Explanation:

A. aws_instance.web.*.name

Incorrect. This returns a **list of all name values** for all instances, not a single instance.

B. aws_instance.web[2].name

Incorrect. Terraform uses **zero-based indexing**, so index 2 would refer to a third instance, which doesn't exist here.

C. aws_instance.web[1].name

Correct. Terraform indexes instances starting at 0, so 1 refers to the **second instance**. This retrieves its name attribute.

D. element(aws_instance.web, 2)

Incorrect. This would return the **third instance**, not the second, due to zero-based indexing.

E. aws_instance.web[1]

Incorrect. This references the **second instance object itself**, but does not access the name attribute. You need .name to get the value.

Question: 162 CertyIQ

Which is a benefit of the Terraform state file?

A. A state file can schedule recurring infrastructure tasks.
B. A state file is a source of truth for resources provisioned with a public cloud console.
C. A state file is a source of truth for resources provisioned with Terraform.
D. A state file is the desired state expressed by the Terraform code files.

Answer: C

Explanation:

A. A state file can schedule recurring infrastructure tasks.

Incorrect. The state file does not schedule tasks; it only tracks the current state of resources.

B. A state file is a source of truth for resources provisioned with a public cloud console.

Incorrect. The state file only tracks resources managed by Terraform, not those created manually in the cloud console.

C. A state file is a source of truth for resources provisioned with Terraform.

Correct. The state file records the current state of all resources managed by Terraform and is used to plan and apply updates accurately.

D. A state file is the desired state expressed by the Terraform code files.

Incorrect. The desired state is expressed in the Terraform configuration files (.tf files). The state file reflects the **actual deployed state**, not just the desired state.

Question: 163 CertyIQ

Running terraform fmt without any flags in a directory with Terraform configuration files will check the formatting of those files, but will never change them.

A. True
B. False

Answer: B

Explanation:
Running terraform fmt without any flags will check the formatting and will automatically overwrite the files with the standardized formatting. To only check without changing, the -check or -diff flags must be used

Question: 164 CertyIQ

Which are advantages of IaC (Infrastructure as Code) patterns instead of manual infrastructure management? Pick the 2 correct responses below

A. Eliminating the need for monitoring.
B. Managing configuration with a GUI.
C. Automating infrastructure deployments.
D. Version controlling infrastructure changes.

Answer: C,D

Explanation:

A. Eliminating the need for monitoring.

Incorrect. IaC does not remove the need for monitoring. Monitoring is still required to ensure infrastructure is healthy and performing as expected.

B. Managing configuration with a GUI.

Incorrect. IaC focuses on code-based, declarative configuration, not GUI management.

C. Automating infrastructure deployments.

Correct. IaC enables automated creation, updating, and teardown of infrastructure, reducing manual intervention and errors.

D. Version controlling infrastructure changes.

Correct. IaC allows infrastructure definitions to be stored in version control systems, enabling tracking, rollback, and collaboration.

Question: 165 CertyIQ

The Exhibit section of this page shows part of a configuration you've been asked to update.
data "azurerm_resource_group" "example"
name = var.resource_group_name

resource "azurerm_virtual_network" "example"
name =

The name of the Azure Virtual Network should be set to the name of the resource group followed by a dash and the word "vnet".
Which expression fulfills this requirement?

A. "$ azurerm_resource_group.example.name -vnet"

B. join("-", var.resource_group_name, "vnet")

C. "$ data.azurerm_resource_group.example.name -vnet"

D. concat(data.azurerm_resource_group.example.name, "-", "vnet")

Answer: C

Explanation:

A. "$ azurerm_resource_group.example.name -vnet"

Incorrect. azurerm_resource_group.example does **not exist** in the configuration. The resource group is referenced as a **data block**, not a resource block.

B. join("-", var.resource_group_name, "vnet")

Incorrect. join expects a **list**, not individual string arguments, so this would cause an error.

C. "$ data.azurerm_resource_group.example.name -vnet"

Correct. The data block data.azurerm_resource_group.example holds the existing resource group. Using interpolation, this expression concatenates the resource group name with -vnet.

D. concat(data.azurerm_resource_group.example.name, "-", "vnet")

Incorrect. concat is used for **lists**, not strings, so this would cause an error in Terraform.

Question: 166 CertyIQ

You have set the TF_LOG_PATH environment variable for Terraform, and you would like to ensure the logs contain all debug-level messages and verbose process logs.
Which action should you take?

A. Run the terraform output command.

B. Add a log argument to the terraform block.

C. Update the Terraform CLI configuration file.

D. Set Set the the TF TF LOG LOG environment variable to TRACE

Answer: D

Explanation:

A. Run the terraform output command.

Incorrect. terraform output only displays output values from the state; it does not control logging.

B. Add a log argument to the terraform block.

Incorrect. Terraform configuration blocks do not have a log argument to control logging.

C. Update the Terraform CLI configuration file.

Incorrect. While you can configure some CLI defaults, debug-level logging is controlled via environment variables, not the CLI config.

D. Set the TF_LOG environment variable to TRACE

Correct. To capture all debug-level and verbose process logs, you set:

TF_LOG=TRACE

This ensures Terraform generates detailed logging at the TRACE level, which includes debug messages and internal operations. Combined with TF_LOG_PATH, the logs will be written to the specified file.

Question: 167 CertyIQ

Which of these workflows is only enabled by the use of Infrastructure as Code?

A. Cost optimization of infrastructure deployment.
B. Automatic scaling of resources based on application load.
C. Reviewing the proposed changes for potential security issues.
D. Role-based access control of cloud resources.

Answer: C

Explanation:

A. Cost optimization of infrastructure deployment.

Incorrect. Cost optimization can be achieved with or without IaC, through careful resource planning and monitoring.

B. Automatic scaling of resources based on application load.

Incorrect. Auto-scaling is a cloud feature and does not require IaC.

C. Reviewing the proposed changes for potential security issues.

Correct. IaC enables workflows like **plan-time review** of infrastructure changes, allowing teams to inspect proposed modifications (via terraform plan, code review, or automated security checks) before they are applied. This workflow is only possible because the infrastructure is defined as code.

D. Role-based access control of cloud resources.

Incorrect. RBAC is a cloud platform feature and can be managed manually; IaC is not required for it.

Question: 168 CertyIQ

Terraform can only manage resource dependencies if you set them explicitly with the depends_on argument.

A. True
B. False

Answer: B

Explanation:

Terraform automatically infers most resource dependencies based on **references between resources** in your configuration (e.g., using attributes of one resource in another). You only need to use depends_on for **explicit,**

manual dependencies that Terraform cannot automatically detect.

Question: 169 CertyIQ

Your team uses HCP Terraform to manage your infrastructure. You need to make a change to an Infrastructure stack running in a public cloud.
Which pattern follows Infrastructure as Code best practices for making a change?

A. Use the public cloud console to make the change.
B. Submit a pull request and wait for an approved merge of the change.
C. Run the public cloud CLI tool to make the change.
D. Make the change via the public cloud API endpoint.
E. Clone the repository containing your infrastructure code and then run the code.

Answer: B

Explanation:

A. Use the public cloud console to make the change.

Incorrect. Making changes directly in the console bypasses version control and Terraform, breaking IaC principles.

B. Submit a pull request and wait for an approved merge of the change.

Correct. Following IaC best practices means all infrastructure changes are made through **version-controlled code**, reviewed via pull requests, and applied automatically (e.g., via HCP Terraform). This ensures traceability, collaboration, and reproducibility.

C. Run the public cloud CLI tool to make the change.

Incorrect. Direct CLI changes bypass Terraform and version control.

D. Make the change via the public cloud API endpoint.

Incorrect. Direct API changes are outside Terraform's control and violate IaC best practices.

E. Clone the repository containing your infrastructure code and then run the code.

Incorrect. While you need to clone the repo to work on code, running it directly without a pull request bypasses code review and collaboration workflows.

Question: 170 CertyIQ

You can configure multiple cloud blocks in your Terraform configuration to connect your workspace to both HCP Terraform and your Terraform Enterprise instance.

A. True
B. False

Answer: B

Explanation:

Terraform **does not allow multiple cloud blocks** in a single configuration. You can only configure **one remote backend per workspace** to connect either to HCP Terraform **or** Terraform Enterprise, but not both simultaneously.

Question: 171 CertyIQ

What feature stops multiple users from operating on the Terraform state at the same time?

A. State locking
B. Remote state storage
C. Version control
D. Provider constraints

Answer: A

Explanation:

A. State locking

Correct. **State locking** prevents multiple users from making concurrent changes to the Terraform state. When a user applies changes, Terraform locks the state to ensure no other operations interfere, avoiding conflicts and corruption.

B. Remote state storage

Incorrect. Remote state storage allows multiple users to access the state file, but by itself does not prevent concurrent modifications — locking is needed.

C. Version control

Incorrect. Version control tracks changes to Terraform configuration files, not the live state of infrastructure.

D. Provider constraints

Incorrect. Provider constraints control which versions of a provider can be used, not state concurrency.

Question: 172 CertyIQ

which features do HCP Terraform workspaces provide that are not available in Terraform Community Edition? Pick the 3 correct responses below

A. Support for multiple cloud providers.
B. Automatic detection of common security issues.
C. Remote execution of Terraform operations.
D. Store Terraform and environment variables in variable sets.
E. Store your configuration in a Version Control System (VCS).
F. State versions and run history.

Answer: B,C,D

Explanation:

A. Support for multiple cloud providers.

Incorrect. Terraform CE (Community Edition) can already manage multiple cloud providers; this is not unique to HCP Terraform.

B. Automatic detection of common security issues.

Correct. HCP Terraform can integrate with **policy checks** (like Sentinel) to automatically detect misconfigurations or security issues before applying changes. This feature is not available in Terraform CE.

C. Remote execution of Terraform operations.

Correct. HCP Terraform workspaces can **execute plan and apply remotely**, so users do not need to run Terraform locally. This ensures consistent execution environments and reduces setup overhead.

D. Store Terraform and environment variables in variable sets.

Correct. HCP Terraform allows storing **variables and environment variables in centralized variable sets**, which can be reused across multiple workspaces and runs. Terraform CE does not provide this centralized management.

E. Store your configuration in a Version Control System (VCS).

Incorrect. Both Terraform CE and HCP Terraform can pull configurations from a VCS, so this is not a feature unique to HCP Terraform.

F. State versions and run history.

Incorrect. While HCP Terraform **does track state and run history**, the way this option is phrased makes it misleading as a "feature not available in Terraform CE." Terraform CE can also maintain state files and you can track changes locally. HCP adds convenience and remote visibility, but **state versioning itself is not exclusive to HCP Terraform**.

Question: 173

What does Terraform use to deploy infrastructure for different cloud providers?

A. Custom APIs developed by HashiCorp
B. Vendor-specific providers
C. Vendors CLI tools
D. Vendors UI

Answer: B

Explanation:

A. Custom APIs developed by HashiCorp

Incorrect. Terraform does not replace or implement its own APIs for cloud providers.

B. Vendor-specific providers

Correct. Terraform uses **providers** that are specific to each cloud vendor (like AWS, Azure, GCP). These providers translate Terraform configuration into API calls for the respective cloud platform.

C. Vendor CLI tools

Incorrect. Terraform does not rely on vendor CLI tools to create resources; it communicates directly through the provider APIs.

D. Vendor UI

Incorrect. Terraform does not use the cloud vendor's UI; everything is done programmatically via providers.

Question: 174 CertyIQ

When should you use the force-unlock command?

A. When apply has failed due to a state lock.
B. When you see a status message stating that you cannot acquire the lock.
C. When automatic unlocking has failed.
D. When you have a high priority change.

Answer: C

Explanation:

A. When apply has failed due to a state lock.

Incorrect. A failed apply does not automatically mean the lock is stale. The lock might still be held by another active operation. Using force-unlock here without verifying could corrupt the state.

B. When you see a status message stating that you cannot acquire the lock.

Incorrect. This message only indicates that another operation currently holds the lock. You should **not** force-unlock in this case, as it could interrupt an active process and corrupt the state.

C. When automatic unlocking has failed.

Correct. terraform force-unlock is intended for situations where Terraform fails to release a lock automatically, such as after a crash or interrupted operation. This safely clears a stale lock so future operations can proceed.

D. When you have a high priority change.

Incorrect. Force-unlock should **never** be used simply to prioritize a change. Doing so could interfere with ongoing operations and corrupt the state.

Question: 175 CertyIQ

You manage two workspaces in your HCP Terraform organization. The first workspace manages your network configuration. The second workspace manages your compute resources, and retrieves values from the networking workspace.
What HCP Terraform feature lets you run an apply operation on the compute workspace every time you update the networking workspace?

A. Run triggers

B. Projects
C. Policy
D. Run tasks

Answer: A

Explanation:

A. Run triggers

Correct. **Run triggers** allow one workspace to automatically trigger a run in another workspace when its outputs change. In this case, updating the networking workspace can trigger an apply in the compute workspace that depends on its outputs.

B. Projects

Incorrect. Projects in HCP Terraform are organizational units to group workspaces, but they do not trigger runs between workspaces.

C. Policy

Incorrect. Policies (e.g., Sentinel) enforce rules on runs and resources but do not automate workspace triggers.

D. Run tasks

Incorrect. Run tasks are actions executed during a run (like notifications or scripts), but they don't automatically trigger another workspace to apply changes.

Question: 176

What does terraform destroy do?

A. Destroys the Terraform state file, leaves the infrastructure intact.
B. Destroys all Terraform code files in the current directory, leaves the state file intact.
C. Destroys all infrastructure in the Terraform state file.
D. Destroys all infrastructure in the configured Terraform provider.

Answer: C

Explanation:

A. Destroys the Terraform state file, leaves the infrastructure intact.

Incorrect. terraform destroy does **not delete the state file**; it uses the state file to determine which resources to destroy.

B. Destroys all Terraform code files in the current directory, leaves the state file intact.

Incorrect. Terraform never deletes your configuration files.

C. Destroys all infrastructure in the Terraform state file.

Correct. terraform destroy reads the state file and **removes all resources tracked by that state**, effectively

tearing down the infrastructure managed by Terraform.

D. Destroys all infrastructure in the configured Terraform provider.

Incorrect. It only destroys resources tracked in the current state file, not all resources in the provider account.

Question: 177 CertyIQ

You're refactoring your Terraform configuration and have moved resources from one large module to multiple smaller ones. When running terraform plan, Terraform wants to destroy and recreate the resources.
What should you add to the configuration to preserve the existing resources?

A. Add a depends_on argument to each resource you move.
B. Add moved blocks for each resource.
C. Add lifecycle prevent_destroy = true to each of the resources.

Answer: B

Explanation:

A. Add a depends_on argument to each resource you move.

Incorrect. depends_on only controls the order in which resources are created or destroyed. It does not preserve existing resources when they are moved between modules.

B. Add moved blocks for each resource.

Correct. **moved blocks** tell Terraform that a resource has been relocated within the configuration (e.g., to a new module or different resource address). This preserves the existing resources in the state and prevents them from being destroyed and recreated.

C. Add lifecycle prevent_destroy = true to each of the resources.

Incorrect. prevent_destroy stops Terraform from destroying a resource entirely, but it does **not update the state or track that a resource was moved**. Without a moved block, Terraform would still try to destroy and recreate the resource.

Question: 178 CertyIQ

A module block is shown in the Exhibit space on this page.
module "vpc"
source = "terraform-aws-modules/vpc/aws"
version = "~>4.0".

That module block limits the module version to major version 4.

A. True
B. False

Answer: A

Explanation:

The version constraint ~> 4.0 in the module block ensures that Terraform will use any version greater than or equal to 4.0.0 but less than 5.0.0. This effectively locks the module to major version 4, allowing minor and patch updates while preventing automatic upgrades to version 5 or higher.

Question: 179 CertyIQ

Terraform requires using a different provider for each cloud provider where you want to deploy resources.

A. True
B. False

Answer: A

Explanation:

Terraform uses **provider plugins** to communicate with different cloud platforms. Each cloud provider (like AWS, Azure, or GCP) requires its own provider configuration. You cannot use a single provider to deploy resources across multiple clouds; you must configure a separate provider for each.

Question: 180 CertyIQ

You want to split a large main.tf file into multiple files for better organization within the same working directory. What impact should you expect when you run a terraform plan?

A. Significant changes should be expected since moving resources between files can trigger a destroy/replacement.
B. No impact since Terraform reads all .tf files as a single configuration when executing a terraform plan.
C. Only minor impacts since Terraform parses the main.tf file first, which could change the order in which resources are created.
D. The terraform plan will fail since variable and provider blocks must appear before resource blocks.

Answer: B

Explanation:

A. Significant changes should be expected since moving resources between files can trigger a destroy/replacement.

Incorrect. Splitting files does **not** change resource addresses or state, so Terraform does not plan to destroy or recreate resources just because they are in different files.

B. No impact since Terraform reads all .tf files as a single configuration when executing a terraform plan.

Correct. Terraform **merges all .tf files in the working directory** into a single configuration before planning or applying. The order of files does not affect resource management.

C. Only minor impacts since Terraform parses the main.tf file first, which could change the order in which resources are created.

Incorrect. Terraform determines resource creation order based on **dependencies**, not file parsing order.

Splitting files does not affect execution order.

D. The terraform plan will fail since variable and provider blocks must appear before resource blocks.

Incorrect. Terraform does not require a specific order for blocks across files. Variable, provider, and resource blocks can appear in any file in the directory.

Question: 181 CertyIQ

As a developer, you want to ensure your plugins are up-to-date with the latest versions. Which Terraform command should you use?

A. terraform refresh -upgrade
B. terraform apply -upgrade
C. terraform providers -upgrade
D. terraform init -upgrade

Answer: D

Explanation:

A. terraform refresh -upgrade

Incorrect. terraform refresh updates the state with real-world infrastructure but does not upgrade provider plugins.

B. terraform apply -upgrade

Incorrect. terraform apply applies changes to infrastructure; the -upgrade flag is not valid here.

C. terraform providers -upgrade

Incorrect. terraform providers lists providers and their versions, but it does not upgrade them.

D. terraform init -upgrade

Correct. Running terraform init -upgrade upgrades all provider plugins to the latest allowed versions according to your version constraints, ensuring your plugins are up-to-date.

Question: 182 CertyIQ

Your security team scanned some Terraform workspaces and found secrets stored in plaintext in state files. How can you protect that data?

A. Delete the state file every time you run Terraform.
B. Store the state in an encrypted backend.
C. Always store your secrets in a secrets.tfvars file.
D. Edit your state file to scrub out the sensitive data.

Answer: B

Explanation:

A. Delete the state file every time you run Terraform.

Incorrect. Deleting the state file would break Terraform's ability to track resources and manage infrastructure. This is not a viable solution.

B. Store the state in an encrypted backend.

Correct. Using an **encrypted remote backend** (like Terraform Cloud, HCP Terraform, or an encrypted S3 bucket) protects sensitive data in the state file at rest, preventing exposure of secrets.

C. Always store your secrets in a secrets.tfvars file.

Incorrect. Storing secrets in .tfvars files in plaintext does not protect them; it can make the problem worse if the files are committed to version control.

D. Edit your state file to scrub out the sensitive data.

Incorrect. Manually editing state files is risky and error-prone. Terraform may lose track of resources, and sensitive data could still be exposed elsewhere.

Question: 183 CertyIQ

You can reference a resource created with for_each using a splat (*) expression.

A. True
B. False

Answer: B

Explanation:

When a resource is created using **for_each**, Terraform assigns **each instance a unique key**. You reference these instances using their **map keys**, like resource_name["key"].attribute.

Splat expressions (*) are used for resources with **count**, which creates a list of instances, not a map. Therefore, splat expressions do **not work with for_each**.

Question: 184 CertyIQ

You add a new resource to an existing Terraform configuration, but do not update the version constraint in the configuration. The existing and new resources use the same provider. The working directory contains a .terraform.lock.hcl file.
How will Terraform choose which version of the provider to use?

A. Terraform will use the latest version of the provider for the new resource and the version recorded in the lock file to manage existing resources.
B. Terraform will use the version recorded in your lock file.
C. Terraform will check your state file to determine the provider version to use.
D. Terraform will use the latest version of the provider available at the time you provision your new resource.

Answer: B

Explanation:

A. Terraform will use the latest version of the provider for the new resource and the version recorded in the lock file to manage existing resources.

Incorrect. Terraform does **not mix provider versions** between resources in the same configuration. The lock file ensures a consistent version for all resources using that provider.

B. Terraform will use the version recorded in your lock file.

Correct. The .terraform.lock.hcl file pins the provider version for the workspace. Terraform will use the **exact version recorded in the lock file** for all resources, both existing and new, unless you explicitly update the lock file.

C. Terraform will check your state file to determine the provider version to use.

Incorrect. The state file tracks deployed resources, but **provider versions are controlled by the lock file**, not the state.

D. Terraform will use the latest version of the provider available at the time you provision your new resource.

Incorrect. Terraform does **not automatically upgrade** to the latest provider version if a lock file exists. You must explicitly run terraform init -upgrade to update.

Question: 185 CertyIQ

All standard backend types support state storage, locking, and remote operations like plan, apply, and destroy.

A. True
B. False

Answer: B

Explanation:
Not all standard backend types in Terraform support state locking and remote operations. For example, the local backend only stores the state on disk and does not provide locking or remote execution. Backends like S3 with DynamoDB, Terraform Cloud, or HCP Terraform do support locking and remote operations. Therefore, it is incorrect to assume that all backends provide full state management and remote capabilities.

Question: 186 CertyIQ

A resource block is shown in the Exhibit section of this page.

```
resource "kubernetes_namespace" "example" {
  name = "test"
}
```

How would you reference the attribute name of this resource in HCL?

A. data.kubernetes_namespace.name

B. kubernetes_namespace.example.name
C. kubernetes_namespace.test.name
D. resource.kubernetes_namespace.example.name

Answer: B

Explanation:

A. data.kubernetes_namespace.name

Incorrect. This syntax is used for **data sources**, not resources.

B. kubernetes_namespace.example.name

Correct. The resource is defined as kubernetes_namespace.example, so you reference its name attribute with kubernetes_namespace.example.name.

C. kubernetes_namespace.test.name

Incorrect. The resource is named example, not test.

D. resource.kubernetes_namespace.example.name

Incorrect. The resource keyword is only used in the configuration block, not when referencing an attribute.

Question: 187

CertyIQ

Which argument(s) are required when declaring a Terraform variable?

A. type
B. default
C. description
D. All of the above
E. None of the above

Answer: E

Explanation:

A. type

Incorrect. Specifying a type is optional; Terraform can infer the type from the default value or usage.

B. default

Incorrect. A default value is optional. If omitted, the variable must be provided during terraform apply or via a .tfvars file.

C. description

Incorrect. A description is optional and used for documentation purposes only.

D. All of the above

Incorrect. None of these arguments are strictly required.

E. None of the above

Correct. When declaring a variable in Terraform, you can simply write variable "name" without specifying type, default, or description. All arguments are optional.

Question: 188 CertyIQ

terraform apply is failing with the following error. What next step should you take to determine the root cause of the problem?
Error loading state: AccessDenied: Access Denied status code: 403, request id: 288766CE5CCA24A0, host id: web.example.com

A. Run terraform login to reauthenticate with the provider.
B. Review /var/log/terraform.log for error messages.
C. Review syslog for Terraform error messages.
D. Set TF_LOG=DEBUG.

Answer: D

Explanation:

A. Run terraform login to reauthenticate with the provider.

Incorrect. terraform login is used for Terraform Cloud or HCP authentication, not for provider-level access issues.

B. Review /var/log/terraform.log for error messages.

Incorrect. Terraform does not automatically log to /var/log/terraform.log.

C. Review syslog for Terraform error messages.

Incorrect. Terraform does not write errors to the system log.

D. Set TF_LOG=DEBUG.

Correct. Setting TF_LOG=DEBUG enables detailed logging of Terraform operations, including provider API requests. This will help identify the root cause of the **AccessDenied (403)** error.

Question: 189 CertyIQ

Which Terraform command checks that your configuration syntax is correct?

A. terraform show
B. terraform init
C. terraform fmt
D. terraform validate

Answer: D

Explanation:

A. terraform show

Incorrect. terraform show displays the current state or a saved plan; it does not check configuration syntax.

B. terraform init

Incorrect. terraform init initializes a working directory and downloads providers or modules, but it does not validate syntax.

C. terraform fmt

Incorrect. terraform fmt formats Terraform configuration files but does not validate syntax.

D. terraform validate

Correct. terraform validate checks the Terraform configuration for **syntax errors and internal consistency** without contacting any remote providers.

Question: 190 CertyIQ

How does the HCP Terraform/Terraform Cloud integration differ from backends such as S3, Consul, etc.?

A. It can execute Terraform runs on dedicated infrastructure in HCP Terraform/Terraform Cloud.
B. It doesn't show the output of a terraform apply locally.
C. It is only available to paying customers.
D. All of the above.

Answer: A

Explanation:

A. It can execute Terraform runs on dedicated infrastructure in HCP Terraform/Terraform Cloud.

Correct. Unlike backends like S3 or Consul, HCP Terraform and Terraform Cloud can **run Terraform operations remotely** on managed infrastructure, not just store state.

B. It doesn't show the output of a terraform apply locally.

Incorrect. While runs are executed remotely, you can still view outputs locally or retrieve them using Terraform commands.

C. It is only available to paying customers.

Incorrect. Terraform Cloud offers a free tier, so this is not strictly true.

D. All of the above.

Incorrect. Only option A correctly describes the difference.

Question: 191 CertyIQ

Which of these actions will prevent two Terraform runs from changing the same state file at the same time?

A. Delete the state before running Terraform.
B. Refresh the state after running Terraform.
C. Run Terraform with parallelism set to 1.
D. Configure state locking for your state backend.

Answer: D

Explanation:

A. Delete the state before running Terraform.

Incorrect. Deleting the state would break Terraform's ability to track resources and would not prevent concurrent modifications safely.

B. Refresh the state after running Terraform.

Incorrect. Refresh updates the state with the real infrastructure but does **not prevent concurrent operations**.

C. Run Terraform with parallelism set to 1.

Incorrect. This only limits the number of concurrent resource operations **within a single run**, but it does not prevent multiple runs from accessing the same state simultaneously.

D. Configure state locking for your state backend.

Correct. **State locking** ensures that only one Terraform operation can modify a state file at a time, preventing conflicts and corruption.

Question: 192

When should you use the force-unlock command?

A. A. When apply has failed due to a state lock.
B. When automatic unlocking has failed.
C. When you see a status message stating that you cannot acquire the lock.
D. When you have a high priority change.

Answer: B

Explanation:

A. When apply has failed due to a state lock.

Incorrect. A failed apply does not automatically mean the lock is stale. The lock may still be held by another active operation, so force-unlock should not be used blindly.

B. When automatic unlocking has failed.

Correct. terraform force-unlock is intended for situations where Terraform **fails to automatically release a state lock**, such as after a crash or interrupted operation. This safely clears the lock so future operations can proceed.

C. When you see a status message stating that you cannot acquire the lock.

Incorrect. This message alone does not indicate a stale lock. If another operation is legitimately holding the lock, forcing unlock could corrupt the state.

D. When you have a high priority change.

Incorrect. Force-unlock should **never** be used just to prioritize a change, as it could interfere with ongoing operations and corrupt the state.

Question: 193 CertyIQ

Which of the following is not a benefit of adopting infrastructure as code?

A. Versioning
B. Automation
C. Reusability of code
D. A Graphical User Interface

Answer: D

Explanation:

A. Versioning

Incorrect. Versioning is a benefit of IaC because infrastructure definitions are stored as code and can be tracked in version control systems.

B. Automation

Incorrect. IaC enables automated provisioning, updates, and teardown of infrastructure, which is a core benefit.

C. Reusability of code

Incorrect. Modules and reusable configurations allow IaC to be applied across multiple projects, environments, or teams.

D. A Graphical User Interface

Correct. IaC focuses on **code-based configuration**, not GUIs. Using a GUI is not a benefit of adopting infrastructure as code.

Question: 194 CertyIQ

You used Terraform to create an ephemeral development environment in the cloud and are now ready to destroy all the infrastructure described by your Terraform configuration. To be safe, you would like to first see all the infrastructure that Terraform will delete.
Which command should you use to show all the resources that will be deleted? (Choose two.)

A. Run terraform state rm *.
B. Run terraform destroy.This will output all the resources that will be deleted before prompting for approval.
C. Run terraform plan -destroy.
D. Run terraform show -destroy.

Answer: B,C

Explanation:

***A. Run terraform state rm .**

Incorrect. terraform state rm removes resources from the state file but does **not show what will be destroyed**. It only tells Terraform to stop managing them.

B. Run terraform destroy. This will output all the resources that will be deleted before prompting for approval.

Correct. terraform destroy shows a plan of all resources that will be destroyed and asks for confirmation before actually deleting them.

C. Run terraform plan -destroy.

Correct. terraform plan -destroy generates a **preview of the destroy operation** without applying it, letting you see which resources would be deleted.

D. Run terraform show -destroy.

Incorrect. terraform show displays the current state or a saved plan but does not generate a destroy plan by itself.

Question: 195 CertyIQ

You created infrastructure outside the Terraform workflow that you now want to manage using Terraform. Which command brings the infrastructure into Terraform state?

A. terraform init
B. terraform get
C. terraform import
D. terraform refresh

Answer: C

Explanation:

A. terraform init

Incorrect. terraform init initializes a working directory, downloads providers, and sets up modules, but does **not import existing resources**.

B. terraform get

Incorrect. terraform get downloads and updates modules, not existing resources.

C. terraform import

Correct. terraform import brings existing infrastructure into Terraform state, allowing Terraform to manage resources created outside of its workflow.

D. terraform refresh

Incorrect. terraform refresh updates the state to match the real-world infrastructure but does **not add**

unmanaged resources to the state.

Question: 196 CertyIQ

A developer launched a VM (virtual machine) outside of the Terraform workflow and ended up with two servers with the same name. They are unsure which VM is managed with Terraform, but they do have a list of all active VM IDs.
Which of the following methods could you use to determine which instance Terraform manages?

A. Run terraform state rm on both VMs, then terraform apply to recreate the correct one that will be managed by Terraform.

B. Run terraform state list to find the names of all VMs, then run terraform state show for each of them to find which VM ID that Terraform manages.

C. Run a terraform apply -refresh to identify the virtual machine IDs that are already managed by Terraform.

D. Modify the Terraform configuration to add an import block for both of the virtual machines.

Answer: B

Explanation:

A. Run terraform state rm on both VMs, then terraform apply to recreate the correct one that will be managed by Terraform.

Incorrect. This would delete Terraform-managed resources and recreate them, which is **destructive** and unnecessary just to identify which VM is managed.

B. Run terraform state list to find the names of all VMs, then run terraform state show for each of them to find which VM ID that Terraform manages.

Correct. terraform state list shows all resources tracked by Terraform. Using terraform state show <resource> lets you inspect each resource's attributes, including the VM ID, to determine which instance Terraform manages.

C. Run a terraform apply -refresh to identify the virtual machine IDs that are already managed by Terraform.

Incorrect. While -refresh updates the state with real-world values, it does not explicitly show which VM is managed, and it could also prompt changes if the state differs from the configuration.

D. Modify the Terraform configuration to add an import block for both of the virtual machines.

Incorrect. There is **no import block** in Terraform configuration. Importing is done with the terraform import CLI command, not by modifying the configuration.

Question: 197 CertyIQ

When do you need to explicitly execute Terraform in refresh-only mode?

A. Before every terraform plan.

B. Before every terraform apply.

C. Before every terraform import.

D. None of the above.

Answer: D

Explanation:

You **do not need to explicitly run Terraform in refresh-only mode** before plan, apply, or import. Terraform automatically refreshes the state as needed during these operations to reflect the real-world infrastructure.

terraform refresh -refresh-only is rarely required manually and is typically only used for **auditing or inspecting the current state** without making any changes.

Question: 198 CertyIQ

Which method for sharing Terraform modules fulfills the following criteria:
1. Keeps the module configurations confidential within your organization.
2. Supports Terraform's semantic version constraints.
3. Provides a browsable directory of your modules.

A. A Git repository containing your modules.
B. A subfolder within your workspace.
C. HCP Terraform/Terraform Cloud private registry.
D. Public Terraform module registry

Answer: C

Explanation:

A. A Git repository containing your modules.

Incorrect. A Git repository can keep modules private, but it does **not support Terraform's semantic versioning** in the same way a registry does, and it does not provide a browsable module directory.

B. A subfolder within your workspace.

Incorrect. A local subfolder keeps modules private but does **not support semantic versioning** or a browsable directory for multiple modules.

C. HCP Terraform/Terraform Cloud private registry.

Correct. A **private registry in HCP Terraform/Terraform Cloud** keeps modules confidential, supports **semantic version constraints**, and provides a **browsable directory** of modules for your organization.

D. Public Terraform module registry.

Incorrect. The public registry supports versioning and browsing but does **not keep modules confidential**, as they are publicly accessible.

Question: 199 CertyIQ

Which of the following is not a way to trigger terraform destroy?

A. Passing --destroy at the end of a plan request.

B. Running terraform destroy from the correct directory and then typing yes when prompted in the CLI.
C. Using the destroy command with auto-approve.

Answer: A

Explanation:

A. Passing --destroy at the end of a plan request.

Correct. Terraform does **not have a --destroy flag** for terraform plan. You cannot trigger a destroy this way.

B. Running terraform destroy from the correct directory and then typing yes when prompted in the CLI.

Incorrect. This is a standard way to trigger terraform destroy.

C. Using the destroy command with auto-approve.

Incorrect. Using terraform destroy -auto-approve is a valid way to trigger destruction without manual confirmation.

Question: 200 CertyIQ

What Terraform command always causes a state file to be updated with changes that might have been made outside of Terraform?

A. terraform apply =lock=false
B. terraform plan -target=state
C. terraform plan -refresh-only
D. terraform show -json

Answer: C

Explanation:

A. terraform apply -lock=false

Incorrect. This flag disables state locking during apply but does **not specifically refresh the state** with external changes.

B. terraform plan -target=state

Incorrect. -target focuses on specific resources but does **not refresh the state** with all external changes.

C. terraform plan -refresh-only

Correct. terraform plan -refresh-only **updates the state file** to reflect any changes made to infrastructure outside of Terraform, without proposing any new resource changes.

D. terraform show -json

Incorrect. terraform show displays the state or a plan in JSON format but **does not modify the state**.

Question: 201 CertyIQ

A developer on your team is going to tear down an existing deployment managed by Terraform and deploy a new one. However, there is a server resource named aws_instance.ubuntu[1] they would like to keep.
What command should they use to tell Terraform to stop managing that specific resource?

A. terraform destroy aws_instance.ubuntu[1]
B. terraform plan rm aws_instance.ubuntu[1]
C. terraform apply rm aws_instance.ubuntu[1]
D. terraform state rm aws_instance.ubuntu[1]

Answer: D

Explanation:

A. terraform destroy aws_instance.ubuntu[1]

Incorrect. Running terraform destroy on a resource will **delete it from the cloud**, which is the opposite of what the developer wants. In this case, they want to **keep** the server intact.

B. terraform plan rm aws_instance.ubuntu[1]

Incorrect. There is no plan rm command in Terraform. terraform plan only previews changes, and rm is not a valid subcommand.

C. terraform apply rm aws_instance.ubuntu[1]

Incorrect. apply rm is not a valid Terraform command. Terraform has no direct rm option within apply.

D. terraform state rm aws_instance.ubuntu[1]

Correct. terraform state rm **removes the resource from Terraform's state file** without deleting the actual infrastructure. This tells Terraform to stop managing that specific resource, allowing the rest of the deployment to be destroyed or recreated while keeping aws_instance.ubuntu[1] intact.

Question: 202 CertyIQ

Which of the following locations can Terraform use as a private source for modules? (Choose two.)

A. Public repository on GitHub.
B. Internally hosted VCS (Version Control System) platform.
C. Public Terraform Registry.
D. Private repository on GitHub.

Answer: B,D

Explanation:

A. Public repository on GitHub

Incorrect. Public GitHub repositories are accessible to anyone, so they are **not private sources**.

B. Internally hosted VCS (Version Control System) platform

Correct. An internally hosted VCS (like GitLab, Bitbucket Server, or an on-prem Git server) can be configured as a **private module source**, restricting access to your organization.

C. Public Terraform Registry

Incorrect. The public Terraform Registry is open to everyone and cannot serve as a private module source.

D. Private repository on GitHub

Correct. A private GitHub repository can store modules that are **only accessible to authorized users**, making it a valid private source.

Question: 203 CertyIQ

You are writing Terraform configuration that calls a child module. This child module declares an output value. How can you ensure that Terraform will print out this value when you run Terraform CLI commands such as terraform apply?

A. Nothing. Terraform will automatically print out all output values from child modules.
B. Nothing. Terraform cannot use output values from child modules.
C. Declare a new output in your root configuration that references the module's output.
D. None of the above.

Answer: C

Explanation:

C. Declare a new output in your root configuration that references the module's output.

Terraform only prints output values that are declared in the **root module**. Outputs from child modules are **not automatically displayed**. To ensure a child module's output is visible when running commands like terraform apply, you need to declare a corresponding output in your root configuration that references the child module's output. This allows Terraform to print the value during execution.

Other options:

A. Nothing. Terraform will automatically print out all output values from child modules.

Incorrect. Child module outputs are not automatically printed.

B. Nothing. Terraform cannot use output values from child modules.

Incorrect. Terraform can use outputs from child modules, but they must be explicitly referenced in the root module to be displayed.

D. None of the above.

Incorrect. Option C is the correct approach.

Question: 204 CertyIQ

You have provisioned some virtual machines (VMs) on Google Cloud Platform (GCP) using the gcloud command line tool. However, you are standardizing with Terraform and want to manage these VMs using Terraform instead. What are the two things you must do to achieve this? (Choose two.)

A. Provision new VMs using Terraform with the same VM names.
B. Use the terraform import command for the existing VMs.
C. Write Terraform configuration for the existing VMs.
D. Run the terraform import-gcp command.

Answer: B,C

Explanation:

A. Provision new VMs using Terraform with the same VM names.

Incorrect. Creating new VMs with the same names would **duplicate resources** and is not required to bring existing infrastructure under Terraform management.

B. Use the terraform import command for the existing VMs.

Correct. terraform import allows Terraform to **associate existing cloud resources with Terraform state**, enabling Terraform to manage them going forward.

C. Write Terraform configuration for the existing VMs.

Correct. Before importing, you must **define the Terraform configuration** that matches the existing resources so Terraform knows how to manage them after import.

D. Run the terraform import-gcp command.

Incorrect. There is no terraform import-gcp command; imports are done using the standard terraform import command.

Question: 205 CertyIQ

Outside of the required_providers block, Terraform configurations always refer to providers by their local names.

A. True
B. False

Answer: A

Explanation:

In Terraform, providers can be given **aliases** or local names in the required_providers block. Outside of that block, you always refer to the provider using the **local name or alias** defined there. This ensures Terraform knows which provider configuration to use for a given resource.

For example, even if the provider source is hashicorp/aws, the local name like aws is what you reference in resources.

Question: 206 CertyIQ

Where does the Terraform local backend store its state?

A. In the terraform.tfstate directory.
B. In the terraform.tfstate file.
C. In the .terraform directory.
D. In the .terraform.lock.hcl file.

Answer: B

Explanation:

A. In the terraform.tfstate directory

Incorrect. There is no terraform.tfstate directory. The local backend stores the state in a **file**, not a directory. Terraform will not create a directory for state unless you explicitly configure it, which is uncommon.

B. In the terraform.tfstate file

Correct. The **local backend** stores the state on your local filesystem in a file called terraform.tfstate by default. This file contains all the information Terraform needs to track your managed resources, including their attributes and metadata.

C. In the .terraform directory

Incorrect. The .terraform directory is used for storing downloaded **providers, modules, and plugin binaries**, not the state file. The state is separate from these artifacts.

D. In the .terraform.lock.hcl file

Incorrect. The .terraform.lock.hcl file is a **provider version lock file**. It ensures consistent provider versions across runs but does **not store the state of resources**.

Question: 207 CertyIQ

A module block is shown in the Exhibit space of this page.

```
module "consul" {
  source = "hashicorp/consul/aws"
}
```

When you use a module block to reference a module from the Terraform Registry such as the one in the example, how do you specify version 1.0.0 of the module?

A. You cannot. Modules stored on the public Terraform Registry do not support versioning.
B. Append ?ref=v1.0.0 argument to the source path.
C. Add a version = "1.0.0" attribute to the module block.
D. Nothing. Modules stored on the public Terraform module Registry always default to version 1.0.0.

Answer: C

Explanation:

A. You cannot. Modules stored on the public Terraform Registry do not support versioning.

Incorrect. Modules on the public Terraform Registry **do support semantic versioning**, and you can specify the version you want to use.

B. Append ?ref=v1.0.0 argument to the source path.

Incorrect. This syntax is used for Git sources, not modules from the Terraform Registry.

C. Add a version = "1.0.0" attribute to the module block.

Correct. When using a module from the Terraform Registry, you specify the desired version with the version attribute in the module block. This ensures Terraform uses that specific version and respects semantic version constraints.

D. Nothing. Modules stored on the public Terraform module Registry always default to version 1.0.0.

Incorrect. The registry defaults to the **latest available version** that matches any constraints; it does not automatically default to 1.0.0.

Question: 208 CertyIQ

You cannot install third party plugins using terraform init.

A. True
B. False

Answer: B

Explanation:

B. False

You **can install third-party plugins** using terraform init. The init command downloads and installs all required provider plugins and modules specified in your Terraform configuration, including those from third-party sources. This allows Terraform to interact with a wide variety of cloud providers and services beyond the built-in ones.

A. True

Incorrect. terraform init is explicitly designed to fetch and install both first-party and third-party plugins.

Question: 209 CertyIQ

You're writing a Terraform configuration that needs to read input from a local file called id_rsa.pub.
Which built-in Terraform function can you use to import the file's contents as a string?

A. file("id_rsa.pub")
B. templatefile("id_rsa.pub")
C. fileset("id_rsa.pub")
D. filebase64("id_rsa.pub")

Answer: A

Explanation:

A. file("id_rsa.pub")

The file() function in Terraform **reads the contents of a local file** and returns it as a string. This is useful for reading things like SSH public keys, configuration files, or scripts directly into your Terraform configuration.

Other options:

B. templatefile("id_rsa.pub")

Incorrect. templatefile() is used to **render a template file with variables**, not simply read a file as a string.

C. fileset("id_rsa.pub")

Incorrect. fileset() is used to return a **set of file names in a directory matching a pattern**, not the file's contents.

D. filebase64("id_rsa.pub")

Incorrect. filebase64() reads the file and returns its contents **encoded in Base64**, not as a plain string.

Question: 210 CertyIQ

When you initialize Terraform, where does it cache modules from the public Terraform Registry?

A. They are not cached.
B. In the /tmp directory.
C. In the .terraform sub-directory.
D. In memory.

Answer: C

Explanation:

A. They are not cached.

Incorrect. Terraform **does cache modules** downloaded from the public registry. Without caching, Terraform would have to download modules every time you run init or apply a configuration, which would be inefficient and error-prone.

B. In the /tmp directory.

Incorrect. Terraform does **not** use temporary system directories like /tmp for storing modules. /tmp is typically cleared by the system and would not provide reliable storage for module caching.

C. In the .terraform sub-directory

Correct. Terraform stores downloaded modules in the .terraform sub-directory of your working directory. This allows Terraform to **reuse modules across runs**, avoid repeated downloads, and support offline operations.

D. In memory.

Incorrect. While Terraform may load module contents into memory temporarily during execution, the **persistent cache is on disk** in the .terraform directory. Memory alone would not allow reuse across multiple Terraform runs.

Question: 211 CertyIQ

Which Terraform collection type should you use to store key/value pairs?

A. map
B. tuple
C. list
D. set

Answer: A

Explanation:

A. map

Correct. A map in Terraform is explicitly designed to store key/value pairs, allowing you to reference values by their keys. Example: "name" = "gourav", "role" = "developer" .

B. tuple

Incorrect. A tuple is an ordered sequence of elements and does not store key/value pairs. Access is by index, not by key.

C. list

Incorrect. A list is an ordered collection of values, also accessed by index, not by key. It cannot directly represent key/value pairs.

D. set

Incorrect. A set is an unordered collection of unique values. It does not support key/value mapping.

Question: 212 CertyIQ

A Terraform backend determines how Terraform loads state and stores updates when you execute which command?

A. apply
B. destroy
C. Bot h of these are correct.
D. Neither of these are correct.

Answer: C

Explanation:

A. apply

Incorrect. While terraform apply does update state, the backend also handles other operations beyond just apply.

B. destroy

Incorrect. terraform destroy also updates state, but the backend is not limited to destroy operations alone.

C. Both of these are correct

Correct. A Terraform backend manages state for **all commands that read or write state**, including apply, destroy, plan, and more. It ensures Terraform knows the current state of infrastructure and can store updates reliably.

D. Neither of these are correct

Incorrect. The backend is directly responsible for storing and retrieving state, so this option is not correct.

Question: 213 CertyIQ

It is best practice to store secret data in the same version control repository as your Terraform configuration.

A. True
B. False

Answer: B

Explanation:

A. True

Incorrect. Storing secrets directly in your Terraform configuration or version control can expose sensitive data, risking security breaches.

B. False

Correct. Best practice is to **store secrets separately** using secure methods such as **Terraform Cloud/Enterprise workspaces, environment variables, or secret management tools** like Vault or AWS Secrets Manager, rather than embedding them in version contro

Question: 214 CertyIQ

Which of these is true about Terraform's plugin-based architecture?

A. You can create a provider for your API if none exists.
B. Terraform can only source providers from the internet.
C. All providers are part of the Terraform core binary.
D. Every provider in a configuration has its own state file for its resources.

Answer: A

Explanation:

A. You can create a provider for your API if none exists

Correct. Terraform's plugin-based architecture allows users to **develop custom providers** to interact with APIs or services that don't have an existing provider.

B. Terraform can only source providers from the internet

Incorrect. Terraform can source providers **from local paths, private registries, or the public registry**. It is not limited to the internet.

C. All providers are part of the Terraform core binary

Incorrect. Providers are **separate binaries** that Terraform communicates with via the plugin system; they are not included in the core Terraform binary.

D. Every provider in a configuration has its own state file for its resources

Incorrect. Terraform stores **all resource state in a single state file** (or backend-managed state), not separate files per provider.

Question: 215 CertyIQ

You are using a networking module in your Terraform configuration with the name my_network. Your root module includes the following configuration:

```
output "net_id" {
  value = module.my_network.vnet_id
}
```

When you run terraform validate, you get the following error:

```
Error: Reference to undeclared output value

  on main.tf line 12, in output "net_id":
  12:   value = module.my_network.vnet_id
```

What must you do to successfully retrieve this value from your networking module?

A. Define the attribute vnet_id as a variable in the networking module.
B. Change the referenced value to my_network.outputs.vnet_id.
C. Change the referenced value to module.my_network.outputs.vnet_id.
D. Define the attribute vnet_id as an output in the networking module.

Answer: D

Explanation:

A. Define the attribute vnet_id as a variable in the networking module

Incorrect. Variables are used to pass **inputs into a module**, not to retrieve values from it. This will not allow you to access vnet_id from the root module.

B. Change the referenced value to my_network.outputs.vnet_id

Incorrect. Terraform requires the module keyword to access module outputs. Simply prefixing with the module name is invalid syntax.

C. Change the referenced value to module.my_network.outputs.vnet_id

Incorrect. Terraform allows you to access outputs using module.<module_name>.<output_name>, **without**

.outputs. Including .outputs will cause a syntax error.

D. Define the attribute vnet_id as an output in the networking module

Correct. To retrieve a value from a module, the module must **declare it as an output**. After adding:

output "vnet_id"

value = azurerm_virtual_network.my_vnet.id

You can access it in the root module using:

module.my_network.vnet_id

This is the proper way to expose module values.

Question: 216 CertyIQ

Where in your Terraform configuration do you specify a state backend?

A. The data source block
B. The terraform block
C. The provider block
D. The resource block

Answer: B

Explanation:

A. The data source block

Incorrect. Data sources are used to read information from existing resources, not to configure where Terraform stores state.

B. The terraform block

Correct. The terraform block is where you specify backend settings, which determine **how and where Terraform stores its state**.

C. The provider block

Incorrect. Provider blocks configure cloud or service providers, not state storage.

D. The resource block

Incorrect. Resource blocks define infrastructure resources, not the backend for state.

Question: 217 CertyIQ

You can execute terraform fmt to standardize all Terraform configurations within the current working directory to Terraform's canonical format and style.

A. True
B. False

Answer: A

Explanation:

A. True

Correct. The terraform fmt command **automatically formats Terraform configuration files** in the current directory (and optionally subdirectories) to follow Terraform's standard style and formatting rules.

B. False

Incorrect. terraform fmt is specifically designed to enforce consistent formatting, so this statement is true.

Question: 218 CertyIQ

The Terraform binary version and provider versions must match each other in a single configuration.

A. True
B. False

Answer: B

Explanation:

A. True

Incorrect. The Terraform binary and provider versions **do not need to match exactly**. Providers can have versions that are compatible with your Terraform version, and you can specify version constraints to ensure compatibility.

B. False

Correct. Terraform allows you to use different provider versions as long as they are **compatible** with the Terraform binary version you are using. Exact matching is not required.

Question: 219 CertyIQ

Multiple team members are collaborating on infrastructure using Terraform and want to format their Terraform code following standard Terraform style convention.
How should they ensure the code satisfies conventions?

A. Use terraform fmt.
B. Terraform automatically formats configuration on terraform apply.
C. Run terraform validate prior to executing terraform plan or terraform apply.
D. Replace all tabs with spaces.

Answer: A

Explanation:

A. Use terraform fmt

Correct. The terraform fmt command **automatically formats Terraform code** according to the standard style convention, ensuring consistency across team members.

B. Terraform automatically formats configuration on terraform apply

Incorrect. Terraform **does not automatically format code** during apply; formatting must be done explicitly with terraform fmt.

C. Run terraform validate prior to executing terraform plan or terraform apply

Incorrect. terraform validate checks for **syntax and configuration errors**, but it does not enforce style or formatting conventions.

D. Replace all tabs with spaces

Incorrect. Manually replacing tabs with spaces is unnecessary if you use terraform fmt, which handles formatting automatically.

Question: 220 CertyIQ

Terraform can only manage resource dependencies if you set them explicitly with the depends_on argument.

A. True
B. False

Answer: B

Explanation:

A. True

Incorrect. Terraform **automatically infers dependencies** between resources based on references in your configuration. Explicit depends_on is only needed in special cases where implicit dependencies are insufficient.

B. False

Correct. Terraform's **graph-based dependency management** tracks which resources depend on others automatically, so you don't always need to use depends_on.

Question: 221 CertyIQ

You decide to move a Terraform state file to Amazon S3 from another location. You write the code shown in the Exhibit space into a file called backend.tf.

```
terraform {
  backend "s3" {
    bucket = "my-tf-bucket"
    region = "us-east-1"
  }
}
```

Which command will migrate your current state file to the new S3 backend?

A. terraform refresh
B. terraform init
C. terraform push
D. terraform state

Answer: B

Explanation:

A. terraform refresh

Incorrect. terraform refresh updates the state file to match the real-world resources, but it **does not migrate the state to a new backend**.

B. terraform init

Correct. Running terraform init after configuring a new backend **initializes the backend** and will **migrate your existing state file** to the S3 backend if configured.

C. terraform push

Incorrect. terraform push is **deprecated** and was used only for legacy Terraform Enterprise workflows; it is not used for backend state migration.

D. terraform state

Incorrect. terraform state is used for **state management commands** like mv, rm, or list, but it does not perform backend migration.

Question: 222 CertyIQ

Terraform encrypts sensitive values stored in your state file.

A. True
B. False

Answer: B

Explanation:

A. True

Incorrect. By default, Terraform **does not encrypt sensitive values** in the local state file. Sensitive data like passwords or secrets are stored in plaintext unless the backend itself provides encryption (e.g., S3 with server-side encryption).

B. False

Correct. Terraform relies on **backend security** or external measures to protect sensitive values; the state file itself is **not automatically encrypted** by Terraform.

Question: 223 CertyIQ

You have declared a variable called var.list which is a list of objects that all have an attribute id . Which options will produce a list of the IDs? (Choose two.)

A. [for o in var.list : o.id]
B. var.list[*].id
C. [var.list[*].id]
D. for o in var.list : o => o.id

Answer: A,B

Explanation:

A. [for o in var.list : o.id]

Correct. This is a **for-expression** that iterates over each object in the list and collects the id attribute, producing a list of IDs.

B. var.list[*].id

Correct. The **splat operator (*)** extracts the id attribute from each object in the list, returning a list of IDs.

C. [var.list[*].id]

Incorrect. Wrapping var.list[*].id in square brackets produces a **nested list** (a list containing one list), which is not the same as a flat list of IDs.

D. for o in var.list : o => o.id

Incorrect. This creates a **map** from the object to its id, not a list of IDs. It does not satisfy the requirement to produce a list.

Question: 224 CertyIQ

You need to deploy resources into two different regions in the same Terraform configuration. To do this, you declare multiple provider configurations as shown in the Exhibit space on this page.

```
provider "aws" {
  region = "us-east-1"
}

provider "aws" {
  alias  = "west"
  region = "us-west-2"
}
```

What meta-argument do you need to configure in a resource block to deploy the resource to the us-west-2 AWS region?

A. provider = west
B. alias = west
C. provider = aws.west
D. alias = aws.west

Answer: C

Explanation:

A. provider = west

Incorrect. You cannot reference the provider just by its alias name alone; Terraform requires the full provider address.

B. alias = west

Incorrect. alias is used **when declaring a provider configuration**, not when assigning a resource to a provider.

C. provider = aws.west

Correct. To use a specific provider configuration with an alias, you set the **provider meta-argument** in the resource block to the **full provider address including the alias**. For example, provider = aws.west tells Terraform to use the aws provider configuration aliased as west.

D. alias = aws.west

Incorrect. alias is not valid inside a resource block; it is only used in provider configuration blocks.

Question: 225 CertyIQ

terraform init retrieves and caches the configuration for all remote modules

A. True
B. False

Answer: A

Explanation:

A. True

Correct. Running terraform init **downloads and caches all remote modules** referenced in your configuration so they can be reused across runs, even offline.

B. False

Incorrect. Terraform does cache remote modules in the .terraform directory, so this statement is true.

Question: 226 CertyIQ

You are a member of an operations team that uses infrastructure as code (IaC) practices with HCP Terraform/Terraform Cloud. You are tasked with making a change to an infrastructure stack running in a public cloud.Which pattern would follow IaC best practices for making a change?

A. Make the change programmatically via the public cloud CLI.
B. Clone the repository containing your infrastructure code and then run the code.
C. Submit a pull request and wait for an approved merge of the proposed changes.
D. Make the change via the public cloud API endpoint.
E. Use the public cloud console to make the change after a database record has been approved.

Answer: C

Explanation:

A. Make the change programmatically via the public cloud CLI

Incorrect. Making changes directly in the cloud bypasses your Terraform configuration, breaking the declarative IaC model and causing **configuration drift**.

B. Clone the repository containing your infrastructure code and then run the code

Incorrect. While cloning is part of working with the code, running it without a review or proper process does not follow **team-based IaC best practices**.

C. Submit a pull request and wait for an approved merge of the proposed changes

Correct. The **pull request workflow** ensures that all changes to infrastructure code are **reviewed, version-controlled, and approved** before being applied. This maintains auditability, collaboration, and the integrity of your infrastructure.

D. Make the change via the public cloud API endpoint

Incorrect. Like the CLI, making direct API changes bypasses Terraform, creating potential **drift** between code and actual infrastructure.

E. Use the public cloud console to make the change after a database record has been approved

Incorrect. Making manual console changes is not aligned with IaC principles; all changes should be made **through version-controlled Terraform configurations**.

Question: 227 CertyIQ

terraform validate uses provider APIs to verify your infrastructure settings.

A. True
B. False

Answer: B

Explanation:

A. True

Incorrect. terraform validate **does not contact any provider APIs**. It only checks the **syntax and internal consistency** of your Terraform configuration.

B. False

Correct. terraform validate performs a **static check** of your configuration files and cannot verify actual infrastructure settings with providers.

Question: 228 CertyIQ

It is _____ to change the Terraform backend from the default “local” backend to a different one after performing your first terraform apply.

A. impossible
B. discouraged
C. optional
D. mandatory

Answer: B

Explanation:

A. impossible

Incorrect. It is not impossible; Terraform allows backend changes, but you must take care to migrate the existing state.

B. discouraged

Correct. Changing the backend **after initial deployment is discouraged** because it can be risky and may require careful state migration to avoid losing track of resources.

C. optional

Incorrect. While technically optional, this doesn’t capture the caution needed—changing the backend is not a routine operation.

D. mandatory

Incorrect. Changing the backend is **not required**; it’s only done if you need a remote or shared state backend.

Question: 229 CertyIQ

Terraform installs its providers during which phase?

A. plan
B. init
C. refresh
D. All of the above

Answer: B

Explanation:

A. plan

Incorrect. The terraform plan phase **uses already installed providers** to generate an execution plan; it does not install them.

B. init

Correct. Terraform installs providers **during the terraform init phase**, when it initializes a configuration and downloads any required provider plugins.

C. refresh

Incorrect. terraform refresh updates the state to match real-world resources, but it does **not install providers**.

D. All of the above

Incorrect. Only the init phase is responsible for installing providers.

Question: 230 CertyIQ

Which of the following is not true of Terraform providers?

A. An individual person can write a Terraform Provider.
B. A community of users can maintain a provider.
C. HashiCorp maintains some providers.
D. Cloud providers and infrastructure vendors can write, maintain, or collaborate on Terraform providers.
E. None of the above.

Answer: E

Explanation:

A. An individual person can write a Terraform Provider

True. Terraform's plugin-based architecture allows an individual to develop a custom provider.

B. A community of users can maintain a provider

True. Providers can be community-maintained and published in public registries.

C. HashiCorp maintains some providers

True. HashiCorp officially maintains core providers like AWS, Azure, and Google Cloud.

D. Cloud providers and infrastructure vendors can write, maintain, or collaborate on Terraform providers

True. Many cloud providers and vendors develop and maintain their own official providers.

E. None of the above

Correct. All the statements A–D are **true**, so the statement "not true" applies to **none of them**.

Question: 231 CertyIQ

HashiCorp Configuration Language (HCL) supports user-defined functions.

A. True
B. False

Answer: B

Explanation:

A. True

Incorrect. HCL provides **built-in functions** (like length(), concat(), lookup(), etc.), but it **does not allow users to define their own functions**.

B. False

Correct. HCL **does not support user-defined functions**; you can only use the built-in set of functions provided by Terraform.

Question: 232 CertyIQ

Which of these statements about HCP Terraform/Terraform Cloud workspaces is false?

A. You must use the CLI to switch between workspaces.
B. Plans and applies can be triggered via version control system integrations.
C. They have role-based access controls.
D. They can securely store cloud credentials.

Answer: A

Explanation:

A. You must use the CLI to switch between workspaces

Correct. This is **false**. While you can switch workspaces via the CLI, you can also switch workspaces through the **Terraform Cloud web UI**. It is not CLI-only.

B. Plans and applies can be triggered via version control system integrations

True. Workspaces can automatically run plans and applies when changes are pushed to connected VCS repositories.

C. They have role-based access controls

True. Workspaces support role-based access control (RBAC) to manage permissions for team members.

D. They can securely store cloud credentials

True. Workspaces can securely store sensitive credentials and variables using Terraform Cloud's secrets management.

Question: 233

CertyIQ

Which of the following is true about terraform apply? (Choose two.)

A. You cannot target specific resources for the operation.

B. Depending on provider specification, Terraform may need to destroy and recreate your infrastructure resources.

C. It only operates on infrastructure defined in the current working directory or workspace.

D. By default, it does not refresh your state file to reflect the current infrastructure configuration.

E. You must pass the output of a terraform plan command to it.

Answer: B,C

Explanation:

A. You cannot target specific resources for the operation

Incorrect. You can target specific resources using the -target option with terraform apply.

B. Depending on provider specification, Terraform may need to destroy and recreate your infrastructure resources

Correct. Some changes require **resource replacement**, which involves destroying the existing resource and creating a new one, depending on the provider and resource type.

C. It only operates on infrastructure defined in the current working directory or workspace

Correct. terraform apply works on the configuration **loaded in the current directory or workspace**, applying changes to the infrastructure defined there.

D. By default, it does not refresh your state file to reflect the current infrastructure configuration

Incorrect. By default, terraform apply **refreshes the state** to reflect the current real-world infrastructure before applying changes.

E. You must pass the output of a terraform plan command to it

Incorrect. You can run terraform apply **directly**, and it will automatically create a plan before applying changes; passing a plan output is optional.

Question: 234 CertyIQ

Which of these are benefits of using Sentinel with HCP Terraform/Terraform Cloud? (Choose three.)

A. You can check out and check in cloud access keys.
B. You can restrict specific resource configurations, such as disallowing the use of CIDR=0.0.0.0/0.
C. Sentinel Policies can be written in HashiCorp Configuration Language (HCL).
D. You can enforce a list of approved AWS AMIs.
E. Policy-as-code can enforce security best practices.

Answer: B,D,E

Explanation:

A. You can check out and check in cloud access keys

Incorrect. Sentinel does **not manage credentials**; it is a policy-as-code framework for governance.

B. You can restrict specific resource configurations, such as disallowing the use of CIDR=0.0.0.0/0

Correct. Sentinel can enforce **fine-grained rules on resource configurations** to prevent insecure settings.

C. Sentinel Policies can be written in HashiCorp Configuration Language (HCL)

Incorrect. Sentinel uses its **own policy language**, not HCL.

D. You can enforce a list of approved AWS AMIs

Correct. Sentinel policies can enforce **approved values for specific attributes**, such as a whitelist of AMIs.

E. Policy-as-code can enforce security best practices

Correct. Sentinel enables **policy-as-code**, allowing automated enforcement of security, compliance, and operational best practices.

Question: 235 CertyIQ

Which of the following is available only in HCP Terraform workspaces and not in Terraform CLI?

A. Secure variable storage.
B. Support for multiple cloud providers.
C. Dry runs with terraform plan.
D. Using one workspace's state as a data source for another.

Answer: D

Explanation:

A. Secure variable storage

Incorrect. While HCP workspaces offer secure variable storage, you can also use local environment variables or encrypted files with Terraform CLI, though less integrated.

B. Support for multiple cloud providers

Incorrect. Terraform CLI also supports multiple providers; this is not unique to HCP workspaces.

C. Dry runs with terraform plan

Incorrect. terraform plan is available in both CLI and HCP workspaces.

D. Using one workspace's state as a data source for another

Correct. This is a **feature specific to HCP Terraform/Terraform Cloud**, where you can reference the state of one workspace as a data source in another workspace. This is **not available in standalone Terraform CLI**.

Question: 236 CertyIQ

Which two steps are required to provision new infrastructure in the Terraform workflow? (Choose two.)

A. import
B. init
C. apply
D. plan
E. validate

Answer: B,C

Explanation:

A. import

Incorrect. terraform import is used to bring **existing resources under Terraform management**, not for provisioning new infrastructure.

B. init

Correct. terraform init **initializes the working directory**, downloads required providers and modules, and is always required before provisioning.

C. apply

Correct. terraform apply **creates or updates infrastructure** according to your configuration and is the step that actually provisions resources.

D. plan

Incorrect. terraform plan is optional; it generates an execution plan to preview changes but does not provision infrastructure by itself.

E. validate

Incorrect. terraform validate only checks **syntax and configuration correctness**; it does not provision resources.

Question: 237 CertyIQ

Which of the following should you add in the required_providers block to define a provider version constraint?

A. version = ">= 3.1"
B. version >= 3.1
C. version ~> 3.1

Answer: A

Explanation:

A. version = ">= 3.1"

Correct. Proper syntax for specifying a version constraint.

B. version >= 3.1

Incorrect. Missing the = and quotes; Terraform will throw a syntax error.

C. version ~> 3.1

Incorrect. The ~> operator **must also be inside quotes and assigned with =**. Correct syntax would be version = "~> 3.1". Without = and quotes, it is invalid.

Question: 238

What is the purpose of state in Terraform?

A. State codifies the dependencies of related resources.
B. State stores variables and lets you quickly reuse existing code.
C. State lets you enforce resource configurations that relate to compliance policies.
D. State maps real world resources to your configuration and keeps track of metadata.

Answer: D

Explanation:

A. State codifies the dependencies of related resources

Incorrect. Terraform infers **dependencies automatically** from resource references, but that is not the primary purpose of the state file.

B. State stores variables and lets you quickly reuse existing code

Incorrect. Variables are defined in configuration, not in state, and reusing code is handled by modules, not the state file.

C. State lets you enforce resource configurations that relate to compliance policies

Incorrect. Compliance enforcement is handled by **Sentinel or policy-as-code**, not the state file.

D. State maps real world resources to your configuration and keeps track of metadata

Correct. Terraform's **state file records the mapping between resources in your configuration and the real infrastructure**, along with metadata like resource IDs, dependencies, and provider information. This allows Terraform to track changes and plan updates accurately.

Question: 239 CertyIQ

Terraform providers are always installed from the Internet.

A. True
B. False

Answer: B

Explanation:

A. True

Incorrect. Terraform can install providers from **local paths, private registries, or the public registry**, not just the Internet.

B. False

Correct. Providers do **not have to come from the Internet**; you can use **local binaries or private registries** for installation.

Question: 240 CertyIQ

Your DevOps team is currently using the local backend for your Terraform configuration. You would like to move to a remote backend to store the state file in a central location.
Which of the following backends would not work?

A. HCP Terraform/Terraform Cloud
B. Git
C. Amazon S3

Answer: B

Explanation:

A. HCP Terraform/Terraform Cloud

Incorrect. HCP Terraform/Terraform Cloud is a **supported remote backend** and can store state centrally.

B. Git

Correct. Git is **not a supported Terraform backend**. Terraform state requires a backend that supports **locking and atomic operations**, which Git alone cannot provide.

C. Amazon S3

Incorrect. S3 is a supported remote backend, often used in combination with **DynamoDB for state locking**.

Question: 241 CertyIQ

Which parameters does terraform import require? (Choose two.)

A. Provider
B. Path
C. Resource ID
D. Resource address

Answer: C,D

Explanation:

A. Provider

Incorrect. You do **not need to specify the provider** explicitly when running terraform import; Terraform uses the provider configured in your code.

B. Path

Incorrect. There is no "path" parameter for terraform import.

C. Resource ID

Correct. Terraform requires the **ID of the existing resource** in the provider's system to import it into state.

D. Resource address

Correct. Terraform requires the **resource address** from your configuration (e.g., aws_instance.example) to know **where to map the imported resource in state**.

Question: 242 CertyIQ

Where can Terraform not load a provider from?

A. Provider plugin cache.
B. Source code.
C. Plugins directory.
D. Official HashiCorp distribution on releases.hashicorp.com.

Answer: B

Explanation:

A. Provider plugin cache

Incorrect. Terraform can load providers from its **plugin cache** (usually .terraform/providers).

B. Source code

Correct. Terraform **cannot load a provider directly from source code**; the provider must be built into a binary before Terraform can use it.

C. Plugins directory

Incorrect. Terraform can load providers from a **local plugins directory** if configured.

D. Official HashiCorp distribution on releases.hashicorp.com

Incorrect. Terraform can download providers directly from the **official HashiCorp releases**.

Question: 243 CertyIQ

Which of the following arguments are required when declaring a Terraform output?

A. sensitive
B. value
C. default
D. description

Answer: B

Explanation:

A. sensitive

Incorrect. The sensitive argument is optional. It hides the output value in the CLI and logs but is not mandatory.

B. value

Correct. The value argument is required because it defines what the output will return. Without it, the output cannot function.

C. default

Incorrect. The default argument is used for variables, not outputs, so it cannot be applied here.

D. description

Incorrect. The description argument is optional and provides a human-readable explanation of the output, but it is not required.

Question: 244 CertyIQ

Which configuration consistency error does terraform validate report?

A. Differences between local and remote state.
B. A mix of spaces and tabs in configuration files.
C. Terraform module isn't the latest version.
D. Declaring a resource identifier more than once.

Answer: D

Explanation:

A. Differences between local and remote state

Incorrect. Detecting differences between real infrastructure and state is handled during plan or apply, not by validate.

B. A mix of spaces and tabs in configuration files

Incorrect. Formatting issues are handled by terraform fmt, not terraform validate.

C. Terraform module isn't the latest version

Incorrect. Terraform does not require modules to be the latest version, and validate does not check for version updates.

D. Declaring a resource identifier more than once

Correct. terraform validate checks for syntax and internal consistency errors, including duplicate resource declarations within the configuration.

Question: 245 CertyIQ

Once you configure a new Terraform backend with a terraform code block, which command(s) should you use to migrate the state file?

A. terraform init
B. terraform push
C. terraform destroy, then terraform apply
D. terraform apply

Answer: A

Explanation:

A. terraform init

Correct. After changing or adding a backend configuration, you run terraform init. Terraform will detect the backend change and prompt you to migrate the existing state to the new backend.

B. terraform push

Incorrect. terraform push is deprecated and not used for backend state migration.

C. terraform destroy, then terraform apply

Incorrect. Destroying and recreating infrastructure is unnecessary and risky. Backend migration does not require resource destruction.

D. terraform apply

Incorrect. terraform apply manages infrastructure changes, not backend migration.

Question: 246 CertyIQ

You want to know from which paths Terraform is loading providers referenced in your Terraform configuration (*.tf files). You need to enable additional logging messages to find this out.
Which of the following would achieve this?

A. Set verbose logging for each provider in your Terraform configuration.
B. Set the environment variable TF_LOG_PATH.

C. Set the environment variable TF_LOG=TRACE.

D. Set the environment variable TF_VAR_log=TRACE.

Answer: C

Explanation:

A. Set verbose logging for each provider in your Terraform configuration

Incorrect. Terraform does not support enabling verbose logging per provider within the configuration files.

B. Set the environment variable TF_LOG_PATH

Incorrect. TF_LOG_PATH only specifies where to write the log file. It does not enable detailed logging by itself.

C. Set the environment variable TF_LOG=TRACE

Correct. Setting TF_LOG=TRACE enables the most detailed logging level in Terraform, which includes information about provider installation and loading paths.

D. Set the environment variable TF_VAR_log=TRACE

Incorrect. TF_VAR_ variables are used to pass input variables to Terraform, not to control logging behavior.

Question: 247 CertyIQ

In a HCP Terraform/Terraform Cloud workspace linked to a version control repository, speculative plan runs start automatically when you merge or commit changes to version control.

A. True

B. False

Answer: B

Explanation:

A. True

Incorrect. When you merge or commit changes to the main branch, Terraform Cloud typically runs a normal plan (and possibly apply, depending on settings). Speculative plans are triggered for pull requests before merging.

B. False

Correct. Speculative plans run automatically for pull requests, not for merges or direct commits to the main branch.

Question: 248 CertyIQ

When you run terraform apply, the Terraform CLI will print output values from both the root module and any child modules.

A. True

B. False

Answer: B

Explanation:

A. True

Incorrect. Terraform only displays **outputs defined in the root module** after terraform apply. Outputs from child modules are not printed unless they are explicitly exposed through the root module.

B. False

Correct. The CLI prints only **root module outputs**. To see child module outputs, you must define corresponding outputs in the root module that reference them.

Question: 249 CertyIQ

Your risk management organization requires that new AWS S3 buckets must be private, and their contents must be encrypted at rest. How can HCP Terraform/Terraform Cloud automatically and proactively enforce this security control?

A. By adding variables to each HCP Terraform/Terraform Cloud workspace to ensure these settings are always enabled.

B. With an S3 module with proper settings for buckets.

C. Auditing cloud storage buckets with a vulnerability scanning tool.

D. With a Sentinel policy, which runs before every apply.

Answer: D

Explanation:

A. By adding variables to each HCP Terraform/Terraform Cloud workspace to ensure these settings are always enabled

Incorrect. Variables can help standardize configuration, but they do not guarantee enforcement. Users could still override or misconfigure resources.

B. With an S3 module with proper settings for buckets

Incorrect. While a module encourages best practices, it does not prevent someone from creating an S3 bucket resource outside the module with insecure settings.

C. Auditing cloud storage buckets with a vulnerability scanning tool

Incorrect. Auditing is reactive and occurs after deployment. The requirement is to enforce controls proactively before infrastructure is created.

D. With a Sentinel policy, which runs before every apply

Correct. Sentinel policies enforce policy-as-code rules during the Terraform workflow and can prevent applies that violate requirements, such as creating public or unencrypted S3 buckets.

Question: 250 CertyIQ

If you don't use the local Terraform backend, where else can Terraform save resource state?

A. In an environment variable.
B. In memory.
C. In a remote location configured in the .terraformrc file, such as HCP Terraform or a cloud storage system.
D. In a remote location configured in the terraform block, such as HCP Terraform or a cloud storage system.

Answer: D

Explanation:

A. In an environment variable

Incorrect. Environment variables are used to pass configuration values, not to store Terraform state.

B. In memory

Incorrect. Terraform state must persist between runs. Storing it only in memory would not allow future operations.

C. In a remote location configured in the .terraformrc file, such as HCP Terraform or a cloud storage system

Incorrect. The .terraformrc file configures CLI behavior and credentials, not backend state storage.

D. In a remote location configured in the terraform block, such as HCP Terraform or a cloud storage system

Correct. Remote backends are configured in the terraform block and allow Terraform to store state in centralized locations like HCP Terraform, Amazon S3, Azure Storage, or similar systems.

Question: 251 CertyIQ

Which of the following statements about Terraform modules is not true?

A. Modules must be publicly accessible.
B. You can call the same module multiple times.
C. Modules can call other modules.
D. A module is a container for one or more resources.

Answer: A

Explanation:

A. Modules must be publicly accessible

This statement is not true. Terraform modules do not have to be publicly accessible. They can exist locally on your filesystem, inside a private Git repository, in a private module registry, or in Terraform Cloud's private registry. Many organizations use private modules to protect internal infrastructure logic and standards. Therefore, public accessibility is not a requirement for modules.

B. You can call the same module multiple times

This statement is true. Terraform allows you to reuse a module multiple times within the same configuration. Each module call can use different input variables, making modules highly reusable and flexible for deploying similar infrastructure in different environments or regions.

C. Modules can call other modules

This statement is true. Terraform supports nested modules, meaning one module can reference and use another module. This enables composition and better organization of complex infrastructure.

D. A module is a container for one or more resources

This statement is true. A module groups together related resources into a single logical unit. Even the root configuration directory is technically a module (the root module).

Question: 252 CertyIQ

How does Terraform determine dependencies between resources when it creates an execution plan?

A. Terraform builds a resource graph based on your configuration and your state file (if present).
B. Terraform builds a resource graph based on your configuration and your state file (if present).
C. Terraform requires resource dependencies to be defined as modules and sourced in order.
D. Terraform requires all dependencies between resources to be specified using the depends_on parameter.

Answer: A

Explanation:

A. Terraform builds a resource graph based on your configuration and your state file (if present).

Correct. Terraform automatically analyzes references between resources in your configuration and builds a dependency graph. It also considers the current state (if it exists) to understand relationships and determine the correct order of operations.

B. Terraform requires resources in your configuration to be listed in the order they will be created to determine dependencies.

Incorrect. Terraform does not rely on the order of resources in configuration files. The dependency graph determines execution order.

C. Terraform requires resource dependencies to be defined as modules and sourced in order.

Incorrect. Modules help organize infrastructure, but they are not required for dependency management.

D. Terraform requires all dependencies between resources to be specified using the depends_on parameter.

Incorrect. Terraform automatically infers most dependencies through references. The depends_on argument is only needed for explicit dependencies that cannot be inferred.

Question: 253 CertyIQ

Which of the following can you do with terraform plan? (Choose two.)

A. Schedule Terraform to run at a planned time in the future.
B. View the execution plan and check if the changes match your expectations.
C. Execute a plan in a different workspace.
D. Save a generated execution plan to apply later.

Answer: B,D

Explanation:

A. Schedule Terraform to run at a planned time in the future

Incorrect. terraform plan does not schedule future runs. Scheduling must be handled externally through CI/CD pipelines or automation tools.

B. View the execution plan and check if the changes match your expectations

Correct. terraform plan shows the proposed changes before they are applied, allowing you to review and verify them.

C. Execute a plan in a different workspace

Incorrect. terraform plan runs in the currently selected workspace. You must switch workspaces before running it.

D. Save a generated execution plan to apply later

Correct. You can save the execution plan to a file and later use it with terraform apply to ensure exactly that plan is executed.

Question: 254

Which statement describes a goal of infrastructure as code (IaC)?

A. The programmatic configuration of resources.
B. Defining a vendor-agnostic API.
C. Write once, run anywhere.
D. A pipeline process to test and deliver software.

Answer: A

Explanation:

A. The programmatic configuration of resources

Correct. A primary goal of IaC is to define and manage infrastructure through machine-readable configuration files rather than manual processes. This enables automation, version control, repeatability, and consistency.

B. Defining a vendor-agnostic API

Incorrect. While some IaC tools aim to be cloud-agnostic, defining a vendor-agnostic API is not the core goal of IaC itself.

C. Write once, run anywhere

Incorrect. This phrase is commonly associated with platform portability concepts, not specifically with IaC principles.

D. A pipeline process to test and deliver software

Incorrect. That describes CI/CD practices, not the primary goal of infrastructure as code.

Question: 255 CertyIQ

Which backend does the Terraform CLI use by default?

A. Remote.
B. Local.
C. HCP Terraform/Terraform Cloud.
D. HTTP.
E. Depends on the cloud provider.

Answer: B

Explanation:

A. Remote

Incorrect. Terraform does not use a remote backend unless you explicitly configure one.

B. Local

Correct. By default, Terraform uses the **local backend**, which stores the state file on your local machine in the working directory.

C. HCP Terraform/Terraform Cloud

Incorrect. This backend must be explicitly configured; it is not the default.

D. HTTP

Incorrect. The HTTP backend must be explicitly configured and is not used by default.

E. Depends on the cloud provider

Incorrect. The backend is independent of the cloud provider and does not change automatically.

Question: 256 CertyIQ

You must initialize your working directory before running terraform validate.

A. True
B. False

Answer: B

Explanation:

A. True

Incorrect. While it is best practice to run terraform init first, it is not strictly required in all cases. terraform validate checks syntax and internal consistency and can run without initialization if no external modules or providers are required.

B. False

Correct. Initialization is recommended but not mandatory for terraform validate to execute.

Question: 257 CertyIQ

When does Terraform create the .terraform.lock.hcl file?

A. After your first terraform plan.
B. After your first terraform apply.
C. After your first terraform init.
D. Whenever you enable state locking.

Answer: C

Explanation:

A. After your first terraform plan

Incorrect. The lock file is created before planning occurs, during initialization.

B. After your first terraform apply

Incorrect. The lock file is not generated during apply; it is created earlier in the workflow.

C. After your first terraform init

Correct. Terraform creates the .terraform.lock.hcl file during terraform init when it selects and installs provider versions. The file records the exact provider versions to ensure consistent future runs.

D. Whenever you enable state locking

Incorrect. State locking is unrelated to the provider dependency lock file.

Question: 258 CertyIQ

You just scaled your VM infrastructure and realized you set the count variable to the wrong value. You correct the value and save your change. What do you do next to make your infrastructure match your configuration?

A. Inspect all Terraform outputs to make sure they are correct.
B. Inspect your Terraform state because you want to change it.
C. Run terraform apply and confirm the planned changes.
D. Reinitialize because your configuration has changed.

Answer: C

Explanation:

A. Inspect all Terraform outputs to make sure they are correct

Incorrect. Outputs only display values; they do not reconcile infrastructure with configuration.

B. Inspect your Terraform state because you want to change it

Incorrect. You should not manually modify the state file. Terraform automatically updates state when changes are applied.

C. Run terraform apply and confirm the planned changes

Correct. After correcting the configuration, you run terraform apply. Terraform will compare the configuration with the current state, generate a plan to adjust the resource count, and then apply the necessary changes.

D. Reinitialize because your configuration has changed

Incorrect. terraform init is only required when changing backends, modules, or providers — not for modifying resource arguments like count.

Question: 259 CertyIQ

You have a Terraform configuration that defines a single virtual machine with no references to it. You have run terraform apply to create the resource, then removed the resource definition from your Terraform configuration file.
What will happen when you run terraform apply in the working directory again?

A. Terraform will destroy the virtual machine.
B. Terraform will remove the virtual machine from the state file, but the resource will still exist.
C. Terraform will error.
D. Nothing.

Answer: A

Explanation:

A. Terraform will destroy the virtual machine.

Correct. Since the resource no longer exists in the configuration but is still present in the state file, Terraform will plan to destroy it to make the real infrastructure match the configuration.

B. Terraform will remove the virtual machine from the state file, but the resource will still exist.

Incorrect. Terraform does not automatically remove resources from state without affecting real infrastructure. That would require a manual terraform state rm command.

C. Terraform will error.

Incorrect. Terraform will not error; it will generate a plan showing that the resource will be destroyed.

D. Nothing.

Incorrect. Terraform detects that the resource exists in state but not in configuration and will take action.

Question: 260 CertyIQ

Which of these commands makes your code more human-readable?

A. terraform output
B. terraform fmt
C. terraform validate
D. terraform show

Answer: B

Explanation:

A. terraform output

Incorrect. terraform output displays output variable values after apply. It does not format or improve code readability.

B. terraform fmt

Correct. terraform fmt reformats Terraform configuration files to a canonical style, improving consistency and human readability.

C. terraform validate

Incorrect. terraform validate checks for syntax and configuration errors but does not change formatting.

D. terraform show

Incorrect. terraform show displays the contents of a state file or execution plan; it does not modify or format configuration files.

Question: 261 CertyIQ

If a DevOps team adopts AWS CloudFormation as their standardized method for provisioning public cloud resources, which of the following scenarios poses a challenge for this team?

A. The DevOps team is tasked with automating a manual, web console-based provisioning process.
B. The team is asked to manage a new application stack built on AWS-native services.
C. The organization decides to expand into Azure and wishes to deploy new infrastructure.
D. The team is asked to build a reusable code base that can deploy resources into any AWS region.

Answer: C

Explanation:

A. The DevOps team is tasked with automating a manual, web console-based provisioning process

Incorrect. CloudFormation is specifically designed to automate infrastructure provisioning, so this aligns well with its purpose.

B. The team is asked to manage a new application stack built on AWS-native services

Incorrect. CloudFormation is built for AWS-native services, so this is not a challenge.

C. The organization decides to expand into Azure and wishes to deploy new infrastructure

Correct. AWS CloudFormation is AWS-specific and does not support provisioning resources in Azure. Expanding to another cloud provider would require adopting another tool or switching to a multi-cloud solution.

D. The team is asked to build a reusable code base that can deploy resources into any AWS region

Incorrect. CloudFormation supports parameterization and templates that can be reused across AWS regions.

Question: 262 CertyIQ

A child module can always access variables declared in its parent module.

A. True
B. False

Answer: B

Explanation:

A. True

Incorrect. Terraform modules are isolated by design. A child module does **not** automatically inherit or access variables from its parent. Any value needed by the child must be explicitly passed from the parent module.

B. False

Correct. Child modules cannot directly access parent variables unless those values are explicitly provided as input variables. This isolation ensures modularity and reusability.

Question: 263 CertyIQ

Which of these actions are forbidden when the Terraform state file is locked? (Choose three.)

A. terraform apply
B. terraform plan
C. terraform fmt
D. terraform destroy
E. terraform state list
F. terraform validate

Answer: A,B,D

Explanation:

A. terraform apply

Correct. terraform apply modifies infrastructure and updates the state file. It requires a state lock, so it cannot

run if the state is already locked.

B. terraform plan

Correct. Although terraform plan primarily reads the state, it still attempts to acquire a state lock by default to prevent concurrent changes. If the state is locked, the command will fail.

C. terraform fmt

Incorrect. terraform fmt only reformats configuration files and does not interact with the state file.

D. terraform destroy

Correct. terraform destroy modifies infrastructure and updates the state file, so it requires a state lock and cannot run if the state is already locked.

E. terraform state list

Incorrect. This command reads the state file but does not modify it, so it does not require a lock in the same way.

F. terraform validate

Incorrect. terraform validate checks configuration syntax and does not interact with the state file.

Question: 264 CertyIQ

Which of the following commands would you use to access all of the attributes and details of a resource managed by Terraform?

A. terraform get 'provider_type.name'
B. terraform state list 'provider_type.name'
C. terraform state show 'provider_type.name'
D. terraform state list

Answer: C

Explanation:

A. terraform get 'provider_type.name'

Incorrect. terraform get is used to download and update modules referenced in your configuration. It does not inspect resources or their attributes.

B. terraform state list 'provider_type.name'

Incorrect. terraform state list only lists the addresses of resources in the state file. It does not show attribute values or detailed information.

C. terraform state show 'provider_type.name'

Correct. terraform state show displays all attributes and current values for a specific resource as stored in the Terraform state. This is the command used to inspect full resource details.

D. terraform state list

Incorrect. This command lists all resources tracked in the state file but provides no details about their attributes.

Question: 265 CertyIQ

Terraform providers are part of the Terraform core binary.

A. True
B. False

Answer: B

Explanation:

A. True

Incorrect. Providers are **not** built into the Terraform core binary. They are separate plugins that Terraform downloads during terraform init. This allows Terraform to support many different platforms and services without modifying the core binary.

B. False

Correct. Terraform Core and Terraform Providers are separate components. Providers are external plugins that interact with APIs (like AWS, Azure, etc.), while Terraform Core handles planning and state management.

Question: 266 CertyIQ

You want to define multiple data disks as nested blocks inside the resource block for a virtual machine. What Terraform feature would help you define the blocks using the values in a variable?

A. Count arguments
B. Dynamic blocks
C. Collection functions
D. Local values

Answer: B

Explanation:

A. Count arguments

Incorrect. The count argument is used to create multiple instances of a resource, not multiple nested blocks inside a single resource.

B. Dynamic blocks

Correct. **Dynamic blocks** allow you to generate multiple nested blocks inside a resource by iterating over a variable or collection. This is the appropriate feature for defining multiple data disks within a single virtual machine resource.

C. Collection functions

Incorrect. Collection functions manipulate data structures (lists, maps, etc.), but they do not create nested configuration blocks.

D. Local values

Incorrect. Local values help simplify expressions and reuse computed values, but they do not dynamically generate nested blocks.

Question: 267 CertyIQ

Before you can use a new backend or HCP Terraform/Terraform Cloud integration, you must first execute terraform init.

A. True
B. False

Answer: A

Explanation:

A. True

Correct. When configuring or changing a backend (including HCP Terraform/Terraform Cloud), you must run terraform init to initialize the working directory. This command configures the backend, downloads required providers, and sets up necessary integration.

B. False

Incorrect. Backend configuration changes require reinitialization using terraform init.

Question: 268 CertyIQ

You have deployed a new webapp with a public IP address on a cloud provider. However, you did not create any outputs for your code. What is the best method to quickly find the IP address of the resource you deployed?

A. Run terraform state list to find the name of the resource, then terraform state show to find the attributes including public IP address.
B. In a new folder, use the terraform_remote_state data source to load in the state file, then write an output for each resource that you find the state file.
C. Run terraform output ip_address to view the result.
D. Run terraform destroy then terraform apply and look for the IP address in stdout.

Answer: A

Explanation:

A. Run terraform state list to find the name of the resource, then terraform state show to find the attributes including public IP address.

Correct. Since no outputs were defined, the fastest way to retrieve the IP address is to inspect the Terraform state. First, use terraform state list to identify the resource name, then use terraform state show to view all its stored attributes, including the public IP.

B. In a new folder, use the terraform_remote_state data source to load in the state file, then write an output for each resource that you find in the state file.

Incorrect. This approach is unnecessarily complex and not the quickest way to retrieve the IP address.

C. Run terraform output ip_address

Incorrect. This command only works if an output named ip_address was already defined in the configuration.

D. Run terraform destroy then terraform apply and look for the IP address in stdout.

Incorrect. Destroying and recreating infrastructure just to find an IP address is unnecessary and risky.

Question: 269 CertyIQ

Which of the following is not a valid Terraform collection type?

A. tree
B. set
C. map
D. list

Answer: A

Explanation:

A. tree

Correct. tree is not a supported collection type in Terraform. Terraform's type system includes specific primitive and collection types, but tree is not one of them. While hierarchical data structures can be represented using nested maps and objects, there is no built-in tree collection type in Terraform.

B. set

Incorrect. set is a valid Terraform collection type. A set is an unordered collection of unique values, meaning duplicate elements are automatically removed. Sets are useful when order does not matter but uniqueness does.

C. map

Incorrect. map is a valid collection type. A map stores key-value pairs, where each key maps to a specific value. It is commonly used for structured configurations like tags or environment variables.

D. list

Incorrect. list is also a valid collection type. A list is an ordered sequence of values and allows duplicate elements. The position of each item is preserved.

Question: 270 CertyIQ

Which type of block fetches or computes information for use elsewhere in a Terraform configuration?

A. local

B. data
C. resource
D. provider

Answer: B

Explanation:

A. local

Incorrect. A local block defines local values within a configuration. It helps simplify expressions and reuse computed values, but it does not fetch external information from providers or APIs.

B. data

Correct. A data block is used to fetch or compute information from a provider without creating or managing the resource. It retrieves existing infrastructure data so it can be referenced elsewhere in the configuration.

C. resource

Incorrect. A resource block is used to create, update, or delete infrastructure components. It manages infrastructure rather than just fetching information.

D. provider

Incorrect. A provider block configures the provider (such as AWS or Azure) that Terraform will use to interact with APIs. It does not fetch resource-specific data itself.

Question: 271 CertyIQ

How can a ticket-based system slow down infrastructure provisioning and limit the ability to scale? (Choose two.)

A. End-users have to request infrastructure changes.
B. The more resources your organization needs, the more tickets your infrastructure team has to process.
C. Ticket-based systems generate a full audit trail of the request and fulfillment process.
D. Users can access a catalog of approved resources from dropdown lists in a request form.

Answer: A,B

Explanation:

A. End-users have to request infrastructure changes.

Correct. When users must submit tickets for infrastructure changes, provisioning becomes dependent on manual approval and processing. This introduces delays and reduces agility compared to automated, self-service infrastructure workflows.

B. The more resources your organization needs, the more tickets your infrastructure team has to process.

Correct. As demand grows, the volume of tickets increases. This creates bottlenecks for the infrastructure team, limiting scalability and slowing down provisioning.

C. Ticket-based systems generate a full audit trail of the request and fulfillment process.

Incorrect. An audit trail is a benefit, not a limitation. It improves governance and traceability rather than slowing scalability directly.

D. Users can access a catalog of approved resources from dropdown lists in a request form.

Incorrect. Providing a catalog of approved resources can actually streamline requests and improve consistency, rather than slow provisioning.

Question: 272 CertyIQ

Running terraform fmt without any flags in a directory with Terraform configuration files will check the formatting of those files, but will never change their contents.

A. True
B. False

Answer: B

Explanation:

A. True

Incorrect. By default, terraform fmt not only checks formatting but also automatically rewrites configuration files to match Terraform's canonical format if differences are found.

B. False

Correct. Running terraform fmt without flags will modify files in place to correct formatting issues.

Question: 273 CertyIQ

Which provider authentication method prevents credentials from being stored in the state file?

A. Using environment variables.
B. Specifying the login credentials in the provider block.
C. Setting credentials as Terraform variables.
D. None of the above.

Answer: A

Explanation:

A. Using environment variables.

Correct. Supplying credentials through environment variables keeps them outside of the Terraform configuration and state. This prevents sensitive values from being written into the state file.

B. Specifying the login credentials in the provider block.

Incorrect. Hardcoding credentials in the provider block can result in sensitive information being stored in configuration files and potentially written to state.

C. Setting credentials as Terraform variables.

Incorrect. Even if variables are marked sensitive, their values can still end up in the state file depending on usage. This does not reliably prevent credentials from being stored in state.

D. None of the above.

Incorrect. Using environment variables is a valid method to prevent credentials from being stored in the state file.

Question: 274 CertyIQ

What is a Terraform provider not responsible for?

A. Provisioning infrastructure in multiple cloud providers.
B. Managing actions to take based on resource differences.
C. Understanding API interactions with a hosted service.
D. Managing resources and data sources based on an API.

Answer: B

Explanation:

A. Provisioning infrastructure in multiple cloud providers.

Incorrect. Providers enable Terraform to provision infrastructure by interacting with a specific cloud or service API. Different providers support different platforms.

B. Managing actions to take based on resource differences.

Correct. Deciding what actions to take when differences are detected between configuration and state (such as create, update, or delete) is handled by **Terraform Core**, not the provider. Terraform Core generates the execution plan.

C. Understanding API interactions with a hosted service.

Incorrect. Providers are responsible for understanding and handling API interactions with their respective services.

D. Managing resources and data sources based on an API.

Incorrect. Providers define and manage resources and data sources by communicating with the service's API.

Question: 275 CertyIQ

When using multiple configurations of the same Terraform provider, what meta-argument must you include in any non-default provider configurations?

A. id
B. depends_on
C. name
D. alias

Answer: D

Explanation:

A. id

Incorrect. id is not a valid meta-argument for provider configuration. It is typically an attribute assigned to resources after creation.

B. depends_on

Incorrect. depends_on is a meta-argument used to define explicit dependencies between resources, not for provider configurations.

C. name

Incorrect. name is not used to distinguish multiple configurations of the same provider.

D. alias

Correct. When defining multiple configurations of the same provider, you must include the alias meta-argument for any non-default provider configuration. This allows Terraform to distinguish between the different provider instances.

Question: 276 CertyIQ

Anyone can publish modules to the public Terraform Registry.

A. True
B. False

Answer: A

Explanation:

A. True

Correct. Anyone can publish modules to the public Terraform Registry as long as they meet the publishing requirements (such as using a supported VCS like GitHub and following the naming conventions). There is no approval process that restricts publishing to a limited group of users.

B. False

Incorrect. The public Terraform Registry is open for module publishing to anyone who follows the required guidelines.

Question: 277 CertyIQ

What type of information can be found on the Terraform Registry when using published modules?

A. Required input variables.
B. Outputs.
C. Optional input variables and default values.

D. All of the above.

Answer: D

Explanation:

A. Required input variables

Incorrect. While required input variables are documented on the Terraform Registry, they are not the only type of information available.

B. Outputs

Incorrect. Module outputs are also documented, but again, they are only part of the available information.

C. Optional input variables and default values

Incorrect. Optional variables and their default values are included in module documentation, but this is not the complete answer.

D. All of the above

Correct. The Terraform Registry displays comprehensive module documentation, including required input variables, optional variables with default values, and defined outputs.

Question: 278

CertyIQ

One cloud block always maps to a single HCP Terraform/Terraform Cloud workspace.

A. True
B. False

Answer: A

Explanation:

A. True

Correct. In HCP Terraform/Terraform Cloud, each cloud block in a configuration corresponds to a single workspace. This ensures that state management, runs, and integrations are scoped to that specific workspace.

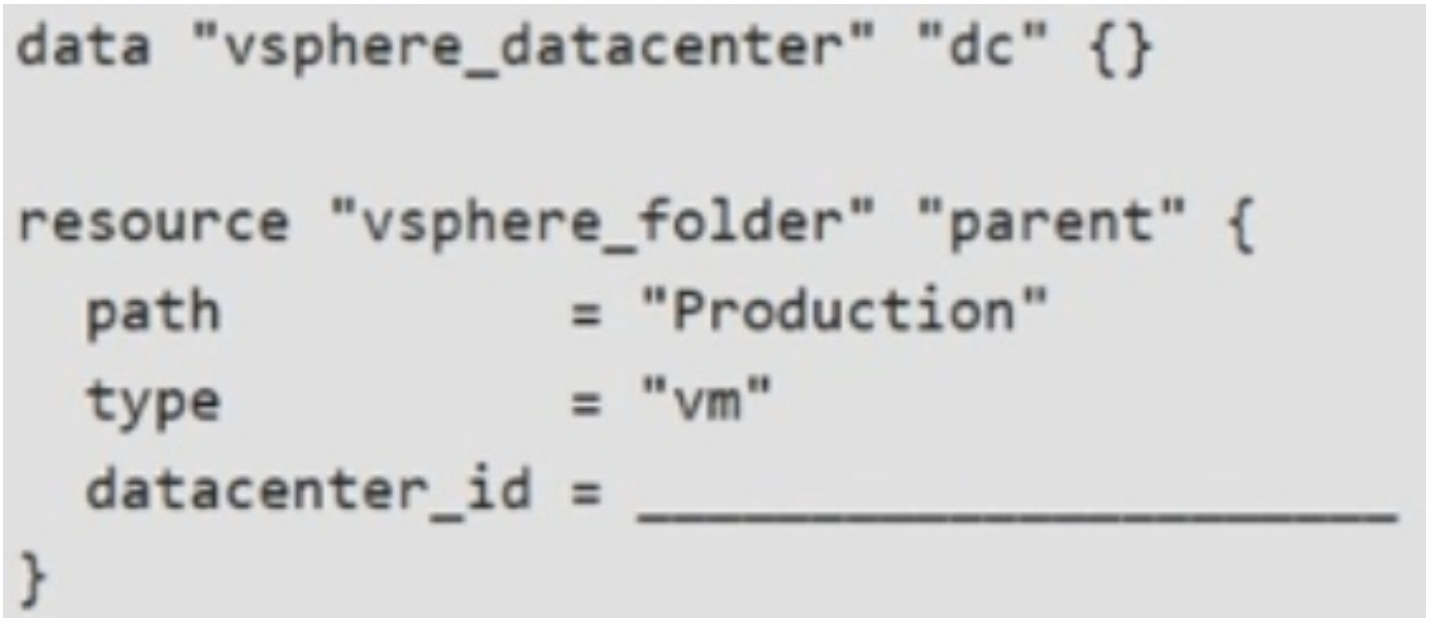
B. False

Incorrect. A cloud block does not span multiple workspaces; it is always tied to one workspace.

Question: 279

CertyIQ

A configuration is shown in the Exhibit space of this page.



How would you reference an attribute from the vsphere_datacenter data source for use with the datacenter_id argument within the vsphere_folder resource in the configuration?

A. data.vsphere_datacenter.dc
B. vsphere_datacenter.dc.id
C. data.dc.id
D. data.vsphere_datacenter.dc.id

Answer: D

Explanation:

The correct way to reference an attribute from a vsphere_datacenter data source for use with the datacenter_id argument is data.vsphere_datacenter.dc.id. This follows the standard Terraform syntax for accessing data source attributes: data.TYPE.NAME.ATTRIBUTE.

Question: 280

Which of these are features of HCP Terraform/Terraform Cloud? (Choose two.)

A. Automatic backups of configuration and state.
B. A web-based user interface (UI).
C. Remote state storage.
D. Automated infrastructure deployment visualization.

Answer: B,C

Explanation:

HCP Terraform and Terraform Cloud are platforms designed to enhance collaboration, automation, and governance for Infrastructure as Code (IaC) workflows using Terraform. Among the listed options, "A web-based user interface (UI)" and "Remote state storage" are distinct and fundamental features.

Option B, "A web-based user interface (UI)," is a core component of HCP Terraform and Terraform Cloud. As a Software as a Service (SaaS) offering, the platform provides a comprehensive web console. This UI allows users to centrally manage workspaces, variables, run policies, review plan outputs, inspect deployment

histories, and execute infrastructure changes without exclusive reliance on the local Terraform CLI. This centralized interface significantly improves accessibility, enhances collaboration across teams, and provides a clear, unified overview of all infrastructure operations, aligning with modern cloud management principles for improved user experience.

Option C, "Remote state storage," is another critical feature provided natively by HCP Terraform/Terraform Cloud. Terraform relies on a state file to maintain a mapping between real-world infrastructure resources and your configuration, tracking metadata and dependencies. For collaborative environments and to ensure idempotency and consistency, storing this state remotely is paramount. HCP Terraform/Terraform Cloud automatically handles remote state management, offering essential capabilities such as state locking to prevent concurrent writes, state versioning for easy rollbacks, and secure access control. This built-in functionality eliminates the complexities and risks associated with manual state file management, ensuring data integrity and reliable infrastructure deployments across multiple developers and teams.

In contrast, Option A, "Automatic backups of configuration and state," is less accurate as a primary, directly configurable user-facing feature for both elements. While the platform offers robust state versioning and internal redundancy for the state file (which acts as a form of backup/recovery for the state), the configuration (your .tf files) is typically managed in an external Version Control System (VCS) like GitHub or GitLab. Terraform Cloud integrates with these VCSs rather than performing "backups" of your source code repository itself. Similarly, Option D, "Automated infrastructure deployment visualization," is not a direct, dynamic graphical feature within the core HCP Terraform/Terraform Cloud UI. While Terraform produces a detailed textual plan output describing intended changes, and terraform graph can show resource dependencies, the platform doesn't inherently offer real-time, dynamic visual diagrams of the deployment process or a live infrastructure topology within its main interface.

Therefore, the most prominent and directly provided features from the given choices are its comprehensive web-based user interface and integrated remote state management.

Authoritative Links for Further Research:

HCP Terraform Overview: https://cloud.hashicorp.com/products/terraform
Terraform Cloud Workspaces (UI functionality): https://developer.hashicorp.com/terraform/cloud/workspaces
Terraform State Management (Remote State): https://developer.hashicorp.com/terraform/cloud/workspaces/state

Question: 281 CertyIQ

What does Terraform not reference when running a terraform apply -refresh-only?

A. Credentials.
B. State file.
C. Terraform resource definitions in configuration files.
D. Cloud provider.

Answer: C

Explanation:

The correct answer is **C. Terraform resource definitions in configuration files.**

Here's a detailed justification:

The terraform apply -refresh-only command serves a very specific purpose within the Terraform workflow: to

update the Terraform state file to accurately reflect the current real-world status of infrastructure resources without making any changes to that infrastructure.

1. **Why A, B, and D are referenced:**

A. Credentials: To interact with any cloud provider to query the current status of resources, Terraform absolutely requires appropriate authentication credentials (e.g., API keys, IAM roles, service principal secrets). Without credentials, Terraform cannot communicate with the cloud API.
B. State file: The refresh-only command's primary objective is to update the state file. It reads the existing state file to identify which resources it currently manages, queries the cloud provider for the actual status of those specific resources, and then writes the refreshed data back into the state file. Therefore, the state file is extensively referenced (read from and written to).
D. Cloud provider: The actual source of truth for the current state of infrastructure is the cloud provider itself. Terraform must communicate with the cloud provider's APIs to fetch the latest attributes and status of all managed resources.

1. **Why C is not referenced in the context of refresh-only:**

Terraform configuration files (.tf files) define the desired state of your infrastructure. When you run a standard terraform plan or terraform apply, Terraform reads these configuration files, compares the desired state with the current state (from the state file and cloud), and generates a plan of changes needed to reconcile any differences.
However, terraform apply -refresh-only explicitly bypasses the configuration file evaluation step for the purpose of generating a plan for changes. Its sole focus is on synchronizing the existing state file with the actual state of resources in the cloud. It doesn't look at your configuration files to determine if new resources should be added, existing ones modified, or old ones removed. It simply updates the recorded attributes of the resources it already knows about from the state file.
Essentially, refresh-only does not consult the configuration files to formulate a new desired state or identify any drift against the configuration. It only identifies drift between the current state file and the actual cloud resources.

In summary, terraform apply -refresh-only operates by querying the cloud provider (using credentials) about resources listed in the current state file, and then updating that state file with the findings. It specifically avoids re-evaluating the Terraform resource definitions in your configuration files because its goal is not to plan or apply changes based on a desired configuration, but merely to update the state to reflect reality.

For further research, refer to:

HashiCorp Terraform Command: terraform
apply Documentation: https://developer.hashicorp.com/terraform/cli/commands/apply (Specifically, look for the -refresh-only flag explanation).
HashiCorp Terraform State Documentation: https://developer.hashicorp.com/terraform/language/state

Question: 282 CertyIQ

Which of the following is not a key principle of infrastructure as code (IaC)?

A. Golden images
B. Self-describing infrastructure
C. Versioned infrastructure
D. Idempotence

Answer: A

Explanation:

The correct answer is A. Golden images. This is because golden images represent a common artifact or strategy used in infrastructure deployment, but they are not a fundamental principle of Infrastructure as Code (IaC) itself. IaC revolves around treating infrastructure configuration and provisioning like software development, emphasizing automated, consistent, and repeatable processes.

Let's examine why the other options are key principles and why golden images are distinct:

Self-describing infrastructure is a core IaC principle because infrastructure is defined using declarative configuration files (e.g., JSON, YAML, HCL). These files explicitly describe the desired state of the infrastructure components, making the infrastructure's design and configuration transparent and human-readable, as well as machine-executable. This clarity reduces ambiguity and allows teams to understand the infrastructure by reading its code.

Versioned infrastructure is crucial, mirroring software development practices. IaC configurations are stored in version control systems (like Git), enabling a complete history of changes, collaboration among teams, easy rollback to previous states, and clear audit trails. This ensures consistency, prevents configuration drift, and supports agile development methodologies.

Idempotence is perhaps the most defining characteristic of IaC. It means that applying the same IaC configuration script multiple times will always yield the same infrastructure state without causing unintended side effects or errors after the initial application. This principle ensures predictability and reliability, allowing for safe and consistent deployments, updates, or reconfigurations, as the system intelligently determines if actions need to be taken or if the desired state is already met.

Golden images, on the other hand, are pre-configured, pre-hardened, and often pre-patched virtual machine or container images. They serve as a base template for deploying instances, ensuring a consistent starting point. While golden images are frequently used in conjunction with IaC to provide the base operating system and software stack, they are an output or artifact of an infrastructure pipeline, rather than a principle guiding how that infrastructure is defined and managed through code. IaC principles dictate how you declare, version, and manage the deployment of instances based on these golden images, and how you configure them further. Using IaC to build and manage the golden images themselves, and then provision from them, enhances the overall automation strategy, but the image itself isn't a principle.

In summary, golden images are a strategy for achieving consistency and immutability often enabled by IaC, but not a fundamental guiding principle of how IaC functions or defines infrastructure through code.

Authoritative Links for further research:

What is Infrastructure as Code?
AWS: https://aws.amazon.com/devops/what-is-iac/
Microsoft Azure: https://learn.microsoft.com/en-us/devops/deliver/what-is-infrastructure-as-code
Idempotence in IaC:
Puppet: https://www.puppet.com/blog/infrastructure-as-code-idc-and-idempotence
Golden Images (in context of IaC):
HashiCorp Blog: https://www.hashicorp.com/blog/packer-0-12-0-released (mentioning Packer for building golden images, which is often part of an IaC workflow)

Question: 283 CertyIQ

You use a Cloud provider account that is shared with other team members. You previously used Terraform to create a load balancer that listens on port 80. After some application changes, you updated the Terraform code to

change the port to 443.
You run terraform plan and see that the execution plan shows the port changing from 80 to 443 like you intended, and step away to grab some coffee.
In the meantime, another team member manually changes the load balancer port to 443 through the Cloud provider console before you get back to your desk.
What will happen when you run terraform apply upon returning to your desk?

A. Terraform will not make any changes to the load balancer and will update the state file to reflect the manual change.

B. Terraform will fail with an error because the state file is no longer accurate.

C. Terraform will change the load balancer port to 80, and then change it back to 443.

D. Terraform will recreate the load balancer.

Answer: C

Explanation:

When terraform apply is executed, Terraform follows a specific reconciliation process. It doesn't simply execute the previously generated plan verbatim without re-evaluation.

Here's a detailed breakdown of what happens:

1. **Implicit Refresh:** The terraform apply command always begins with an implicit terraform refresh. During this phase, Terraform queries the cloud provider (in this case, for the load balancer) to fetch its actual current configuration. It discovers that the load balancer's port is now 443 due to the manual intervention, while its own terraform.tfstate file still records the port as 80.
2. **Drift Detection:** This discrepancy between the terraform.tfstate file (port 80) and the actual cloud infrastructure (port 443) is identified as "drift."
3. **Reconciling Desired vs. Actual vs. State:** Terraform's goal is to bring the actual infrastructure in line with the desired state defined in the HCL configuration.

Desired State (HCL): Port 443
Actual State (Cloud): Port 443
Recorded State (tfstate file): Port 80

1. **The "Revert and Reapply" Mechanism:** While ideally Terraform would recognize that the actual state already matches the desired state and simply update the state file, certain cloud providers or resource types in Terraform can exhibit a "revert and reapply" behavior in drift scenarios. This happens because Terraform's core logic is to apply the transformation from the recorded state in the tfstate file to the desired state in the HCL. The terraform plan you ran before the manual change indicated a transition from 80 to 443.
2. **Provider Interpretation of Updates:** When Terraform instructs the provider to "update the port from 80 to 443," the provider's update logic might not always be perfectly idempotent with respect to out-of-band changes. If the provider's API or its internal logic isn't designed to handle the scenario where the target attribute already matches the desired state, it might perform a two-step process:

It might first attempt to reconcile the resource to the state it expected it to be in (i.e., port 80, as recorded in the state file). This can manifest as the load balancer port being temporarily changed back to 80.
Immediately after, it then applies the final desired state from the HCL configuration, changing the port from 80 to 443.

1. **Final State:** As a result, the load balancer's port will temporarily revert to 80 and then be changed back to 443, aligning both the cloud infrastructure and the updated terraform.tfstate file with the desired configuration. This action, although seemingly redundant, is a known behavior in specific drift scenarios where Terraform enforces the explicit path of change defined by its state and

configuration.

Therefore, the correct answer is C, as Terraform attempts to apply the planned transformation from its last known state, potentially causing a temporary reversion before achieving the final desired state.

Authoritative Links for Further Research:

Terraform State: Understanding the role of the state file is fundamental.
HashiCorp Learn - Terraform State: https://developer.hashicorp.com/docs/terraform/concepts/state
Terraform Refresh and Drift: Explains how Terraform detects and handles discrepancies.
HashiCorp Docs - terraform refresh: https://developer.hashicorp.com/docs/terraform/cli/commands/refresh
HashiCorp Blog - Detecting and Correcting Configuration Drift with Terraform: https://www.hashicorp.com/blog/detecting-and-correcting-configuration-drift-with-terraform (While it states apply updates the state file, it doesn't preclude the revert/reapply for certain providers/resources).

Question: 284 CertyIQ

When using Terraform to deploy resources into Azure, which scenarios are true regarding state files?

A. Changing resources via the Azure Cloud Console records the change in the current state file.
B. When you change a resource via the Azure Cloud Console. Terraform records the changes in a new state file.
C. When you change a Terraform-managed resource via the Azure Cloud Console. Terraform updates the state file to reflect the change during the next plan or apply.
D. Changing resources via the Azure Cloud Console does not update current state file.

Answer: C

Explanation:

Terraform state files are crucial for tracking the infrastructure resources it manages. They serve as a mapping between your configuration files and the real-world resources existing in the cloud provider (Azure, in this case). The state file records metadata about your infrastructure, improves performance for large infrastructures, and, most importantly, keeps track of the actual properties of your deployed resources.

When a Terraform-managed resource is modified directly through the Azure Cloud Console (or CLI/API), this is referred to as "out-of-band" or "manual" modification. Terraform itself does not have a real-time monitoring mechanism actively watching for such changes in your cloud environment. Therefore, the state file is not automatically updated the moment a change is made in Azure.

The update process for the state file occurs when you next execute a terraform plan or terraform apply command. During a terraform plan operation, Terraform fetches the current actual state of your infrastructure resources from Azure. It then compares this actual state with the desired state defined in your Terraform configuration files and with the last known state recorded in its state file. If a discrepancy is found —meaning the actual resource in Azure differs from what Terraform believes it should be or what's recorded in its state—Terraform identifies this as "drift."

The terraform plan output will show what actions Terraform proposes to take to reconcile this drift, typically aiming to bring the Azure resource back into alignment with your Terraform configuration. If you then proceed with terraform apply, Terraform will execute these proposed changes in Azure. Upon successful completion of the apply operation, Terraform updates its state file to reflect the new, reconciled state of the resources. This ensures that the state file accurately reflects the infrastructure as managed by Terraform after the apply operation has concluded.

Therefore, option C correctly describes this behavior: "When you change a Terraform-managed resource via the Azure Cloud Console. Terraform updates the state file to reflect the change during the next plan or apply." The detection happens during plan, and the update of the state file occurs as a result of the apply operation, after Terraform has taken action to reconcile the resource. Options A and B are incorrect because Terraform doesn't automatically record direct changes; it's not a real-time monitoring system. Option D is incorrect because the state file does get updated, but only when Terraform interacts with the cloud provider during plan or apply to discover and reconcile changes.

Authoritative Links for Further Research:

Terraform State: https://developer.hashicorp.com/terraform/language/state
Terraform Drift Detection: https://developer.hashicorp.com/terraform/cli/commands/plan#detecting-and-managing-drift
Terraform Apply: https://developer.hashicorp.com/terraform/cli/commands/apply

Question: 285 CertyIQ

You have developed a new cloud-based service that uses proprietary APIs and want to use Terraform to create, manage, and delete users from the service. How can Terraform interact with the service?

A. Terraform can manage users for any service that is hosted on a public cloud provider.
B. Develop and publish a customer provider to interact with the service using its proprietary APIs.

Answer: B

Explanation:

Terraform is a powerful Infrastructure as Code (IaC) tool designed for provisioning and managing resources across a multitude of platforms and services. Its core functionality relies on a plugin-based architecture, where "providers" act as the essential bridge between Terraform configurations and external APIs. A Terraform provider is essentially a plugin that understands the Application Programming Interface (API) of a specific service and translates Terraform's desired state configurations into specific API calls to manage resources on that service.

For widely adopted services like AWS, Azure, Google Cloud, or Kubernetes, official or community-maintained providers already exist, enabling seamless resource management. However, the scenario describes a new cloud-based service that utilizes proprietary APIs. This crucial detail signifies that the service's interface for managing resources, such as users, is unique and not standardized or universally recognized. Consequently, no pre-existing Terraform provider would inherently understand how to communicate with these specific, bespoke APIs.

To enable Terraform to interact with such a custom service and manage its users, a **custom provider must be developed**. This custom provider would encapsulate the necessary logic to authenticate with the proprietary service, handle the specific request formats for creating, reading, updating, and deleting (CRUD) user resources via its unique APIs, and interpret the service's API responses back into Terraform's state. This process aligns perfectly with Terraform's extensible architecture, allowing it to adapt to virtually any API-driven system.

Option A is incorrect because Terraform's capability to manage resources is not universally granted simply by a service being "hosted on a public cloud provider." Terraform does not magically gain the ability to understand proprietary APIs just because a service is cloud-hosted; it always requires a specific provider to bridge the communication gap. While many public cloud services do have Terraform providers, this is because cloud vendors or the community developed them, not an inherent Terraform feature for all services

irrespective of their API design. Therefore, developing and publishing a custom provider is the definitive and only method for integrating a service with proprietary APIs into a Terraform workflow for user management.

Authoritative Links for Further Research:

Terraform Providers Overview: https://developer.hashicorp.com/terraform/providers
Developing Custom Terraform Providers: https://developer.hashicorp.com/terraform/plugin/how-to/begin
What is Infrastructure as Code (IaC): https://www.redhat.com/en/topics/automation/what-is-infrastructure-as-code

Question: 286 CertyIQ

You have just developed a new Terraform configuration for two virtual machines with a cloud provider. You would like to create the infrastructure for the first time.
Which Terraform command should you run first?

A. terraform apply
B. terraform plan
C. terraform show
D. terraform init

Answer: D

Explanation:

When developing a new Terraform configuration for virtual machines and aiming to create infrastructure for the first time, the terraform init command is the essential first step. This command prepares the working directory for all subsequent Terraform operations. Its primary function is to download and install the necessary provider plugins specified in your configuration, such as those for AWS, Azure, or Google Cloud Platform. These plugins are crucial as they enable Terraform to interact with the respective cloud provider's APIs to provision, manage, and query resources like virtual machines. Without terraform init, Terraform would lack the fundamental components required to communicate with your chosen cloud provider.

Furthermore, terraform init initializes the configured backend, which dictates where Terraform state files will be stored. The state file is vital for Terraform to map your configuration to real-world infrastructure, track resource metadata, and understand their current status. The command also processes and downloads any specified modules from their source. During its execution, terraform init creates the .terraform directory, which houses all the downloaded plugins, modules, and other essential configuration data. It also generates or updates the .terraform.lock.hcl file, which records the exact versions of providers used, ensuring consistency across environments.

Subsequent commands, like terraform plan and terraform apply, entirely depend on this initialized environment. For example, terraform plan requires the provider plugins to query the cloud provider and ascertain the proposed changes. Similarly, terraform apply needs both the plugins and the backend configuration to actually provision the resources and accurately update the state. Attempting to run terraform apply or terraform plan before terraform init would inevitably result in errors, as Terraform would lack the foundational components to function. The terraform show command is used to display plan output or the current state and is therefore not an initial setup step. Thus, terraform init lays the necessary groundwork, making it the mandatory prerequisite for any new Terraform project.

For further research:

Terraform init command documentation: https://developer.hashicorp.com/terraform/cli/commands/init

Terraform Providers: https://developer.hashicorp.com/terraform/language/providers
Terraform State: https://developer.hashicorp.com/terraform/language/state

Question: 287 CertyIQ

You want to define a single input variable to capture configuration values for a server. The values must represent memory as a number, and the server name as a string.
Which variable type could you use for this input?

A. List
B. Map
C. Object
D. Terraform does not support complex input variables of different types

Answer: C

Explanation:

When defining a single input variable in Terraform to capture configuration values of different types, such as a memory value (number) and a server name (string), the object variable type is the most appropriate and robust choice.

An object type allows you to define a structured collection of named attributes, where each attribute can have its own explicit type constraint. This means you can specify that an attribute named memory must be a number and an attribute named server_name must be a string, all within the confines of a single parent variable. This provides strong type checking and clear schema definition for your input data.

Let's look at why other options are less suitable:

A. List: A list variable type is an ordered collection of values, where all elements are typically expected to be of the same type or a very consistent structure. While you could technically put different types into a list if the element type is not explicitly defined, it lacks the named attributes and schema validation necessary to clearly distinguish "memory" from "server name" and enforce their specific types. It wouldn't be clear which element represents what without external context.
B. Map: A map variable type is an unordered collection of key-value pairs. While keys are always strings, the values can be of different types. However, a map does not allow you to define a fixed schema for its values where you explicitly state, for instance, that the value associated with the key "memory" must be a number, and the value for "server_name" must be a string. While map(any) allows mixed types, it sacrifices type safety and explicit validation that object provides for structured data. For structured data with known attribute names and types, object is preferred.
D. Terraform does not support complex input variables of different types: This statement is incorrect. Terraform explicitly supports complex variables that can encapsulate data of different types, with object being the primary mechanism for this.

The object type aligns perfectly with the requirement because it allows you to define a custom, explicit schema for your server configuration. For example, you would declare the variable like this:

variable "server_config" description = "Configuration details for a server" type = object(memory = number serve

This approach ensures that when users provide input for server_config, Terraform validates that memory is indeed a number and server_name is a string, preventing common configuration errors. This explicit schema definition enhances readability, maintainability, and reliability of Infrastructure as Code (IaC) templates, which is crucial in cloud computing environments where precise resource provisioning is paramount. Using objects

promotes modularity and helps maintain consistent configurations across deployments by enforcing data structures.

For further research:

Terraform Documentation on Input Variables: https://developer.hashicorp.com/terraform/language/values/variables
Terraform Type Constraints (specifically Object, List, Map): https://developer.hashicorp.com/terraform/language/expressions/type-constraints

Question: 288 CertyIQ

What feature stops multiple users from operating on the Terraform state at the same time?

A. Remote state storage
B. Provider constraints
C. State locking
D. Version control

Answer: C

Explanation:

Terraform state acts as the definitive source of truth, reflecting the real-world infrastructure managed by your configuration. If multiple users or automated processes attempt to modify this critical state concurrently, it can lead to severe issues such as race conditions, data corruption, and inconsistent infrastructure deployments. This scenario undermines the reliability and idempotency of Infrastructure as Code (IaC), potentially causing resource loss or misconfigurations.

To prevent these detrimental outcomes, Terraform employs a crucial feature known as **state locking**. State locking ensures that only one operation can acquire an exclusive lock on the Terraform state at any given moment. When a terraform plan, terraform apply, or terraform destroy command is executed, Terraform first attempts to acquire this lock. If the lock is successfully obtained, the operation proceeds; if another operation already holds the lock, the new operation will either wait for the lock to be released or fail, depending on the backend configuration. This mechanism implements a form of mutual exclusion, which is fundamental for maintaining consistency in distributed systems and collaborative cloud environments.

State locking is inherently tied to the use of remote backends (e.g., Amazon S3 with DynamoDB, Azure Storage Accounts with Blob locking, HashiCorp Consul, HashiCorp Cloud). These backends not only store the state file remotely but also provide the underlying infrastructure to implement robust locking primitives. While "Remote state storage" (Option A) is essential for sharing state across teams, it is the specific locking mechanism provided by the backend that actively stops concurrent modifications. Without such a mechanism, even a remotely stored state file would be vulnerable to corruption from simultaneous writes.

Conversely, "Provider constraints" (Option B) define acceptable versions for Terraform providers, ensuring compatibility and stability within the configuration, but they do not regulate access to the state file itself. "Version control" (Option D), such as Git, is excellent for managing the .tf configuration files and their history, allowing collaboration on the code. However, it does not manage the dynamic Terraform state file directly or prevent concurrent operations on the active state during live terraform apply executions. Therefore, state locking is the explicit and dedicated feature designed to guarantee serial execution of state-modifying operations, safeguarding the integrity and consistency of your infrastructure deployments in multi-user environments.

For further research:

Terraform Backends and State
Locking: https://developer.hashicorp.com/terraform/language/settings/backends/configuration
Example of State Locking with S3
Backend: https://developer.hashicorp.com/terraform/language/settings/backends/s3#state-locking-and-consistency

Question: 289 CertyIQ

terraform validate confirms that your infrastructure matches the Terraform state file.

A. True
B. False

Answer: B

Explanation:

The statement "terraform validate confirms that your infrastructure matches the Terraform state file" is **False**.

terraform validate serves a distinct purpose within the Terraform workflow: it primarily checks the configuration files (.tf) in the working directory for syntactical validity and internal consistency. This includes verifying that resource attributes are of the correct type, identifying undeclared variables, and ensuring proper interpolation of values. It performs a static analysis of the code, much like a compiler or linter, to catch configuration errors before any interaction with a cloud provider or the state file.

Crucially, terraform validate does not interact with your actual cloud infrastructure or compare the configuration against the existing Terraform state file (terraform.tfstate). Its scope is limited to the local configuration definitions. It operates entirely offline, without needing cloud provider credentials or an initialized backend.

The command responsible for comparing your desired infrastructure configuration with the actual existing infrastructure and the Terraform state file is terraform plan. terraform plan queries the current state of your resources from the cloud provider APIs, compares this with the records in the state file, and then evaluates the differences against your desired configuration files. It generates an execution plan detailing what actions Terraform proposes to take (create, modify, destroy) to bring the real infrastructure into alignment with your configuration.

Following terraform plan, the terraform apply command executes these proposed changes, reconciling the actual infrastructure with the desired state defined in your configuration. After successful execution, terraform apply then updates the Terraform state file to reflect the new actual state of your infrastructure. Therefore, while the state file acts as a record of your infrastructure, and plan/apply ensure alignment, validate is merely a preliminary syntax and configuration check.

For further research, refer to the official Terraform documentation:

terraform validate: https://developer.hashicorp.com/terraform/cli/commands/validate
terraform plan: https://developer.hashicorp.com/terraform/cli/commands/plan
Terraform State: https://developer.hashicorp.com/terraform/language/state

Question: 290 CertyIQ

What is the provider for the resource shown in the Exhibit space on this page?

```
resource "aws_vpc" "main" {
  name = "test"
}
```

A. test
B. main
C. vpc
D. aws

Answer: D

Question: 291 CertyIQ

terraform validate confirms the syntax of Terraform files.

A. True
B. False

Answer: A

Explanation:

The statement "terraform validate confirms the syntax of Terraform files" is **True**.

Detailed Justification:

terraform validate is a fundamental command in the Terraform workflow designed specifically to check the configuration files in a directory for syntactical correctness and internal consistency. It performs static analysis on the Terraform code written in HashiCorp Configuration Language (HCL). This means it parses the files to ensure they conform to the HCL grammar, verifying that blocks are properly opened and closed, arguments are correctly assigned, and resource types, data sources, variables, outputs, and module references are declared in a valid format.

Beyond just basic syntax, terraform validate also checks for other configuration validity issues. For instance, it ensures that variable declarations are well-formed, provider configurations are correctly specified (e.g., required arguments are present), and expressions are parsable. Crucially, this command **does not** interact with any remote state or cloud provider APIs. It operates purely on the local configuration files, making it a fast and efficient way to catch errors early in the development cycle.

This early validation is invaluable in an Infrastructure as Code (IaC) context, especially when managing resources across various cloud platforms like AWS, Azure, or Google Cloud. By confirming the syntax and basic configuration validity locally, developers can identify and fix common mistakes before attempting more resource-intensive operations like terraform plan or terraform apply. While terraform plan also implicitly performs a validation step, terraform validate is a standalone command specifically for this purpose, providing quicker feedback and allowing it to be integrated easily into pre-commit hooks or Continuous Integration (CI)

pipelines. This ensures that only syntactically sound code proceeds through the deployment pipeline, reducing the risk of errors during actual infrastructure provisioning.

For further research, refer to the official HashiCorp documentation:

terraform validate command: https://developer.hashicorp.com/terraform/cli/commands/validate
Terraform CLI Overview: https://developer.hashicorp.com/terraform/cli

Question: 292 CertyIQ

You can access state stored in a local backend by using the terraform_remote_state data source.

A. True
B. False

Answer: B

Explanation:

Here's a detailed justification for why accessing local backend state with the terraform_remote_state data source is incorrect:

The terraform_remote_state data source is specifically designed to retrieve state data from remote backends, not local backends. Local backends store Terraform state files on the local machine, typically in a file named terraform.tfstate. They are intended for development, experimentation, and single-user environments.

The primary purpose of terraform_remote_state is to access state information from remote backends like AWS S3, Azure Blob Storage, HashiCorp Cloud, or Google Cloud Storage. These backends provide centralized, versioned, and often encrypted storage for state files, enabling collaboration among team members and managing infrastructure at scale.

To access the outputs of a Terraform configuration using a local backend, you do not use terraform_remote_state. Instead, the configurations would typically be combined within the same workspace or the state file can be accessed via the terraform output command. The terraform output command retrieves the values of output variables defined in a Terraform configuration.

The concept of remote state management is crucial for collaboration and avoiding state corruption. Using a remote backend ensures that all team members have access to the same, up-to-date state file, preventing conflicts and data loss. Because terraform_remote_state is tailored for this specific purpose, it cannot be used with local backends. It requires the configuration of a specific remote backend during initialization.

The statement is false because using terraform_remote_state with a local backend will not work and is not the intended use of that data source. It's essential to understand the difference between local and remote backends and when to use the terraform_remote_state data source.

For further information, consult the official Terraform documentation:

Terraform State
terraform_remote_state
Backends Overview

Question: 293 CertyIQ

You have a list of numbers that represents the number of free CPU cores on each virtual cluster: numcpus = [18, 3, 7, 11, 2]
What Terraform function could you use to select the largest number from the list?

A. high[numcpus]
B. max(numcpus)
C. top(numcpus)
D. ceil(numcpus)

Answer: B

Explanation:

The correct Terraform function to select the largest number from a list is max(numcpus).

The max() function in Terraform is specifically designed to return the greatest of a given set of numbers. It can accept multiple individual number arguments or a single list (or tuple) of numbers, iterating through them to identify and return the highest value. In this scenario, with numcpus = [18, 3, 7, 11, 2], applying max(numcpus) would correctly evaluate to 18, representing the virtual cluster with the most free CPU cores. This capability is crucial in cloud computing environments where resource visibility is key, allowing administrators to quickly identify the most available resources for deploying new services or scaling existing ones, such as additional Symantec Web Protection (Edge SWG R2) instances.

Let's examine why the other options are incorrect:

A. high[numcpus]: This is not a valid Terraform function or syntax for retrieving the maximum value from a list. It resembles an attempt at array indexing or a custom function name, neither of which aligns with Terraform's built-in mathematical functions.
C. top(numcpus): Terraform does not have a built-in top() function for this purpose. While some programming languages or data manipulation tools might offer a top function (often for retrieving the top N elements), it is not a standard part of the Terraform language for finding a single maximum value.
D. ceil(numcpus): The ceil() function in Terraform (and general mathematics) calculates the smallest integer greater than or equal to a given single number (the "ceiling" value). For example, ceil(3.14) would return 4. It is not designed to operate on a list of numbers to find the maximum value, and attempting to pass a list to ceil() would typically result in a type error.

Therefore, max(numcpus) is the appropriate and only correct function among the choices for determining the largest number in the provided list, which is essential for effective resource management and capacity planning in a dynamic cloud infrastructure.

For further research on Terraform's built-in functions:

Terraform max() function documentation: https://developer.hashicorp.com/terraform/language/functions/max
Terraform ceil() function documentation: https://developer.hashicorp.com/terraform/language/functions/ceil

Question: 294 CertyIQ

Your root module contains a variable named num_servers. Which is the correct way to pass its value to a child module with an input named servers?

A. servers = var(num_servers)
B. servers = var.num_servers
C. servers = num_servers

D. $ var.num_servers

Answer: B

Explanation:

The correct way to pass the value of a root module variable named num_servers to a child module input named servers is servers = var.num_servers. This is based on the widely adopted HashiCorp Configuration Language (HCL) syntax, which tools like Symantec Web Protection's Edge SWG R2 (if it uses an HCL-like configuration) would typically follow for defining and managing infrastructure as code.

Here's a detailed justification:

1. **Modules in Infrastructure as Code:** Modules serve as reusable, self-contained units of configuration. A root module orchestrates the overall infrastructure, often by calling and configuring child modules. Child modules encapsulate specific sets of resources and expose input variables (also known as arguments or parameters) to allow the calling module to customize their behavior.
2. **Defining Variables:** In the root module, num_servers would be declared as an input variable using a variable block, for example:

variable "num_servers" description = "The number of servers to deploy." type = number default = 1

1. This declaration makes num_servers available for use within the root module.
2. **Accessing Variable Values:** To access the value of a declared variable within its own module (the root module in this case), HCL uses the var. prefix followed by the variable's name. Thus, var.num_servers refers to the specific value assigned to or defaulted for the num_servers variable in the root module.
3. **Calling a Child Module:** When the root module instantiates a child module, it does so using a module block. Inside this block, you assign values to the child module's declared input variables. The child module would likely have an input variable defined as variable "servers" .
4. **Passing the Value:** To pass the value of the root module's num_servers to the child module's servers input, you assign var.num_servers to the child module's input:

module "my_child_module" source = "./modules/my_child_module" # Path to your child module servers = var.num_ser

1. Here, servers is the input variable expected by the child module, and var.num_servers is the expression that retrieves the value from the root module's num_servers variable.
2. **Why Other Options Are Incorrect:**

A. servers = var(num_servers): This syntax is incorrect. var is not a function that takes a variable name as an argument; it's a namespace prefix.
C. servers = num_servers: This is incorrect because num_servers by itself, without the var. prefix, is not a valid way to reference the value of a declared variable in HCL. It would only work if num_servers were a local value (referenced as local.num_servers) or an output from another module (e.g., module.another.num_servers), which is not the context here.
D. $ var.num_servers : This represents string interpolation. While it might implicitly convert the number to a string and pass it, it's generally not the correct or idiomatic way to pass a direct value to an input that expects a number (or a type other than string). String interpolation is primarily used when you want to embed a variable's value within a larger string, e.g., name = "server-$ var.num_servers ". For direct value assignment, servers = var.num_servers is precise and type-aware.

This method ensures clear, maintainable, and type-safe configuration in cloud environments, promoting modularity and reusability crucial for managing complex infrastructure on platforms like AWS, Azure, or Google Cloud, and by extension, specialized solutions like Symantec's Edge SWG R2 if they adhere to similar

configuration paradigms.

Authoritative Links for Further Research:

Terraform - Input Variables: https://developer.hashicorp.com/terraform/language/values/variables
Terraform - Modules: https://developer.hashicorp.com/terraform/language/modules
Terraform - Expressions: https://developer.hashicorp.com/terraform/language/expressions/references

Question: 295 CertyIQ

What task does the terraform import command perform?

A. Imports resources from one Terraform state file to another.
B. Imports existing resources into Terraform's state file.
C. Imports a new Terraform module into Terraform's state file.
D. Imports all infrastructure from the configured cloud provider.
E. Imports provider configuration from one state file to another.

Answer: B

Explanation:

The terraform import command is a crucial utility within Terraform, designed to bring existing, manually created, or externally managed infrastructure resources under Terraform's declarative management. The correct answer, **B. Imports existing resources into Terraform's state file,** accurately describes its primary function.

When you use terraform import, you instruct Terraform to locate an already provisioned resource in your cloud provider (e.g., an AWS EC2 instance, an Azure Virtual Machine, or a Google Cloud Storage bucket) and record its current configuration and state within Terraform's tfstate file. This process is typically a one-time operation for a given resource. To perform an import, you first define a corresponding resource block in your Terraform configuration files (.tf) that matches the type of the existing resource. Then, you execute terraform import [resource_address] [resource_id], providing the address of your defined resource in the configuration and the actual ID of the existing resource from the cloud provider.

This command is invaluable for "brownfield" deployments or when transitioning an existing infrastructure to an Infrastructure as Code (IaC) methodology without needing to recreate everything from scratch. It allows organizations to adopt Terraform incrementally, capturing the current state of critical infrastructure components that were not initially provisioned by Terraform. After a successful import, Terraform considers the resource "managed" and will track its state and reconcile it with the desired state defined in your HCL configuration during subsequent terraform plan and terraform apply operations. The terraform.tfstate file then acts as the single source of truth for that resource's management.

Let's briefly look at why the other options are incorrect:

A describes state file manipulation, which is distinct from importing external resources.
C misrepresents how modules are handled; modules define resources, which are then managed in the state.
D overstates the command's capability; terraform import requires specific resource identification, not a bulk import of an entire cloud environment.
E incorrectly suggests transferring provider configurations via import; these are defined in your .tf files.

In essence, terraform import bridges the gap between pre-existing cloud resources and Terraform's desired state management, enabling a smoother adoption of IaC practices for established environments.

For further research:

Terraform import documentation: https://developer.hashicorp.com/terraform/cli/commands/import
Terraform State documentation: https://developer.hashicorp.com/terraform/language/state

Question: 296 CertyIQ

What is one disadvantage of using dynamic blocks in Terraform?

A. Dynamic blocks can construct repeatable nested blocks.
B. They cannot be used to loop through a list of values.
C. They make configuration harder to read and understand.
D. Terraform will run more slowly.

Answer: C

Explanation:

Dynamic blocks in Terraform are a powerful feature designed to construct repeatable nested configuration blocks when the number or content of those blocks is not known until runtime, or when you need to define multiple similar blocks based on a collection of values. While incredibly useful for flexibility and reducing boilerplate, their syntax and structure can undeniably make configurations harder to read and understand, which is a significant disadvantage.

The dynamic block syntax, utilizing for_each and an iterator variable within a content block, requires a mental parsing step to visualize the resulting static configuration. Unlike explicitly defined blocks where the final structure is immediately apparent, dynamic blocks introduce an abstraction layer. A reader must understand the input collection (for_each), how the iterator maps to values, and then interpret how these values are used within the content to form the actual nested block.

This increased cognitive load becomes particularly pronounced in complex scenarios, such as when dynamic blocks are deeply nested or when the logic within the content block is intricate. For teams collaborating on Infrastructure as Code (IaC), readability is paramount for maintainability, debugging, and onboarding new members. Configurations that are difficult to parse can lead to errors, slower troubleshooting, and increased development time because engineers spend more effort deciphering intent rather than building.

Furthermore, while dynamic blocks promote DRY (Don't Repeat Yourself) principles, over-reliance or overly complex implementations can inadvertently obscure the declarative nature of Terraform, making the "what" less obvious than the "how." Debugging issues related to dynamically generated blocks can also be more challenging, as errors might point to the dynamic block definition rather than a specific instance of a generated resource. Therefore, despite their utility, the trade-off in configuration clarity is a valid and often experienced drawback.

Regarding the other options:

A. Dynamic blocks can construct repeatable nested blocks. This is the primary purpose and advantage of dynamic blocks, not a disadvantage.
B. They cannot be used to loop through a list of values. This is incorrect. Dynamic blocks are explicitly designed to loop through lists or maps using the for_each argument to generate multiple instances of a nested block.
D. Terraform will run more slowly. While Terraform must evaluate dynamic blocks during the plan phase, the performance impact is generally negligible compared to the time spent on API calls to cloud providers or actual resource provisioning. It's not considered a primary disadvantage in the context of operational speed or

efficiency compared to the cognitive burden they can impose.

For further research:

Terraform Documentation - Dynamic Blocks: https://developer.hashicorp.com/terraform/language/expressions/dynamic-blocks
HashiCorp Learn - Understanding Dynamic Blocks: https://developer.hashicorp.com/terraform/tutorials/configuration-language/dynamic-blocks

Question: 297 CertyIQ

When you include a module block in your configuration that references a module from the Terraform Registry, the "version" attribute is required.

A. True
B. False

Answer: B

Explanation:

The statement is False. While the version attribute is highly recommended and considered a best practice when referencing modules from the Terraform Registry, it is not strictly required for the Terraform configuration to be valid or for terraform init to function.

If the version attribute is omitted from a module block, Terraform will default to using the latest available version of that module from the registry when terraform init is first executed or when terraform init -upgrade is run. This behavior can lead to unpredictable deployments because the "latest" version might change over time as module authors publish updates. Subsequent terraform init runs (without -upgrade) will typically use the cached version unless explicitly told to update.

However, relying on the latest implicit version introduces significant risks in cloud environments. It undermines the principle of infrastructure as code by making deployments non-reproducible. Different runs of terraform apply could potentially utilize different module versions if new releases occur between deployments, leading to inconsistencies, unexpected changes in behavior, or even breaking changes without explicit user intent. This unpredictability can cause instability, security vulnerabilities, or operational overhead when troubleshooting.

Therefore, although technically optional, specifying a version attribute — often using a version constraint (e.g., ~> 1.0 or ^2.1.0) — is crucial for ensuring deterministic and stable infrastructure deployments. It allows for controlled updates (e.g., compatible bug fixes) while preventing unintended major version upgrades that could introduce breaking changes. Explicit versioning is a cornerstone of robust dependency management in modern cloud infrastructure provisioning.

For further research:

Terraform Documentation on Module Versions: https://developer.hashicorp.com/terraform/language/modules/syntax#module-versions
Terraform terraform init command: https://developer.hashicorp.com/terraform/cli/commands/init

Question: 298 CertyIQ

Terraform variables and outputs that set the description argument will store that description in the state file.

A. True
B. False

Answer: B

Explanation:

The statement is **False**.

The Terraform state file (terraform.tfstate) is meticulously designed to store the actual state of the infrastructure managed by Terraform. Its core purpose is to map real-world cloud resources to your configuration, tracking their attributes, dependencies, and current values to reconcile with the desired state defined in your HCL files.

For input variables, the description argument within a variable block serves purely as documentation. It explains to a user what kind of input is expected, particularly when Terraform prompts for variable values during CLI execution (e.g., terraform apply or terraform plan). While the value provided for a variable (whether a default or user-provided override) might implicitly be part of the state's understanding of resource attributes, the description itself is not.

Similarly, for output values, the description argument in an output block also functions solely as documentation. It clarifies what the exposed output value represents when displayed to the user after an apply operation or queried via terraform output. The state file will store the computed value of the output, but it does not store its associated descriptive text.

These description arguments are considered configuration metadata. They exist only within the HCL configuration files (.tf files) and are primarily used by Terraform's CLI for enhanced user experience and clarity. They are not attributes of the provisioned infrastructure, nor do they represent a dynamic state that Terraform needs to track for lifecycle management. Storing such static descriptive text in the state file would introduce unnecessary redundancy and bloat, as the state file is optimized to track runtime attributes and values of managed resources.

For further research:

Terraform State: https://developer.hashicorp.com/terraform/language/state
Input Variables: https://developer.hashicorp.com/terraform/language/values/variables
Output Values: https://developer.hashicorp.com/terraform/language/values/outputs

Question: 299 CertyIQ

What does this code do?
terraform required_providers aws = “~> 3.0”)

A. Requires any version of the AWS provider >= 3.0 and < 4.0.
B. Requires any version of the AWS provider >= 3.0.
C. Requires any version of the AWS provider after the 3.0 major release, like 4.1.
D. Requires any version of the AWS provider > 3.0.

Answer: A

Explanation:

The code snippet required_providers aws = "~> 3.0" within a Terraform configuration specifies a version constraint for the AWS provider. The ~> operator, known as the "optimistic locking" or "approximate version" operator, is designed to allow minor and patch updates while preventing potentially breaking major version changes.

When applied as ~> X.Y, this operator translates to version >= X.Y and version < X.(Y+1). In the context of ~> 3.0, it precisely means that any version of the AWS provider greater than or equal to 3.0 is acceptable, but it must be strictly less than 4.0. This range includes 3.0.0, 3.5.0, 3.99.0, but explicitly excludes 4.0.0 or any subsequent major versions.

This constraint is crucial for maintaining stability in infrastructure as code deployments. Major version increments (e.g., from 3.x to 4.x) often introduce breaking changes, such as renamed resources, altered attribute behaviors, or removal of deprecated functionalities. By using ~> 3.0, a developer ensures that their configuration will continue to work with newer patch and minor releases of the 3.x series, which are typically backward-compatible, without inadvertently pulling in a 4.x version that could break the deployment.

Therefore, option A, "Requires any version of the AWS provider >= 3.0 and < 4.0," accurately describes the behavior of the ~> 3.0 version constraint. Option B is incorrect because it lacks the crucial upper bound, allowing 4.x, 5.x, etc. Option C is entirely wrong as it suggests versions after the 3.0 major release, which is the opposite of the constraint's intent. Option D is incorrect because it excludes version 3.0 itself, which the ~> operator includes as its lower bound. This precise control over provider versions is a fundamental aspect of robust cloud infrastructure management with Terraform.

Authoritative Links for Further Research:

1. **HashiCorp Terraform Documentation - Provider Requirements:**https://developer.hashicorp.com/terraform/language/providers/requirements#version-constraints
2. **HashiCorp Terraform Documentation - Version Constraints Syntax:**https://developer.hashicorp.com/terraform/language/expressions/version-constraints

Question: 300 CertyIQ

When developing a Terraform module, how would you specify the version when publishing it to the official Terraform Registry?

A. Configure it in the module’s Terraform code.
B. Tag a release in your module's source control repository.
C. Mention it on the module's configuration page on the Terraform Registry.
D. The Terraform Registry does not support versioning modules.

Answer: B

Explanation:

When publishing a Terraform module to the official Terraform Registry, the module's version is primarily specified by **tagging a release in its source control repository**. The Terraform Registry integrates directly with Git-based repositories, such as GitHub or GitLab, to manage modules.

Here's a detailed justification:

1. The Terraform Registry monitors a specified Git repository to discover and catalog new module versions.

2. Crucially, it relies on Git tags pushed to the module's repository as the definitive source for version information.
3. These tags must adhere to semantic versioning principles, typically formatted as vX.Y.Z (e.g., v1.0.0) or simply X.Y.Z.
4. When a valid Git tag is created and pushed to the repository's default branch (e.g., main), the Terraform Registry automatically detects it as a new module version.
5. This mechanism establishes the source control repository as the authoritative source for both the module's code and its release history.
6. Tagging a release ensures that each published module version corresponds to an immutable snapshot of the code at a specific commit.
7. This immutability is fundamental for reliable Infrastructure as Code (IaC), guaranteeing that a module version, once published, will consistently refer to the exact same codebase, preventing unexpected changes or regressions.
8. It provides robust traceability, allowing module consumers to easily inspect the precise code corresponding to any specific version listed on the registry.
9. This approach aligns with industry best practices for software versioning and release management.
10. It also facilitates automated Continuous Integration/Continuous Delivery (CI/CD) pipelines, where tagging a commit can trigger an automated release of a new module version.
11. Such automation streamlines the publishing workflow, reduces manual effort, and minimizes potential errors.
12. Configuring the version within the module's Terraform code (option A) is incorrect; while Terraform code specifies provider versions, it does not define the module's own published version on the registry.
13. Similarly, mentioning the version on the module's configuration page (option C) would be insufficient for a robust, verifiable versioning system.
14. Option D is incorrect; the Terraform Registry absolutely supports and relies heavily on module versioning to provide stability and choice to users.
15. Therefore, using Git tags for versioning ensures consistency, auditability, and a clear, verifiable chain of custody for all module releases on the Terraform Registry, which are critical aspects of managing cloud infrastructure efficiently.

Authoritative Links for Further Research:

Publishing Modules to the Terraform Registry:https://developer.hashicorp.com/terraform/registry/modules/publish
Terraform Module Versioning:https://developer.hashicorp.com/terraform/registry/modules/publish#versioning
Semantic Versioning Specification (SemVer):https://semver.org/

Question: 301 CertyIQ

terraform destroy is the only way to remove infrastructure.

A. True
B. False

Answer: B

Explanation:

The statement "terraform destroy is the only way to remove infrastructure" is false. While terraform destroy is the primary and most robust command for deprovisioning infrastructure managed by a Terraform state file,

several other methods exist for removing cloud resources.

Firstly, infrastructure can always be removed manually directly through the respective cloud provider's management console (GUI), command-line interface (CLI), or software development kits (SDKs). For example, an EC2 instance can be terminated via the AWS console, or a resource group deleted in Azure using the Azure CLI. These methods bypass Terraform entirely.

Secondly, Terraform itself offers commands to manipulate its state file without directly destroying infrastructure in the cloud. terraform state rm can remove a resource's entry from the state file, meaning Terraform will no longer track or manage it, but the resource will persist in the cloud. Similarly, terraform state mv moves resources within the state, but doesn't delete the underlying infrastructure.

Furthermore, cloud providers have their own lifecycle management features that can lead to infrastructure removal. Examples include S3 bucket lifecycle policies that automatically delete objects after a certain period, or auto-scaling groups that terminate instances based on demand. These actions occur independently of Terraform.

Other Infrastructure as Code (IaC) tools, such as AWS CloudFormation, Azure Resource Manager (ARM) templates, or Pulumi, also manage and remove infrastructure. If resources were initially provisioned with a different IaC tool, they would be removed using that tool's specific deprovisioning commands, not terraform destroy.

Finally, even within Terraform, specific changes to configurations that remove a resource block can, upon terraform apply, lead to the destruction of that resource, although terraform apply -destroy or terraform destroy is more explicit. Therefore, while terraform destroy is the intended and safest method for removing Terraform-managed infrastructure, it is far from the only way to remove cloud resources.

For further research:

Terraform Documentation on terraform destroy: https://www.terraform.io/docs/cli/commands/destroy.html
Terraform Documentation on State Management: https://www.terraform.io/docs/language/state/index.html
AWS CLI Documentation (example of manual removal): https://awscli.amazonaws.com/v2/documentation/api/latest/index.html

Question: 302 CertyIQ

Infrastructure as code (IaC) can be stored in a version control system along with application code.

A. True
B. False

Answer: A

Explanation:

Storing Infrastructure as Code (IaC) in a version control system alongside application code is not only possible but a fundamental best practice in modern software development and cloud operations. IaC artifacts, such as Terraform configurations, AWS CloudFormation templates, Azure Resource Manager templates, or Kubernetes manifests, are text-based files that declaratively define the desired state of infrastructure resources. Treating these configuration files as code allows them to leverage the same robust benefits that application source code enjoys from version control systems like Git.

Firstly, version control provides a complete, auditable history of all infrastructure changes, including who made them, when, and why, enabling comprehensive traceability and accountability. This historical record is

crucial for troubleshooting issues, understanding environment evolution, and ensuring compliance with regulatory requirements. Secondly, it facilitates collaborative development, allowing multiple team members to work on infrastructure definitions concurrently, with robust mechanisms for merging changes and resolving conflicts. Should an infrastructure deployment introduce an undesirable state, version control enables quick rollbacks to previous, stable configurations, minimizing downtime and risk. The ability to branch, merge, and tag specific infrastructure versions ensures environment reproducibility, making it easier to provision identical development, testing, and production environments consistently.

Storing IaC alongside application code in the same repository or a closely linked set of repositories fosters a unified development workflow, breaking down silos between development and operations teams. This integration aligns perfectly with DevOps principles, promoting collaboration, automation, and continuous delivery across the entire software lifecycle. Changes to application code might necessitate corresponding infrastructure modifications (e.g., new database, message queue), and having both definitions versioned together streamlines the entire deployment pipeline. This approach is a cornerstone of GitOps, where Git acts as the single source of truth for both declarative infrastructure and applications, driving automated deployments and environment management. By automating infrastructure provisioning and management through IaC and version control, organizations achieve greater speed, consistency, and reliability in their cloud environments. This practice underpins the concept of immutable infrastructure, where changes are made by deploying new, version-controlled infrastructure rather than modifying existing components.

Authoritative Links for Further Research:

AWS on Infrastructure as Code: https://aws.amazon.com/devops/what-is-iac/
Microsoft Azure on Infrastructure as Code: https://learn.microsoft.com/en-us/devops/deliver/what-is-infrastructure-as-code
CNCF on GitOps: https://www.cncf.io/blog/2020/03/10/what-is-gitops-and-how-does-it-work/

Question: 303 CertyIQ

What does the default local Terraform backend store?

A. Provider plugins
B. tfplan files
C. Terraform binary
D. State file

Answer: D

Explanation:

The default local Terraform backend stores the **state file**.

This file, typically named terraform.tfstate by default, is a critical component of Terraform's operation. It acts as a canonical record of the infrastructure Terraform manages, mapping the real-world cloud resources to the configuration defined in your .tf files. This state file is essential for Terraform to understand the current state of your infrastructure. Without it, Terraform wouldn't know which resources it previously created, their attributes, or how to update or destroy them.

When no specific backend configuration is defined in your Terraform code, Terraform automatically defaults to the "local" backend. This means the terraform.tfstate file is created and stored directly in the root directory of your Terraform working directory. It contains a snapshot of the infrastructure's metadata, resource IDs, and attributes, allowing Terraform to perform operations like planning, applying, and destroying resources effectively.

The local backend is simple and suitable for individual developers working on isolated projects. However, it lacks features crucial for team collaboration or CI/CD pipelines, such as state locking, versioning, and shared access. For these advanced scenarios, remote backends like Amazon S3, Azure Blob Storage, Google Cloud Storage, or HashiCorp Terraform Cloud are utilized to store the state file centrally and securely.

The other options are incorrect:

Provider plugins are downloaded by Terraform into the .terraform directory or a global cache, not stored by the backend itself as the primary purpose of a state backend.
tfplan files (terraform.tfplan) are temporary binary files generated by terraform plan to preview changes; they are not the persistent state.
The **Terraform binary** is the executable program itself, separate from any backend's storage function.

Therefore, the fundamental purpose of the default local Terraform backend is to manage and persist the all-important state file.

Authoritative Links for Further Research:

Terraform State Documentation: https://developer.hashicorp.com/terraform/language/state
Terraform Backends Documentation: https://developer.hashicorp.com/terraform/language/settings/backends/configuration
Terraform Local Backend: https://developer.hashicorp.com/terraform/language/settings/backends/local

Question: 304 CertyIQ

What command can you run to generate DOT (Document Template) formatted data to visualize Terraform dependencies?

A. terraform graph
B. terraform refresh
C. terraform output
D. terraform show

Answer: A

Explanation:

The correct command to generate DOT (Document Template) formatted data for visualizing Terraform dependencies is **A. terraform graph**.

Detailed Justification:

terraform graph is purpose-built within the Terraform CLI to expose the internal dependency graph that Terraform constructs before performing any operations. When you run this command, it analyzes your Terraform configuration files (e.g., .tf files) and identifies all declared resources, data sources, and modules, along with their explicit and implicit relationships. The output of terraform graph is a textual representation of this graph in the Graphviz DOT language.

The DOT language is a plain text graph description language widely used for representing structural information like networks, hierarchies, and dependencies. This standardized format allows the output from terraform graph to be easily consumed by external visualization tools, most notably Graphviz, which can then render these textual descriptions into visual diagrams such as PNG, SVG, or PDF images.

Visualizing Terraform dependencies is crucial for managing complex cloud infrastructure defined through

Infrastructure as Code (IaC). It provides a clear overview of how different components of your cloud environment, such as virtual machines, databases, networking components, and storage, are interconnected and in what order they would be provisioned, updated, or destroyed. This helps engineers understand the impact of changes, identify potential circular dependencies, optimize resource allocation, and debug issues related to resource provisioning order. It's an invaluable tool for enhancing clarity and maintainability in large-scale cloud deployments.

In contrast, the other options serve different purposes:

B. terraform refresh updates the Terraform state file by querying the real-world infrastructure to detect any drift, without generating a dependency graph.
C. terraform output displays the values of output variables defined in your configuration from the current state, not the architectural dependencies.
D. terraform show displays the human-readable plan or state file contents, which can include resource attributes but does not generate a graphical dependency representation in DOT format.

Therefore, terraform graph is the only command specifically designed to generate the DOT-formatted data required for visualizing Terraform's dependency tree, which is essential for effective cloud resource orchestration and management.

Authoritative Links for Further Research:

Terraform CLI: terraform graph command documentation: https://developer.hashicorp.com/terraform/cli/commands/graph
Graphviz Official Website: https://graphviz.org/
Graphviz DOT Language Guide: https://graphviz.org/doc/info/lang.html

Question: 305 CertyIQ

A resource block is shown in the Exhibit space of this page. What is the Terraform resource name of that resource block?

```
resource "azurerm_resource_group" "dev" {
  name = "test"
  location = "westus"
}
```

A. azurerm
B. azurerm_resource_group
C. test
D. dev

Answer: D

Explanation:

D. dev

resource "azurerm_resource_group" "dev"

name = "test"

So:

azurerm_resource_group → resource type

dev → Terraform resource name

test → actual Azure resource name

Question: 306 CertyIQ

You have a simple Terraform configuration containing one virtual machine (VM) in a cloud provider. You run terraform apply and the VM is created successfully.
What will happen if you delete the VM using the cloud provider console, then run terraform apply again without changing any Terraform code?

A. Terraform will recreate the VM.
B. Terraform will report an error.
C. Terraform will remove the VM from the state file.
D. Terraform will not make any changes.

Answer: A

Explanation:

When you run terraform apply, Terraform's primary goal is to bring the real-world infrastructure into alignment with the desired state defined in your configuration files. It achieves this by comparing three key elements: your Terraform code (the desired state), the current actual state of resources in the cloud provider, and the information stored in its state file (terraform.tfstate).

Initially, after the first successful terraform apply, the virtual machine (VM) is created in the cloud provider, and its details (such as its ID and properties) are meticulously recorded within the Terraform state file. This state file acts as a cache of the infrastructure Terraform manages, linking your configuration to the real-world cloud resources.

If you subsequently delete the VM directly using the cloud provider's console, you're introducing "configuration drift." The VM no longer exists in the cloud, but the Terraform state file remains unaware of this manual change; it still holds the record that the VM should be present and managed by Terraform. Your Terraform configuration code also still defines that the VM should exist.

When you run terraform apply again without modifying the Terraform code, Terraform performs its reconciliation process. It first reads your configuration, then consults its state file to understand the resources it believes it manages. Crucially, it then queries the actual cloud provider to verify the current status of those resources.

During this comparison, Terraform will identify a discrepancy: your configuration dictates that a VM should exist, the state file indicates it was previously created and managed, but the cloud provider reports that the VM is missing. To resolve this drift and restore the infrastructure to the desired state defined by your code and reflected in its state file, Terraform will generate an execution plan to recreate the VM. It considers the absence of the VM as a deviation from the managed state and takes action to correct it. This behavior is fundamental to Terraform's idempotent nature and its ability to ensure infrastructure consistency.

For further research, consider these authoritative links:

Terraform State: https://developer.hashicorp.com/terraform/language/state
Terraform Apply: https://developer.hashicorp.com/terraform/cli/commands/apply
Configuration Drift: https://developer.hashicorp.com/terraform/tutorials/state/drift

Question: 307 CertyIQ

You want to use API tokens and other secrets within your team's Terraform workspaces, where does HashiCorp recommend you store these sensitive values? (Choose three.)

A. In a plaintext document on a shared drive.
B. In HashiCorp Vault.
C. In a terraform.tfvars file, checked into your version control system.
D. In an HCP Terraform/Terraform Cloud variable, with the sensitive option checked.
E. In a terraform.tfvars file, securely managed and shared with your team.

Answer: B,D,E

Explanation:

Storing sensitive information like API tokens and secrets securely is paramount in infrastructure as code (IaC) to prevent unauthorized access and data breaches. HashiCorp recommends several secure methods for managing these values within Terraform workspaces.

B. In HashiCorp Vault. HashiCorp Vault is purpose-built for secret management, providing a centralized and highly secure solution. It encrypts secrets at rest and in transit, offers dynamic secrets (generating secrets on demand with a limited lifespan), and supports robust authentication and authorization mechanisms. Terraform can natively integrate with Vault to fetch secrets just-in-time during provisioning, ensuring that sensitive data is never hardcoded or exposed in configuration files. This approach adheres to the principle of least privilege and significantly reduces the attack surface.

Further Research: HashiCorp Vault Documentation: https://www.vaultproject.io/docs

D. In an HCP Terraform/Terraform Cloud variable, with the sensitive option checked.HCP Terraform (formerly Terraform Cloud) offers a managed environment for Terraform workflows, including variable management. When creating a variable in HCP Terraform and marking it as "sensitive," the value is encrypted at rest and never displayed in the UI, logs, or plan/apply outputs. This ensures that even collaborators with access to the workspace cannot directly view the secret, enhancing team security and compliance while still allowing Terraform to use the value for provisioning.

Further Research: HCP Terraform Variables: https://developer.hashicorp.com/terraform/cloud/workspaces/variables#sensitive-variables

E. In a terraform.tfvars file, securely managed and shared with your team. While less robust than Vault or sensitive HCP Terraform variables, using a .tfvars file for secrets can be acceptable if it is securely managed and never committed to version control systems (VCS). "Securely managed" implies storing it in an encrypted location, restricting access, and sharing it through secure, out-of-band methods (e.g., not email, not unencrypted shared drives). This method is often suitable for local development environments or specific, highly controlled scenarios where other secret management tools might be overkill, provided strict operational security policies are enforced to prevent secret sprawl.

Options A (plaintext on a shared drive) and C (terraform.tfvars file checked into VCS) are critical security anti-patterns. Plaintext storage is immediately vulnerable to anyone with access, and committing secrets to VCS creates a permanent record, making them nearly impossible to fully revoke and leading to widespread secret

exposure. Therefore, these choices are strongly discouraged.

Question: 308 CertyIQ

If one of your modules uses a local value, you can expose that value to callers of the module by defining a terraform output in the module's configuration.

A. True
B. False

Answer: A

Explanation:

Yes, it is entirely possible and a common practice in Terraform to expose a local value to callers of a module by defining a Terraform output in that module's configuration. This statement is **True**.

Local values in Terraform (locals) provide a way to assign a name to an expression, allowing it to be used multiple times within a module without repeating the expression. They improve readability, reduce redundancy, and encapsulate complex logic within the module itself, acting as computed constants or derived attributes internal to the module's scope.

Outputs, on the other hand, define the values that a module will export or return to its calling module or to the Terraform CLI. They serve as the public interface for a module, detailing what information it makes available to the outside world.

The mechanism to expose a local value is straightforward: you simply define an output block and set its value attribute to reference the desired local value. For instance, if you have a local value local.full_resource_name, you can create an output like output "resource_name" value = local.full_resource_name . This pattern is crucial for building robust and reusable modules in cloud infrastructure as code.

It enables modules to perform internal calculations or string manipulations, derive new attributes from inputs, or combine multiple pieces of information into a single, meaningful value, and then selectively expose these computed results. This maintains a clean separation of concerns, where the module's internal implementation details (local values) are hidden, while its useful derived outputs are made accessible. This principle aligns with good software engineering practices, promoting abstraction and modularity critical for managing complex cloud environments efficiently.

For example, a module might use local values to construct a unique resource identifier, a specific ARN, or a complex tag set based on various inputs. Exposing these derived values via outputs allows parent modules or other parts of the infrastructure to reference and depend on these dynamically generated attributes, facilitating interconnected resource deployments. This approach simplifies the caller's configuration by abstracting away the underlying computation.

For further research:

Terraform locals Documentation: https://developer.hashicorp.com/terraform/language/values/locals
Terraform outputs Documentation: https://developer.hashicorp.com/terraform/language/values/outputs

Question: 309 CertyIQ

What does state locking accomplish?

A. Encrypts any credentials stored within the state file.
B. Copies the state file from memory to disk.
C. Prevents accidental deletion of the state file.
D. Blocks Terraform commands from modifying the state file.

Answer: D

Explanation:

State locking is a critical feature in Terraform designed to maintain the integrity and consistency of the state file, especially in collaborative or automated environments. The Terraform state file acts as a crucial bridge, mapping the resources defined in your configuration to their real-world counterparts in the cloud or on-premises infrastructure. It tracks metadata about your infrastructure, resource dependencies, and actual attribute values.

When multiple users or automated processes attempt to run Terraform commands (like terraform apply or terraform plan -out=tfplan) that modify the state file concurrently, a race condition can occur. Without state locking, these simultaneous operations could lead to state file corruption, lost updates, or inconsistencies between the configuration and the deployed infrastructure. This would result in unexpected behavior, resource drift, or even infrastructure outages.

State locking prevents these issues by ensuring that only one operation can acquire an exclusive lock on the state file at any given time. When a Terraform command that modifies the state is executed, it first attempts to acquire this lock. If the lock is already held by another operation, the new command will wait until the lock is released or will fail, preventing concurrent write operations.

Therefore, state locking accomplishes blocking Terraform commands from modifying the state file if another operation is already in progress. This mechanism safeguards the state file's integrity and prevents accidental or malicious concurrent modifications. It is particularly vital in cloud computing environments where teams often share infrastructure codebases and collaborate on deploying and managing resources. By preventing concurrent writes, state locking ensures that the infrastructure as code remains consistent with the actual deployed infrastructure, supporting idempotency and reliable deployments.

To clarify why other options are incorrect:

A. Encrypts any credentials stored within the state file: State locking does not encrypt credentials; encryption is a separate security feature often handled by the backend itself or by secrets management solutions.
B. Copies the state file from memory to disk: This describes the basic process of saving state, not state locking, which is about concurrency control.
C. Prevents accidental deletion of the state file: While it helps prevent corruption which could necessitate a state rebuild, state locking doesn't directly prevent file deletion at the operating system level.

Authoritative Links for Further Research:

Terraform Documentation on State Locking: https://developer.hashicorp.com/terraform/language/state/locking
Terraform Documentation on Backends (where state locking is often implemented): https://developer.hashicorp.com/terraform/language/settings/backends/configuration

Question: 310 CertyIQ

Which of the following is allowed as a Terraform variable name?

A. source
B. name
C. version
D. count

Answer: B

Explanation:

B is correct because "name" is a plain Terraform identifier that does not collide with Terraform's meta-arguments or reserved block attributes.

Terraform distinguishes simple identifiers (valid for variable names) from special block attributes and meta-arguments; input variable identifiers follow the normal identifier rules and are free to be any non-reserved identifier, and "name" is not a documented meta-argument or module/provider attribute that would introduce syntactic or semantic collision. In contrast, Terraform defines several context-specific attributes and meta-arguments (for example, module "source"/"version" and resource meta-argument "count") that are treated specially by the language parser and runtime, and therefore are reserved in their respective syntactic contexts. From an exam-focused perspective, pick identifiers that are not listed as meta-arguments or special block attributes to avoid naming conflicts; "name" meets that criterion. Practically, using identifiers that shadow special attributes can lead to confusing code or parser errors and is explicitly discouraged by Terraform documentation and examples.

Critical evaluation of the incorrect options:

A. source — reserved as the module/provider source attribute (module block uses "source"), a syntactically significant attribute for locating modules.
C. version — used as a special attribute for modules/providers (versioning constraints) and therefore treated as a contextual attribute rather than a generic variable identifier.
D. count — a canonical Terraform meta-argument that controls resource/module instance counts and is not a plain identifier in resource/module contexts.

Referenceshttps://developer.hashicorp.com/terraform/language/values/variableshttps://developer.hashicorp.com/terraform/language/meta-arguments/counthttps://developer.hashicorp.com/terraform/language/modules/sources

Question: 311 CertyIQ

Which syntax check errors when you run terraform validate?

A. The code contains tabs for indentation instead of spaces.
B. There is a missing value for a variable.
C. The state file docs not match the current infrastructure.
D. None of the above.

Answer: D

Explanation:

D is correct because terraform validate only performs static HCL syntax and configuration consistency checks without requiring runtime variable values, consulting state, or enforcing whitespace style.

Reasoning (concise, exam-focused):

terraform validate performs parsing and static semantic checks of the configuration files in a directory; it does not contact providers, inspect remote state, or execute plans, so errors must be detectable purely from the HCL text and static type/structure analysis.
Indentation style is not a semantic error: HCL treats tabs and spaces as whitespace for structure and terraform fmt can normalize formatting; therefore tabs do not cause validate to fail.
Variable values are runtime inputs; absent user-supplied values are resolved at plan/apply time (or via defaults/CLI/env), so validate will not fail simply because a variable has no supplied value.
State/infrastructure divergence is an operational/runtime condition reflected in plan/apply or state commands; validate never compares configuration to the state or remote resources and thus cannot error for a mismatched state.

Critical evaluation of incorrect options:

A — Tabs for indentation: Incorrect. Whitespace (tabs vs spaces) is syntactically acceptable in HCL and formatting is cosmetic; formatting issues are addressed by terraform fmt, not terraform validate.
B — Missing variable value: Incorrect. Missing input values are resolved during plan/apply (or via provided defaults/vars); validate performs static checks and does not require runtime inputs to succeed.
C — State mismatch with infrastructure: Incorrect. State and real infrastructure are outside validate’s scope; detecting drift requires plan/apply or state interrogation, not validate.

References:https://developer.hashicorp.com/terraform/cli/commands/validatehttps://developer.hashicorp.com/terr

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!